About Software Change Management Standard - (SCMS)

About Software Change Management Standard - (SCMS)

Canonical Definition

Software Change Management Standard (SCMS) is the Parent Standard within SLGA™ that governs how proposed, required, and emerging changes to software are identified, evaluated, coordinated, authorized through applicable governance mechanisms, directed into the appropriate software lifecycle processes, verified against their intended outcomes, and formally closed or retained as unresolved obligations.

SCMS establishes software change as a governed transition between

identifiable software conditions rather than merely an edit to source code, a deployment request, or an operational modification.

It requires changes to remain associated with their origin, purpose,

affected software objects, current state, intended state, impact, dependencies, applicable authority, lifecycle requirements, implementation evidence, and resulting operational consequences.

The standard governs changes arising from defects, security

vulnerabilities, functional requirements, dependency updates, configuration modifications, operational incidents, infrastructure transitions, regulatory requirements, automated recommendations, and autonomous software agents.

SCMS recognizes that different changes require different lifecycle pathways and that a change cannot legitimately bypass necessary requirements, creation, dependency, configuration, quality, security, release, or deployment governance merely because it is urgent, automated, or technically executable.

Its principal governance output is the Software Change Governance Profile.

Operational Role

SCMS establishes the governance structure connecting a recognized need for

software change with the lifecycle processes required to evaluate, implement, verify, and close that change.

It governs:

• Software change recognition

• Change request identity

• Change origin and justification

• Change purpose and intended outcome

• Affected software objects

• Current and proposed software states

• Change classification

• Change impact assessment

• Change feasibility and dependencies

• Change scope and boundaries

• Change coordination

• Applicable authority and approval references

• Lifecycle re-entry

• Emergency change conditions

• Automated and autonomous changes

• Implementation traceability

• Change outcome verification

• Residual obligations

• Change closure and continuity

SCMS does not replace the specialized governance mechanisms responsible for software creation, quality verification, security, release authorization, or deployment execution.

It governs how a change moves through those mechanisms and whether the resulting change remains identifiable, justified, coordinated, and verifiably resolved.

Core Question

How can software changes remain identifiable, justified, coordinated, authorized, traceable, and verifiably resolved throughout the software lifecycle when change requirements, affected components, dependencies, operational conditions, implementation actors, and resulting software states may continuously evolve?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

SCMS occupies the ninth position in the SLGA™ architecture.

Its position does not mean that software changes occur only after deployment.

Software change may arise during any lifecycle stage.

SCMS governs the recognition, assessment, coordination, and resolution of

those changes and directs affected work back into the appropriate lifecycle processes.

Change Recognition → Change Assessment → Lifecycle Re-entry → Governed

Implementation → Outcome Verification → Change Closure

SCMS therefore provides a governed change-management mechanism across the

software lifecycle without replacing the individual Parent Standards.

• Governed Space

Software Change Management Governance

This space governs the formation, assessment, coordination, lifecycle

routing, implementation traceability, verification, and closure of software changes.

It begins when a potential or required change becomes identifiable.

It continues while the change is under assessment, approved for applicable lifecycle processing, implemented, verified, deferred, rejected, superseded, or maintained as an unresolved obligation.

The governed space includes changes initiated by humans, organizations,

automated systems, AI agents, external dependencies, security events, and operational conditions.

It ends for an individual change only when an appropriate closure or disposition state has been established and supported by evidence.

Governed Object

The primary Governed Object is:

A Software Change Case representing the identifiable and

evidence-supported relationship between a proposed or required software change, its originating condition, affected software objects, intended outcome, assessed impact, applicable governance requirements, lifecycle pathway, implementation references, resulting state, and closure decision.

The governed object must preserve, where applicable:

Change Case ID

Change Request ID

Software Product ID

Change Origin

Change Purpose

Affected Software Objects

Current State Reference

Intended State Reference

Change Classification

Impact Assessment Reference

Applicable Authority Reference

Lifecycle Re-entry Reference

Implementation References

Verification Evidence

Residual Conditions

Change Status

Version

Scope

SCMS governs software changes involving:

Functional Behavior

Software Requirements

Source Code

Software Architecture

Dependencies

Build and Configuration

Quality Conditions

Security Conditions

Software Releases

Deployment Conditions

Operational Software States

Interfaces and Integrations

Data Compatibility

Automation Logic

Autonomous Agent Behavior

Software Retirement Dependencies

Its applicability depends on the nature and consequences of the change

rather than the particular technology used.

Foundational Principle

A software change must remain governed by what it alters, why the alteration is necessary, which lifecycle mechanisms are affected, and whether the resulting state can be legitimately and verifiably established.

A small code modification may produce significant operational consequences.

A dependency update may require security, quality, configuration, and release reassessment.

An emergency correction may still require deployment authorization.

An autonomous agent may propose a useful modification without possessing authority to introduce it into production.

Change Implemented ≠ Change Governed.

Change Approved ≠ Every Lifecycle Requirement Satisfied.

Technical Modification ≠ Verified Software Improvement.

Software Change Identity

Every governed software change must have a stable identity.

Relevant elements include:

Change Case ID

Change Request ID

Software Product ID

Change Origin

Affected Object References

Change Owner or Responsible Function

Creation Time

Current Status

Change Version

A change may evolve through multiple assessments and implementation attempts without losing its historical identity.

Change Origin

Software changes may originate from:

New Requirements

Defect Reports

Security Findings

Operational Incidents

Dependency Changes

Configuration Changes

Performance Findings

Compatibility Requirements

Infrastructure Changes

Regulatory or Contractual Requirements

Automated Analysis

Autonomous Agent Recommendations

Retirement or Migration Requirements

The origin must remain identifiable and distinguishable from the actor proposing or implementing the change.

Change Purpose

Every change must have an identifiable purpose.

Examples include:

Correct a Defect

Introduce Functionality

Remove Functionality

Improve Performance

Address a Vulnerability

Restore Compatibility

Update a Dependency

Modify Configuration

Reduce Operational Risk

Support Migration

Restore Operational Capability

Prepare Software Retirement

A proposed technical modification without a sufficiently defined purpose

may be unsuitable for governance assessment.

Change Justification

Change justification explains why the proposed modification should be considered.

Relevant elements include:

Identified Problem or Opportunity

Supporting Evidence

Expected Benefit

Operational Necessity

Applicable Requirement

Urgency

Alternatives Considered

Consequences of No Change

Known Uncertainty

Justification must not be confused with authorization.

A well-supported need for change does not automatically authorize implementation.

Current Software State

SCMS requires a suitable reference to the software state before the

proposed change.

Relevant elements include:

Current Requirements

Current Software Artifacts

Current Dependencies

Current Configuration

Current Quality Conditions

Current Security Conditions

Current Release State

Current Deployment State

Known Defects or Restrictions

The necessary baseline depends on the scope and impact of the proposed

change.

Intended Software State

The intended state identifies what the change is expected to establish.

Relevant elements include:

Target Functionality

Target Software Behavior

Target Artifact or Configuration

Target Dependency Conditions

Target Quality Requirements

Target Security Conditions

Target Compatibility

Target Operational Outcome

Verification Criteria

The intended state must remain distinguishable from the state actually produced by implementation.

Change Scope

Change scope identifies the objects and conditions intended to be modified.

Relevant dimensions include:

Affected Components

Affected Interfaces

Affected Dependencies

Affected Configurations

Affected Environments

Affected Data Structures

Affected Operational Functions

Affected Users or Systems

Excluded Elements

Scope must be sufficiently defined to support impact assessment and lifecycle coordination.

Change Boundary

The change boundary identifies the permitted extent of the proposed

modification and the conditions requiring additional assessment.

Relevant boundaries include:

Software Object Boundary

Functional Boundary

Dependency Boundary

Configuration Boundary

Security Boundary

Environment Boundary

Data Boundary

Authority Boundary

Lifecycle Boundary

Operational Impact Boundary

A change request must not become an unrestricted permission to modify related software or infrastructure.

Change Classification

SCMS must distinguish changes according to their governance-relevant characteristics.

Potential classifications include:

Functional Change

Corrective Change

Security Change

Dependency Change

Configuration Change

Compatibility Change

Performance Change

Infrastructure-Related Software Change

Emergency Change

Migration Change

Retirement-Related Change

Automated Change

Autonomous Agent-Initiated Change

Multiple classifications may apply to one change.

Classification must not replace impact assessment.

Change Significance

Change significance concerns the potential consequences of the modification.

Relevant dimensions include:

Functional Impact

Operational Impact

Security Impact

Data Impact

Dependency Impact

Infrastructure Impact

User Impact

Reversibility

Verification Complexity

Recovery Consequences

A change with few modified code lines may still have high governance significance.

Change Impact Assessment

SCMS requires assessment of material consequences before a change proceeds

through applicable lifecycle mechanisms.

Impact assessment may include:

Affected Software Objects

Affected Requirements

Affected Dependencies

Affected Configurations

Affected Quality Conditions

Affected Security Conditions

Affected Releases

Affected Deployments

Affected Operational Environments

Affected Recovery Conditions

The assessment must preserve known limitations and unresolved impact questions.

Direct and Indirect Impact

Change assessment must distinguish direct modifications from indirect consequences.

Direct Impact

The software objects or conditions intentionally modified.

Indirect Impact

Other objects, functions, dependencies, environments, or operational

conditions affected through the modification.

A change affecting one component may create consequences across multiple services or lifecycle stages.

Limited Change Scope ≠ Limited Change Impact.

Dependency Impact

Software changes may alter dependency relationships.

Relevant considerations include:

Introduced Dependencies

Removed Dependencies

Dependency Version Changes

Transitive Dependencies

Compatibility Requirements

Dependency Availability

Security Exposure

Continuity Requirements

SCMS identifies the change-related impact.

SDS governs the underlying dependency mechanisms.

Configuration Impact

Changes may affect software configuration and build conditions.

Relevant elements include:

Configuration Baseline

Build Inputs

Artifact Identity

Environment Parameters

Runtime Configuration

Configuration Compatibility

Reproducibility Requirements

Configuration Restoration

SCMS coordinates applicable configuration governance through SCFS.

Quality Impact

Change assessment must identify relevant quality consequences.

These may include:

Functional Correctness

Reliability

Performance

Compatibility

Maintainability

Test Coverage

Regression Exposure

Known Defects

Quality Verification Requirements

SQS governs the applicable quality assessment and verification mechanisms.

Security Impact

Software changes may introduce, remove, or alter security conditions.

Relevant elements include:

Attack Surface

Access Permissions

Authentication Behavior

Authorization Logic

Data Exposure

Dependency Vulnerabilities

Security Configuration

Secret Handling

Security Testing Requirements

Remediation Obligations

SSS governs the applicable software security mechanisms.

Operational Impact

SCMS must assess the potential operational consequences of a change.

Relevant dimensions include:

Service Availability

Deployment Complexity

Resource Consumption

Data Integrity

Infrastructure Compatibility

Monitoring Requirements

Recovery Feasibility

User Disruption

Physical-System Behavior Where Applicable

Operational impact must not be inferred solely from the technical size of the modification.

Change Feasibility

Change feasibility concerns whether the proposed modification can reasonably be implemented and verified under applicable conditions.

Relevant factors include:

Technical Capability

Required Resources

Required Expertise

Dependency Availability

Infrastructure Constraints

Verification Capability

Time Constraints

Operational Restrictions

Recovery Capability

Authority Requirements

Feasibility does not independently establish legitimacy or desirability.

Change Alternatives

Where material consequences justify comparison, SCMS should preserve relevant alternatives.

These may include:

Implement Proposed Change

Implement a Different Solution

Restrict Existing Functionality

Apply a Temporary Mitigation

Defer Implementation

Reject the Change

Retire or Replace the Affected Software

Alternative assessment must consider the actual problem and applicable governance requirements.

Change Priority and Urgency

SCMS must distinguish change priority from change authorization.

Relevant factors include:

Operational Severity

Security Exposure

Business Criticality

User Impact

Dependency Deadline

Regulatory Requirement

Recovery Necessity

Implementation Risk

Urgency may affect coordination and permitted procedures but does not independently create authority.

Urgent Change ≠ Automatically Authorized Change.

Change Coordination

Software changes may require coordination among multiple functions.

Relevant elements include:

Affected Teams

Responsible Functions

Required Reviews

Dependency Coordination

Configuration Coordination

Quality Verification

Security Review

Release Planning

Deployment Planning

Operational Communication

Coordination requirements must reflect the change's scope and consequences.

Change Authority

SCMS must identify the applicable authority references governing change decisions.

Relevant elements include:

Change Decision Authority

Approval Scope

Decision Actor

Delegation Conditions

Applicable Restrictions

Validity

Required Escalation

Decision Evidence

Change approval must remain distinct from release authorization and deployment authorization.

Change Decision

SCMS recognizes the following principal decisions:

Accepted for Lifecycle Processing

Accepted With Conditions

Deferred

Rejected

Reassessment Required

Withdrawn

Acceptance for lifecycle processing does not mean that the proposed change has already satisfied all implementation, verification, release, or deployment requirements.

Conditional Change Acceptance

A change may be accepted subject to explicit conditions.

Examples include:

Additional Impact Assessment

Security Verification

Dependency Compatibility Evidence

Required Test Coverage

Independent Review

Restricted Implementation Scope

Additional Deployment Conditions

Recovery Planning

Conditions must remain traceable to the relevant lifecycle mechanism.

Lifecycle Re-entry

Software change may require returning to one or more earlier lifecycle governance stages.

Examples include:

Requirements Reassessment

Software Creation

Dependency Assessment

Configuration Revision

Quality Verification

Security Verification

Release Formation

Deployment Planning

Lifecycle re-entry must be determined by the nature and consequences of the change.

It must not be reduced to a fixed sequence applied identically to every modification.

Lifecycle Re-entry Principle

A software change must re-enter every lifecycle governance mechanism materially affected by the change, but it should not be forced through unrelated mechanisms without a justified governance need.

This principle preserves proportionality without weakening necessary

control.

A documentation correction may require limited lifecycle processing.

A dependency update may require configuration, quality, security, release, and deployment reassessment.

A modification to autonomous system behavior may require additional authority, safety, and operational verification.

Change Size ≠ Required Governance Depth.

Lifecycle Re-entry Determination

Re-entry determination must identify:

Affected Lifecycle Stages

Reason for Re-entry

Required Governance Activities

Applicable Standards

Existing Evidence Reusability

Evidence Requiring Renewal

Required Decisions

Completion Conditions

A previously completed lifecycle assessment may remain applicable where its assumptions and evidence remain valid.

Existing Evidence Reuse

SCMS permits reuse of existing evidence when it remains applicable.

Relevant considerations include:

Evidence Scope

Affected Software Object

Change Impact

Evidence Freshness

Unchanged Assumptions

Dependency Conditions

Verification Coverage

Known Limitations

Evidence reuse must be justified rather than assumed.

Previously Verified ≠ Still Verified After Material Change.

Change Implementation Reference

SCMS must preserve traceability to the mechanisms performing the change.

Relevant elements include:

Implementation Work Item

Affected Artifact

Implementation Actor

Development Reference

Configuration Reference

Dependency Reference

Verification Reference

Release Reference

Deployment Reference

SCMS coordinates implementation evidence without replacing the governance

of individual technical activities.

Change Implementation Boundaries

Implementation must remain within the assessed and legitimately permitted scope.

Material deviations may include:

Additional Component Changes

Unplanned Dependency Introduction

New Data Modification

Unexpected Infrastructure Action

Unapproved Configuration Change

Expanded Operational Scope

Changed Execution Actor

New Security Exposure

Material deviations require reassessment or additional authorization where applicable.

Automated Change Management

Automated systems may generate, classify, route, or implement software changes.

SCMS must preserve:

Automation Identity

Assigned Function

Change Origin

Decision Policy Reference

Permitted Actions

Affected Objects

Applicable Constraints

Verification Requirements

Escalation Conditions

Execution Evidence

Automation must not convert technical modification capability into unrestricted change authority.

Autonomous Agent-Initiated Change

Autonomous AI agents may identify defects, propose code changes, modify dependencies, or recommend operational corrections.

SCMS must distinguish:

Agent Recommendation

Change Request

Impact Assessment

Change Decision

Authorized Implementation

Verified Outcome

An agent's confidence in a proposed solution does not establish that the solution is correct, safe, authorized, or suitable for production.

Autonomous Execution Boundary

Where autonomous agents implement changes, SCMS must preserve:

Agent Identity

Assigned Objective

Permitted Software Objects

Permitted Tools

Data Access Boundaries

Infrastructure Access Boundaries

Required Reviews

Escalation Conditions

Human Override Requirements

Evidence

Task Authority ≠ Constituent Action Authority.

A software change objective does not establish permission for every tool call, repository modification, dependency installation, or deployment action used to achieve it.

Emergency Change Governance

Emergency changes may be necessary to address critical vulnerabilities, operational failures, or other urgent conditions.

SCMS must preserve:

Emergency Trigger

Urgency Justification

Applicable Emergency Authority

Permitted Scope

Mandatory Safeguards

Temporary Exceptions

Required Verification

Deployment Conditions

Post-Implementation Review

Emergency processing may accelerate applicable governance procedures without eliminating the need for legitimate authority and operational evidence.

Change Exception Governance

SCMS must identify exceptions to applicable change-management requirements.

Relevant elements include:

Exception ID

Affected Requirement

Reason

Competent Authority

Permitted Deviation

Conditions

Validity

Residual Obligations

Review Requirement

An exception must not be treated as legitimate merely because it appears in a change record.

Change Conflict

A change conflict exists when relevant requirements, decisions, implementation conditions, or evidence are materially inconsistent.

Examples include:

Conflicting Requirements

Incompatible Dependency Changes

Contradictory Security Conditions

Overlapping Changes

Incompatible Configuration Revisions

Competing Release Objectives

Conflicting Deployment Restrictions

Inconsistent Verification Results

Material conflicts must be identified and resolved or explicitly governed before the affected change can proceed.

Concurrent Changes

Multiple software changes may affect the same objects or operational conditions.

SCMS must identify:

Related Change Cases

Shared Components

Overlapping Dependencies

Conflicting Configuration Changes

Execution Order Dependencies

Combined Operational Impact

Coordination Requirements

Verification Implications

Individually acceptable changes may produce unacceptable consequences when combined.

Change Sequence and Dependency

Some changes require a defined implementation order.

Relevant elements include:

Prerequisite Changes

Dependent Changes

Compatibility Transitions

Migration Requirements

Release Dependencies

Deployment Dependencies

Recovery Dependencies

A change must not be treated as independently executable when its success depends on unresolved preceding changes.

Change Traceability

SCMS must preserve the relationship:

Change Origin → Change Purpose → Affected Objects → Impact Assessment →

Change Decision → Lifecycle Re-entry → Implementation → Verification → Resulting State → Closure

This relationship must remain reconstructable across relevant systems, teams, and automated execution paths.

Change Verification

Change verification evaluates whether the intended modification was

implemented and whether the required resulting conditions are supported by evidence.

Relevant elements include:

Change Objective

Implemented Modification

Affected Artifacts

Verification Criteria

Quality Evidence

Security Evidence

Configuration Evidence

Release Evidence

Deployment Evidence

Observed Result

Verification must reflect the actual scope and consequences of the change.

Intended–Actual Change Alignment

SCMS must distinguish the intended change from the modification actually implemented.

Relevant relationships include:

Requested Scope ↔ Implemented Scope

Intended Artifact ↔ Resulting Artifact

Expected Behavior ↔ Verified Behavior

Expected Dependencies ↔ Actual Dependencies

Expected Configuration ↔ Actual Configuration

Expected Operational Outcome ↔ Observed Outcome

Material misalignment must remain visible.

Change Outcome Assessment

A change may produce different outcomes.

Examples include:

Objective Achieved

Objective Partially Achieved

Objective Not Achieved

Unintended Consequences Identified

Verification Incomplete

Outcome Unknown

Additional Change Required

Outcome assessment must not be reduced to whether implementation tasks were marked complete.

Change Failure

Change failure may occur when:

Implementation Cannot Be Completed

Required Verification Fails

New Defects Are Introduced

Security Conditions Become Unacceptable

Dependencies Become Incompatible

Operational Conditions Deteriorate

Intended Outcome Is Not Achieved

Material Scope Violations Occur

A failed change must remain traceable and may require a new change case, reassessment, rollback, or recovery.

Change Reassessment

Reassessment may be required when:

Change Scope Expands

New Impact Evidence Emerges

Dependencies Change

Security Conditions Change

Implementation Deviates

Verification Fails

Operational Conditions Change

Authority Changes

Related Changes Introduce Conflicts

Recovery Becomes Necessary

Reassessment must identify which earlier conclusions remain valid.

Change Version Continuity

SCMS must preserve material revisions to a change case.

Relevant elements include:

Change Case Version

Previous Version

Changed Scope

Changed Purpose

Changed Impact

Changed Decision

Changed Lifecycle Pathway

Changed Verification Requirements

Revision Reason

Revision Evidence

A revised change case must not silently replace the version associated with an earlier decision.

Residual Change Obligations

A software change may leave unresolved obligations.

Examples include:

Deferred Verification

Temporary Configuration

Known Compatibility Limitation

Unresolved Security Condition

Additional Deployment Requirement

Outstanding Documentation

Required Monitoring

Further Migration Work

Future Retirement Dependency

Residual obligations must remain identifiable and assigned to an applicable governance mechanism.

Change Closure

Change closure requires an explicit disposition supported by evidence.

Relevant elements include:

Change Objective

Implementation Status

Verification Results

Resulting Software State

Release or Deployment References

Residual Obligations

Closure Decision

Closure Authority

Closure Time

Closure Evidence

Closure does not require that every proposed change be implemented.

A rejected, withdrawn, or superseded change may be legitimately closed with an appropriate disposition record.

Change Closure Conditions

A change may be closed when its disposition is sufficiently established.

Possible closure conditions include:

Implemented and Verified

Implemented With Accepted Residual Conditions

Rejected With Justification

Withdrawn

Superseded

Transferred to Another Governed Change Case

Resolved Through an Alternative Mechanism

A change must not be classified as successfully implemented merely because the administrative case is closed.

Change Governance Classification

SCMS recognizes the following classifications:

Change Identified

Request Pending

Under Assessment

Impact Assessment Incomplete

Evidence Insufficient

Conflict Identified

Accepted for Lifecycle Processing

Accepted With Conditions

Deferred

Rejected

Implementation Pending

Implementation In Progress

Verification Pending

Partially Verified

Implemented and Verified

Implementation Failed

Reassessment Required

Residual Obligations Outstanding

Withdrawn

Superseded

Closed

Unknown

Conflicted

Classification must remain associated with the relevant change case and supporting evidence.

Core Modules

SCMS is implemented through five Core Modules. 58.1 Change Request Management Module (CRMM)

Governed Space: Software Change Request Governance

CRMM governs the identification, formation, classification, maintenance,

and disposition of software change requests.

It establishes the identifiable relationship between a change origin,

purpose, affected software objects, requested modification, supporting evidence, and initial request status.

Principal Output: Software Change Request Profile

58.2 Change Impact Assessment Module (CIAM)

Governed Space: Software Change Impact Governance

CIAM governs the assessment of direct and indirect consequences associated

with proposed software changes.

It evaluates affected requirements, artifacts, dependencies,

configurations, quality, security, operational conditions, feasibility, uncertainty, and required lifecycle reassessment.

Principal Output: Software Change Impact Profile

58.3 Change Coordination Control Module (CHCM)

Governed Space: Software Change Coordination Governance

CHCM governs the coordination of software changes across affected actors,

software objects, dependencies, implementation activities, decision references, and lifecycle functions.

It preserves change relationships, sequencing, applicable coordination

conditions, and material conflicts.

Principal Output: Software Change Coordination Profile

58.4 Lifecycle Re-entry Control Module (LRECM)

Governed Space: Software Change Lifecycle Re-entry Governance

LRECM governs how a software change is directed into the appropriate SLGA™ lifecycle mechanisms according to its assessed impact and applicable requirements.

It identifies which lifecycle processes must be repeated, renewed, extended, or otherwise revisited and preserves evidence of their completion.

Principal Output: Software Lifecycle Re-entry Profile

58.5 Change Closure Management Module (CCLM)

Governed Space: Software Change Closure Governance

CCLM governs the verification, classification, disposition, and formal

closure of software change cases.

It establishes whether the intended change outcome was achieved, which

evidence supports the conclusion, and which residual obligations or unresolved conditions remain.

Principal Output: Software Change Closure Profile

Parent Standard Governance Flow

Change Origin → Change Recognition → Change Request → Change Purpose →

Affected Software Objects → Change Scope → Impact Assessment → Change Classification → Coordination → Applicable Authority → Change Decision → Lifecycle Re-entry → Governed Implementation → Verification → Outcome Assessment → Residual Obligations → Change Closure

Where material impact is unknown:

Change Request → Evidence Gap → Additional Assessment → Reclassification /

Deferral

Where a change requires additional lifecycle governance:

Change Impact → Affected Lifecycle Standards → Required Re-entry →

Applicable Governance Outputs → Implementation Continuation

Where implementation deviates materially:

Approved Change Scope → Actual Modification → Material Deviation →

Reassessment → Revised Decision / Corrective Action

Where the intended outcome is not achieved:

Implementation → Verification Failure → Change Reassessment → Additional

Implementation / Recovery / Rejection / Alternative Resolution

Governance Requirements

A conforming SCMS implementation must satisfy the following requirements. Change Identity and Formation

• Establish identifiable software change cases

• Preserve change origin and purpose

• Identify affected software objects

• Establish current and intended state references

• Define change scope and boundaries

• Preserve supporting evidence

• Identify change classification and significance

• Preserve material uncertainty

Impact and Feasibility

• Evaluate direct change impact

• Evaluate material indirect impact

• Identify affected dependencies and configurations

• Identify quality and security implications

• Evaluate operational consequences

• Evaluate change feasibility where applicable

• Identify material alternatives where justified

• Preserve impact evidence and limitations

Authority and Coordination

• Identify applicable change decision authority

• Preserve decision actor eligibility

• Identify affected coordination functions

• Identify related and concurrent changes

• Preserve implementation sequencing requirements

• Identify conflicts and restrictions

• Govern emergency change conditions

• Preserve automation and autonomous agent boundaries

Lifecycle Re-entry and Implementation

• Determine applicable lifecycle re-entry requirements

• Identify affected SLGA™ Parent Standards

• Preserve the basis for evidence reuse

• Identify evidence requiring renewal

• Preserve implementation references

• Identify material implementation deviations

• Require reassessment where applicable

• Preserve release and deployment governance boundaries

Verification and Closure

• Establish change verification requirements

• Compare intended and actual change outcomes

• Preserve verification evidence

• Identify change failure

• Preserve residual obligations

• Establish change outcome classification

• Preserve closure decisions

• Maintain historical change continuity

Governance Outputs

The principal output is:

Software Change Governance Profile

Supporting outputs include:

Software Change Request Profile

Software Change Impact Profile

Software Change Coordination Profile

Software Lifecycle Re-entry Profile

Software Change Closure Profile

Additional governance records may include:

Change Identity Record

Change Classification

Affected Software Object Register

Current–Intended State Comparison

Change Feasibility Assessment

Change Decision Reference

Change Authority Reference

Lifecycle Re-entry Register

Implementation Traceability Record

Change Verification Record

Residual Obligation Register

Change Version History

Change Closure Record

Governance Evidence

SCMS must preserve evidence sufficient to reconstruct material change decisions and outcomes.

Relevant evidence includes:

Original Change Request

Change Origin Evidence

Affected Object References

Impact Assessment

Authority and Decision Records

Coordination Records

Lifecycle Re-entry Decisions

Implementation Evidence

Quality and Security Verification

Release and Deployment References

Observed Outcome

Residual Obligations

Closure Decision

Evidence must remain associated with the change case and the conclusions it supports.

Governance Continuity

Software change governance must preserve continuity when:

Change Ownership Transfers

Implementation Actors Change

Automation Replaces Manual Execution

Change Scope Evolves

Lifecycle Re-entry Expands

Related Changes Are Merged or Separated

Implementation Is Deferred

Verification Is Repeated

Residual Obligations Continue

Continuity requires the change's identity, decision history, evidence, and

unresolved obligations to remain reconstructable.

Cross-Architecture and Lifecycle Boundary

SCMS coordinates change-related governance across SLGA™ without replacing

specialized Parent Standards.

SRS — Software Requirements Standard

Governs changed requirements, scope, and traceability.

SCS — Software Creation Standard

Governs creation and modification of software artifacts.

SDS — Software Dependency Standard

Governs dependency identity, suitability, introduction, and continuity.

SCFS — Software Configuration Standard

Governs configuration baselines, builds, artifact identities, and

configuration changes.

SQS — Software Quality Standard

Governs quality criteria, testing, defects, and quality assessment.

SSS — Software Security Standard

Governs security requirements, testing, vulnerabilities, and remediation.

SRLS — Software Release Standard

Governs release readiness, authorization, records, and withdrawal.

SDPS — Software Deployment Standard

Governs deployment planning, authorization, production version identity,

monitoring, and recovery.

SRTS — Software Retirement Standard

Governs retirement, migration, support termination, and residual

dependencies.

SLCS — Software Lifecycle Control Standard

Governs lifecycle state consistency, transitions, exceptions, control

transfer, and continuity.

SCMS also consumes applicable authority, accountability, autonomous

systems, and operational reality governance references from the wider OOF® methodology where relevant.

Practical Applications

Enterprise Software Development

Coordinating changes across development teams, software products, release

schedules, and production environments.

Continuous Integration and Delivery

Maintaining traceability between change requests, code modifications,

verification, releases, and deployments.

AI-Assisted Software Engineering

Ensuring AI-generated modifications remain associated with legitimate

change requests, assessed impacts, permitted actions, and verified outcomes.

Autonomous Software Agents

Preventing autonomous agents from treating an assigned objective as

unrestricted authority to modify software, dependencies, infrastructure, or production environments.

Cybersecurity Remediation

Coordinating urgent vulnerability corrections while preserving security

verification, release governance, and deployment authorization.

Critical Infrastructure

Ensuring software modifications affecting consequential operations receive

appropriate impact assessment, coordination, verification, and lifecycle control.

Robotics and Embedded Systems

Governing firmware, control logic, configuration, and behavioral changes

with potentially significant physical consequences.

Use Case 1 — A Small Dependency Update Creates a Major Operational

Change

A development team proposes updating a software library to address a minor

compatibility issue.

The modification requires only a few lines of code.

However, the new library version changes authentication behavior and

introduces an incompatible runtime dependency.

SCMS evaluates:

Change Request → Dependency Impact → Security Impact → Configuration

Impact → Quality Requirements → Lifecycle Re-entry

The assessment identifies:

• The technical modification is small

• The dependency impact is material

• Security behavior requires reassessment

• Configuration and quality verification are necessary

• The change requires additional lifecycle governance before release

Small Code Change ≠ Small Governance Impact.

Use Case 2 — An Autonomous Agent Fixes a Defect but Introduces an

Unauthorized Production Change

An autonomous software engineering agent receives an objective to correct a

reported application defect.

The agent modifies the source code and successfully passes local tests.

It then deploys the modified application directly into production using an

available tool.

SCMS evaluates:

Change Objective → Agent Authority → Modified Artifacts → Impact

Assessment → Lifecycle Re-entry → Release Governance → Deployment Authorization

The assessment identifies:

• The defect correction may be technically valid

• The agent's assigned objective does not establish unrestricted

• Required lifecycle verification may be incomplete

• Release and deployment authorization cannot be inferred from

• The production modification requires appropriate governance assessment

execution authority. successful local tests.

Defect Corrected ≠ Change Legitimately Introduced into Production.

Use Case 3 — Multiple Individually Acceptable Changes Create a Combined

Failure

Two teams prepare separate software changes.

One modifies a shared service interface.

The other updates a dependent application.

Both changes pass their individual tests.

When deployed together, their combined behavior produces an operational failure.

SCMS evaluates:

Change A → Change B → Shared Dependencies → Combined Impact → Coordination

→ Integration Verification

The assessment identifies:

• Each change has an identifiable purpose

• Individual verification does not establish combined compatibility

• The changes affect a shared operational dependency

• Coordination and integration verification are required

• The resulting failure creates reassessment and corrective-action

obligations.

Individually Verified Changes ≠ Combined Operational Compatibility.

Critical Distinctions

Change Implemented ≠ Change Governed.

Change Approved ≠ Every Lifecycle Requirement Satisfied.

Technical Modification ≠ Verified Software Improvement.

Change Request ≠ Change Authorization.

Change Justification ≠ Implementation Permission.

Limited Change Scope ≠ Limited Change Impact.

Change Size ≠ Required Governance Depth.

Urgent Change ≠ Automatically Authorized Change.

Previously Verified ≠ Still Verified After Material Change.

Agent Recommendation ≠ Authorized Software Change.

Task Authority ≠ Constituent Action Authority.

Local Test Success ≠ Production Deployment Authorization.

Implementation Complete ≠ Intended Outcome Verified.

Administrative Closure ≠ Successful Change Implementation.

Individually Verified Changes ≠ Combined Operational Compatibility.

Software Change Management ≠ Replacement of Specialized Lifecycle

Governance.

Canonical Boundary

SCMS governs the identification, assessment, coordination, lifecycle

routing, implementation traceability, verification, and closure of software changes across the SLGA™ software lifecycle.

It establishes the relationship between change origin, purpose, affected

software objects, impact, applicable authority, required lifecycle re-entry, implementation evidence, resulting state, and closure.

SCMS does not independently replace software requirements governance, software creation, dependency governance, configuration control, quality verification, security assessment, release authorization, deployment governance, or software retirement.

Its five Core Modules are:

Change Request Management Module (CRMM)

Change Impact Assessment Module (CIAM)

Change Coordination Control Module (CHCM)

Lifecycle Re-entry Control Module (LRECM)

Change Closure Management Module (CCLM)

Its principal governance output is the:

Software Change Governance Profile

Canonical Closing Statement

Software change is not governed merely because a modification was requested, approved, implemented, or successfully deployed.

A minor code edit may introduce significant dependency consequences.

A security correction may create new compatibility problems.

Two independently verified modifications may become incompatible when combined.

An automated pipeline may implement a change without establishing that every affected lifecycle requirement has been satisfied.

An autonomous agent may solve the assigned technical problem while exceeding its authority or introducing unverified operational consequences.

And a completed change may leave unresolved obligations that remain relevant long after implementation.

SCMS establishes the governance structure required to identify and control

these conditions.

It requires software changes to remain associated with their origin,

purpose, affected objects, assessed consequences, legitimate decisions, implementation pathways, and verified outcomes.

It directs changes into the lifecycle mechanisms that are materially affected without unnecessarily repeating unrelated governance activities.

It preserves the distinction between change approval and implementation

authorization, between technical modification and operational improvement, and between implementation completion and evidence-supported closure.

It ensures that emergency changes, automated modifications, and autonomous agent actions remain subject to applicable authority and lifecycle boundaries.

And it maintains continuity when changes evolve, overlap, fail, require reassessment, or produce additional obligations.

A software change must not be judged only by whether the intended modification was made. It must be judged by whether the change was justified, its consequences were adequately assessed, the applicable lifecycle requirements were satisfied, and the resulting software state was legitimately and verifiably established.

Software Change Management Standard (SCMS) transforms software change from

an isolated modification activity into a traceable, coordinated, and evidence-supported governance process that preserves software lifecycle integrity as systems evolve.

Nahlad

Software Change Management Standard (SCMS)

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Canonical Definition: Software Change Management Standard (SCMS)

establishes how software changes are identified, justified, assessed, coordinated, directed through applicable lifecycle governance processes, verified, and formally resolved while preserving traceability between the original change requirement and the resulting software state.

Governed Space: Software Change Management Governance

Foundational Principle: Every software change must remain governed by what it alters, why the alteration is necessary, which lifecycle mechanisms are affected, and whether the resulting state can be legitimately and verifiably established.

Core Question: How can software changes remain identifiable, justified, coordinated, authorized, traceable, and verifiably resolved throughout the software lifecycle?

Operational Role: SCMS governs change requests, impact assessment,

coordination, lifecycle re-entry, implementation traceability, and closure without replacing specialized software lifecycle standards.

Core Modules: Change Request Management (CRMM), Change Impact Assessment

(CIAM), Change Coordination Control (CHCM), Lifecycle Re-entry Control (LRECM), and Change Closure Management (CCLM).

Canonical Closing Statement: A software change is not governed merely because it was implemented; it is governed when its purpose, consequences, authority, lifecycle requirements, and resulting operational state can be established through evidence.

View Standard

Aboit standard

About Software Change Management Standard (SCMS)

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: About Parent Standard

Governed Space: Software Change Management Governance

Parent Standard: Software Change Management Standard (SCMS)

Standard ID: SLGA-SCMS-09

Version: 1.0

Status: Canonical Open

Effective Date: 14.10.2026

Authority: OOF® — OriginOpen® Foundation

Canonical Language: English

Canonical Definition

Software Change Management Standard (SCMS) is the Parent Standard within SLGA™ that governs how proposed, required, and emerging software changes are identified, justified, assessed, coordinated, directed through applicable software lifecycle governance processes, verified against their intended outcomes, and formally closed or retained as unresolved obligations.

SCMS establishes software change as a governed transition between

identifiable software conditions rather than merely a modification of source code, configuration, dependencies, or deployed functionality.

It preserves the relationship between the reason for a change, the software

objects affected, the consequences introduced, the authority applicable, the lifecycle processes required, and the operational state ultimately established.

Its purpose is to ensure that software evolution remains traceable, proportionate, and governable even when changes are frequent, automated, urgent, or initiated by autonomous systems.

• What It Is

SCMS is a methodological governance standard for managing software change throughout the software lifecycle.

It establishes five interconnected governance functions:

Change Request Management — identifying what should change, why the change is proposed, and which software objects are affected.

Change Impact Assessment — determining the direct and indirect

consequences of the proposed modification.

Change Coordination Control — coordinating affected actors, dependencies,

decisions, implementation activities, and related changes.

Lifecycle Re-entry Control — determining which software lifecycle

governance processes must be revisited because of the change.

Change Closure Management — verifying the resulting change outcome and

preserving unresolved obligations.

Together, these functions establish a continuous governance relationship

between change recognition and change resolution.

SCMS applies to changes initiated by developers, organizations, automated

delivery systems, AI coding assistants, autonomous agents, security incidents, dependency updates, and evolving operational requirements.

• What It Is Not

SCMS is not a software development methodology, project management framework, issue-tracking application, source control system, or deployment automation platform.

It does not prescribe a specific programming language, development workflow, branching strategy, release schedule, or technical change-management tool.

It does not independently authorize every action required to implement a change.

It does not replace software requirements governance, software creation, dependency management, configuration control, quality verification, security assessment, release authorization, or deployment governance.

It does not assume that every software change requires repeating the entire software lifecycle.

It also does not assume that a technically successful modification represents a legitimate, beneficial, or operationally acceptable change.

SCMS governs the change and its lifecycle consequences; specialized

standards govern the technical and operational mechanisms affected by that change.

• Why It Is Important

Modern software changes continuously.

A minor dependency update can alter authentication behavior.

A small code modification can introduce a critical operational failure.

A configuration adjustment can affect multiple services.

A security patch can create compatibility problems.

Two individually acceptable changes can become incompatible when implemented together.

An AI agent can generate and apply software modifications faster than conventional review processes can assess their consequences.

These situations reveal a fundamental governance problem: software

modifications may be technically executable while their full lifecycle implications remain unknown, unverified, or unauthorized.

SCMS addresses this problem by ensuring that changes are not evaluated only by their apparent size, implementation speed, or immediate technical outcome.

Instead, changes must be assessed according to their affected objects, dependencies, consequences, authority, required lifecycle processing, and verified resulting state.

The governance significance of a software change is determined by its consequences, not by the number of lines modified.

Position in Architecture Governance Flow

SCMS is the ninth Parent Standard within SLGA™ — Software Lifecycle Governance Architecture.

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

Although SCMS occupies the ninth architectural position, software changes

may arise at any stage of the lifecycle.

A new requirement may trigger changes during development.

A vulnerability may require changes to an existing production release.

A dependency failure may require configuration, quality, security, and deployment reassessment.

A retirement decision may require migration-related changes before a system can be withdrawn.

SCMS connects these situations to the appropriate governance processes.

It ensures that a change enters or re-enters the lifecycle mechanisms materially affected by the modification.

Software Change Management is not a replacement for the lifecycle. It is the governance mechanism that determines how changes move through the lifecycle.

Standard Governance Flow

Change Origin → Change Recognition → Change Request → Change Purpose →

Affected Software Objects → Change Scope → Impact Assessment → Change Classification → Coordination → Applicable Authority → Change Decision → Lifecycle Re-entry → Governed Implementation → Verification → Outcome Assessment → Residual Obligations → Change Closure

This flow establishes five essential governance transitions.

Recognition to Assessment

A proposed modification becomes an identifiable change case with a defined

purpose, affected objects, and supporting evidence.

Assessment to Coordination

The change's direct and indirect consequences determine the necessary

coordination, restrictions, decisions, and dependencies.

Coordination to Lifecycle Re-entry

The change is directed into the software lifecycle mechanisms required by its actual impact.

Implementation to Verification

The modification performed is compared with the intended change and applicable verification requirements.

Verification to Closure

The change is classified according to its verified outcome, unresolved conditions, and legitimate final disposition.

Where evidence is insufficient, material conditions change, or implementation deviates from the approved scope, reassessment becomes necessary.

Practical Applications

Enterprise Software Development

SCMS supports coordinated software changes across multiple teams, products,

components, and production environments while preserving decision and implementation traceability.

Continuous Integration and Delivery

It connects change requests with implementation records, verification

evidence, release decisions, and deployment outcomes without treating pipeline completion as proof of change legitimacy.

AI-Assisted Software Engineering

It establishes governance boundaries for AI-generated modifications,

ensuring that proposed changes remain subject to impact assessment, appropriate lifecycle processing, and verification.

Autonomous Software Agents

It distinguishes an agent's assigned change objective from its authority to

modify repositories, install dependencies, alter configurations, or deploy software.

Cybersecurity Remediation

It supports urgent vulnerability corrections while preserving applicable

security, quality, configuration, release, and deployment governance.

Critical Infrastructure

It establishes traceable change assessment and coordination where software modifications may affect essential services, operational continuity, or physical systems.

Robotics and Embedded Software

It governs changes to firmware, control logic, dependencies, and

operational behavior where technical modifications can produce consequences beyond the software itself.

Governance Outputs

The principal governance output is the:

Software Change Governance Profile

SCMS produces this profile through five Core Modules.

Change Request Management Module (CRMM)

Output: Software Change Request Profile

Change Impact Assessment Module (CIAM)

Output: Software Change Impact Profile

Change Coordination Control Module (CHCM)

Output: Software Change Coordination Profile

Lifecycle Re-entry Control Module (LRECM)

Output: Software Lifecycle Re-entry Profile

Change Closure Management Module (CCLM)

Output: Software Change Closure Profile

Together, these outputs establish a traceable relationship between the

original change requirement, its assessed consequences, the applicable lifecycle pathway, the implemented modification, and its verified outcome.

Core Governance Distinctions

Change Request ≠ Change Authorization.

A request establishes what is proposed; authorization establishes which actions are legitimately permitted.

Change Justification ≠ Implementation Permission.

A necessary or beneficial modification may still require applicable decisions and restrictions.

Small Code Change ≠ Small Governance Impact.

Technical size does not determine operational significance.

Change Approval ≠ Lifecycle Requirements Satisfied.

Accepting a change does not eliminate required quality, security, release, or deployment governance.

Previously Verified ≠ Still Verified After Material Change.

Existing evidence must remain applicable to the modified software state.

Agent Objective ≠ Unrestricted Modification Authority.

An autonomous agent's assigned task does not authorize every tool action required to achieve it.

Implementation Completed ≠ Change Outcome Verified.

Technical completion does not establish that the intended result was achieved.

Administrative Closure ≠ Successful Change Implementation.

A change case may be closed as rejected, withdrawn, superseded, or unresolved through another governed disposition.

Software Change Management ≠ Software Lifecycle Replacement.

SCMS coordinates the affected lifecycle mechanisms rather than substituting

for them.

Canonical Closing Statement

Software change becomes a governance problem when the ability to modify a system advances faster than the ability to understand, authorize, and verify the consequences of that modification.

A change may be technically simple but operationally significant.

It may solve one problem while creating another.

It may pass isolated tests while failing in combination with other changes.

It may be generated by an autonomous agent whose technical capabilities exceed its legitimate authority.

And it may be successfully implemented while leaving unresolved obligations elsewhere in the software lifecycle.

Software Change Management Standard (SCMS) establishes a methodological

structure for governing these conditions.

It ensures that change identity, purpose, impact, coordination, lifecycle re-entry, implementation evidence, and closure remain connected.

It preserves the necessary distinction between modifying software and

establishing a legitimate, verified software state.

Software evolution must not be governed by the speed at which changes can be made, but by the ability to establish what changed, why it changed, which consequences followed, and whether the resulting software reality remains acceptable.

SCMS makes software change a traceable and verifiable lifecycle governance

process rather than an isolated technical event.

Modul1

Change Request Management Module (CRMM)

Architecture

SLGA™ — Software Lifecycle Governance Architecture

Parent Standard

Software Change Management Standard (SCMS)

Category

Governance & Enforcement

Subcategory

Software Change Management Governance

Type

Core Module

Governed Space

Software Change Request Governance

Module ID

SLGA-SCMS-CRMM-01

Origin ID

OOF-SLGA-SCMS-CRMM

Version

1.0

Status

Canonical Open

Effective Date

10.2026

Compatibility

Methodology OS — Modular Compatibility

GIE-Compatible

Yes

AI-Readable

Yes

Authority

OOF® — OriginOpen® Foundation

Protection

Protected Methodological Work

Canonical Language

English

Canonical Definition

Change Request Management Module (CRMM) is the Core Module within Software Change Management Standard (SCMS) that governs how proposed, required, and emerging software changes are recognized, expressed, identified, recorded, classified, maintained, and directed toward appropriate assessment or disposition.

CRMM establishes a software change request as an identifiable governance

object representing a proposed or required alteration to an existing or intended software state.

It requires each change request to preserve its origin, purpose, affected

software objects, current-state reference, proposed modification, supporting evidence, urgency, known constraints, and initial governance status.

The module distinguishes the recognition of a need for change from proof

that the proposed solution is suitable, the acceptance of a request from authorization to implement it, and a technically generated modification from a legitimately established change request.

CRMM governs change requests originating from human actors, organizations,

operational incidents, security findings, dependency developments, automated systems, AI coding assistants, and autonomous agents.

It establishes the initial identity and continuity required for subsequent

impact assessment, coordination, lifecycle re-entry, implementation traceability, and closure.

Its principal governance output is the Software Change Request Profile.

Operational Role

CRMM establishes the entry point for software change governance.

It governs:

• Change request recognition

• Request identity

• Change origin

• Requesting actor identification

• Problem or opportunity description

• Change purpose

• Current-state reference

• Proposed modification

• Affected software object identification

• Preliminary change scope

• Supporting evidence

• Initial urgency and priority

• Known dependencies and constraints

• Request completeness

• Duplicate and related requests

• Request classification

• Request revision

• Request withdrawal

• Request deferral

• Initial disposition

• Handoff to change impact assessment

CRMM does not determine the full impact of a change, authorize implementation, coordinate all affected lifecycle activities, or verify the final change outcome.

Operational Question

How can a proposed or required software change become an identifiable, sufficiently defined, evidence-supported, and traceable governance request without confusing the recognition of a problem, the proposal of a solution, the urgency of a modification, or an automated recommendation with authorization to change software?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

SCMS Core Module Flow

Change Request Management Module (CRMM) → Change Impact Assessment Module

(CIAM) → Change Coordination Control Module (CHCM) → Lifecycle Re-entry Control Module (LRECM) → Change Closure Management Module (CCLM)

CRMM is the first Core Module within SCMS.

It establishes the change request identity and initial governance record.

Its outputs provide the basis for CIAM to assess consequences and for

subsequent modules to coordinate, route, verify, and close the change.

A request may be revised or reassessed during later stages without losing its original identity and historical continuity.

• Governed Space

Software Change Request Governance

This governed space concerns the recognition, formation, identification,

classification, maintenance, and initial disposition of software change requests.

It begins when a potential change requirement, problem, opportunity, or modification proposal becomes identifiable.

It includes requests originating from development, testing, security,

production operations, dependencies, external requirements, automated analysis, and autonomous agents.

It continues until the request has an appropriate initial disposition and

sufficient information for its next governance step, or has been legitimately rejected, deferred, withdrawn, or otherwise classified.

The request remains traceable throughout the broader software change

lifecycle.

Governed Object

The primary Governed Object is:

A Software Change Request representing an identifiable proposal or

requirement to alter a software object or software condition, associated with its origin, requesting actor, purpose, current-state reference, proposed change, preliminary scope, supporting evidence, and governance status.

The governed object must preserve, where applicable:

Change Request ID

Software Product ID

Request Origin

Requesting Actor Reference

Request Creation Time

Change Purpose

Problem or Opportunity Reference

Affected Software Objects

Current State Reference

Proposed Modification

Preliminary Scope

Supporting Evidence

Urgency Reference

Known Constraints

Related Request References

Request Status

Request Version

Scope

CRMM governs:

Change Request Formation

Change Request Identification

Request Origin Attribution

Requesting Actor Identification

Purpose Definition

Problem and Opportunity Recognition

Proposed Change Description

Affected Object Identification

Current-State Reference

Preliminary Scope Definition

Evidence Association

Urgency Recognition

Request Completeness

Duplicate Request Detection

Related Request Association

Request Classification

Request Revision

Request Disposition

Historical Request Continuity

Assessment Handoff

CRMM does not prescribe a particular ticketing system, issue tracker, repository platform, development methodology, or organizational approval workflow.

Foundational Principle

A software change request establishes what change is being proposed or required and why it deserves governance consideration; it does not establish that the change is suitable, authorized, or ready for implementation.

A defect report may justify investigation without proving that a particular correction is appropriate.

A security alert may establish urgency without authorizing unrestricted modification.

An automated system may identify a dependency problem without establishing the legitimacy of its proposed replacement.

An autonomous agent may generate a technically plausible patch without authority to introduce it into the software lifecycle.

Change Need Identified ≠ Proposed Solution Validated.

Change Request Accepted ≠ Change Implementation Authorized.

Change Request Identity

Every governed change request must have a stable identity.

Relevant elements include:

Change Request ID

Software Product ID

Origin Reference

Creation Time

Requesting Actor

Current Status

Request Version

Evidence Reference

A request identity must remain stable through material revisions, reassessment, deferral, or transfer.

Where a request is replaced or separated into multiple requests, the relationship must remain traceable.

Change Request Formation

A change request is formed when a recognizable change need or proposal is expressed in a manner suitable for governance assessment.

Minimum formation elements include:

Identifiable Software Context

Recognizable Change Origin

Stated Problem or Opportunity

Requested or Proposed Change

Known Affected Scope

Requesting Source

Initial Evidence or Justification

A request may enter governance with incomplete information.

Its incompleteness must remain explicit.

Change Origin

CRMM must identify the source from which the need for change emerged.

Potential origins include:

New Requirement

Defect Discovery

Security Vulnerability

Operational Incident

Dependency Change

Configuration Issue

Quality Finding

Performance Limitation

Compatibility Requirement

Infrastructure Change

External Obligation

Customer Feedback

Automated Detection

Autonomous Agent Recommendation

Software Retirement Requirement

The origin must remain distinguishable from the actor submitting the request.

Requesting Actor Identity

CRMM must identify the actor or system responsible for submitting or generating the request.

Potential actors include:

Developer

Software Maintainer

Quality Engineer

Security Function

Operations Function

Product Owner

Customer Representative

Automated Monitoring System

AI Coding Assistant

Autonomous Software Agent

External System

Where the source is automated, the responsible system and its assigned function must remain identifiable.

Origin–Actor Distinction

A software change request may originate from one condition while being submitted by another actor.

For example:

A monitoring system detects an operational anomaly.

An autonomous agent interprets the anomaly.

A human operator submits the formal change request.

CRMM must preserve the distinction between:

Change Origin

Detection Source

Requesting Actor

Proposed Solution Source

These identities must not be collapsed into a single undifferentiated requester field where doing so would obscure material provenance.

Problem Recognition

CRMM must establish the problem or condition motivating a corrective change request.

Relevant elements include:

Observed Condition

Expected Condition

Material Difference

Affected Software

Observation Time

Operational Context

Supporting Evidence

Known Uncertainty

A reported problem must remain distinguishable from a verified defect or established root cause.

Opportunity Recognition

Not every change request originates from a failure.

Potential improvement opportunities include:

New Functionality

Performance Improvement

Resource Efficiency

Usability Improvement

Compatibility Expansion

Technical Simplification

Maintainability Improvement

Dependency Modernization

Automation Improvement

Operational Capability Enhancement

An opportunity must still have an identifiable purpose and a sufficiently described intended outcome.

Change Purpose Definition

Every request must state what the proposed change is intended to accomplish.

Relevant elements include:

Purpose Statement

Problem or Opportunity

Expected Benefit

Affected Function

Intended Outcome

Success Reference

Known Constraints

A request that only specifies a technical action may require clarification of the underlying purpose.

Requested Action ≠ Established Change Purpose.

Requested Modification

CRMM must distinguish the requested modification from the underlying change need.

Examples include:

Modify Source Code

Replace Dependency

Update Configuration

Change Software Behavior

Introduce Functionality

Remove Functionality

Correct a Security Condition

Change an Interface

Update Firmware

Modify Deployment-Related Software Settings

The proposed modification may later be replaced by a different solution following impact assessment.

Current-State Reference

CRMM must preserve the available reference to the software state that motivates the request.

Relevant elements include:

Current Software Version

Current Artifact

Current Configuration

Current Dependency

Current Behavior

Current Operational Condition

Known Defect

Known Restriction

Evidence Time

The reference may initially be incomplete.

CRMM must identify missing information relevant to subsequent assessment.

Proposed-State Reference

Where possible, the request must describe the intended resulting state.

Relevant elements include:

Expected Functionality

Expected Behavior

Target Dependency Condition

Target Configuration

Target Compatibility

Target Security Condition

Target Performance

Expected Operational Outcome

The proposed state is not a verified implementation result.

Affected Software Object Identification

CRMM must identify the software objects believed to be affected.

Potential objects include:

Software Product

Application

Service

Source Component

Library

Dependency

Configuration

Build Artifact

Interface

Database Schema

Firmware

Automation Logic

Agent Behavior Configuration

Deployment-Related Software Object

Preliminary identification must not be treated as a complete impact assessment.

Preliminary Change Scope

CRMM establishes the scope proposed by the requester.

Relevant elements include:

Affected Software Objects

Requested Functional Changes

Known Dependency Relationships

Target Environments

Expected Data Changes

Known Exclusions

Implementation Constraints

Known Uncertainty

The scope remains provisional until appropriate impact assessment and

governance decisions establish its applicability.

Request Boundary

The request boundary identifies what the request includes and what remains

outside its stated purpose.

Relevant dimensions include:

Functional Boundary

Software Object Boundary

Dependency Boundary

Configuration Boundary

Data Boundary

Environment Boundary

Authority Boundary

Operational Boundary

A request must not become an unrestricted authorization to modify adjacent systems.

Supporting Evidence

CRMM must associate available evidence with the request.

Evidence may include:

Defect Report

Test Result

Security Finding

Monitoring Observation

Dependency Advisory

User Report

Compatibility Report

Performance Measurement

Operational Incident Record

Automated Analysis

Agent-Generated Assessment

Evidence must remain distinguishable from assumptions and proposed explanations.

Evidence Provenance

Material evidence must retain relevant provenance.

Relevant elements include:

Evidence ID

Evidence Source

Collection Method

Collection Time

Observed Software Object

Evidence Integrity Reference

Known Limitations

Verification Status

Automated or AI-generated evidence must not be treated as independently verified merely because it is presented in a structured report.

Evidence Sufficiency at Request Formation

CRMM evaluates whether the request contains sufficient information for its

next governance step.

Relevant questions include:

Is the software context identifiable?

Is the change purpose understandable?

Are affected objects sufficiently identified?

Is the proposed modification described?

Is supporting evidence available?

Are material uncertainties visible?

Can the request be meaningfully assessed?

A request may be valid for investigation without being ready for full impact assessment.

Request Completeness

CRMM recognizes the following completeness conditions:

Complete for Initial Assessment

Partially Complete

Clarification Required

Evidence Required

Scope Undefined

Origin Uncertain

Request Conflicted

Completeness is assessed relative to the intended next governance step.

It does not imply that the proposed change is feasible or authorized.

Change Request Classification

CRMM must establish an initial classification.

Potential categories include:

Functional Change Request

Corrective Change Request

Security Change Request

Dependency Change Request

Configuration Change Request

Compatibility Change Request

Performance Change Request

Emergency Change Request

Migration Change Request

Retirement-Related Change Request

Automated Change Request

Autonomous Agent-Initiated Change Request

A request may belong to multiple categories.

Classification may change following further assessment.

Request Urgency

CRMM must preserve the stated urgency of a request.

Relevant factors include:

Operational Severity

Security Exposure

Service Disruption

User Impact

External Deadline

Dependency Deadline

Safety-Relevant Condition

Continuity Requirement

Potential Consequence of Delay

Urgency must remain distinguishable from implementation authority.

Urgent Request ≠ Authorized Emergency Change.

Request Priority

Priority concerns the relative importance of processing a request.

Relevant elements include:

Requested Priority

Priority Justification

Known Impact

Urgency

Resource Considerations

Related Requests

Applicable Coordination Needs

The requester may propose priority.

The competent governance function determines applicable prioritization

according to its authority and criteria.

Request Justification

Request justification must identify why the proposed change deserves consideration.

Relevant elements include:

Problem Evidence

Expected Benefit

Operational Necessity

Applicable Requirement

Risk of Inaction

Known Alternatives

Time Sensitivity

Known Uncertainty

A persuasive justification does not independently establish solution suitability or implementation permission.

Request Assumptions

CRMM must preserve assumptions materially affecting the proposed change.

Examples include:

Assumed Root Cause

Assumed Dependency Compatibility

Assumed Environment State

Assumed Software Version

Assumed Resource Availability

Assumed Implementation Simplicity

Assumed Recovery Feasibility

Assumptions must remain distinguishable from verified facts.

Request Constraints

Known constraints may include:

Operational Availability Requirements

Security Restrictions

Compatibility Requirements

Resource Limits

Implementation Deadlines

Data Protection Conditions

Infrastructure Limitations

External Dependencies

Physical-System Restrictions

Constraints must be preserved for subsequent impact assessment and coordination.

Request Dependencies

A change request may depend on other changes or conditions.

Relevant elements include:

Prerequisite Change Request

Related Defect

Dependency Update

Configuration Revision

Required Infrastructure Change

External System Requirement

Required Migration

Required Recovery Action

Dependencies must remain identifiable even when their complete consequences are not yet known.

Duplicate Request Detection

Multiple requests may concern the same underlying change need.

CRMM must support identification of:

Identical Requests

Substantially Overlapping Requests

Related but Distinct Requests

Repeated Reports of One Defect

Different Solutions to One Problem

Duplicate detection must not erase independent evidence or distinct operational consequences.

Request Consolidation

Related requests may be consolidated where justified.

Consolidation must preserve:

Original Request IDs

Original Sources

Distinct Evidence

Affected Objects

Combined Purpose

Material Differences

Consolidation Decision

Historical References

Requests must not be merged merely because they share a broad software category.

Request Separation

A request may contain multiple materially distinct changes.

Separation may be required where changes differ in:

Purpose

Affected Software Objects

Impact

Authority

Urgency

Implementation Pathway

Verification Requirements

Operational Consequences

Separated requests must retain traceability to their common origin.

Request Relationship Governance

CRMM must preserve relationships among change requests.

Potential relationships include:

Duplicate Of

Related To

Depends On

Blocks

Supersedes

Superseded By

Derived From

Split From

Merged Into

These relationships support subsequent coordination without predetermining

implementation decisions.

Automated Request Generation

Automated systems may create change requests from predefined conditions.

Examples include:

Dependency Update Detection

Security Alert

Failed Quality Test

Configuration Drift

Operational Monitoring Trigger

Performance Threshold

Compatibility Failure

CRMM must preserve the generating system, detection basis, request scope, and evidence.

Automated Detection ≠ Validated Change Requirement.

AI-Generated Change Requests

AI systems may formulate change requests using code analysis, operational observations, documentation, or user instructions.

CRMM must identify:

AI System or Agent

Assigned Function

Input Context Reference

Proposed Change

Supporting Evidence

Generated Assumptions

Known Limitations

Required Review

AI-generated explanations must not be treated as established facts without appropriate verification.

Autonomous Agent Request Boundary

An autonomous agent may be authorized to identify or propose changes without being authorized to implement them.

CRMM must preserve:

Agent Identity

Assigned Objective

Request Generation Scope

Permitted Evidence Sources

Affected Software Objects

Decision Boundary

Escalation Requirements

Applicable Authority Reference

Agent Permission to Propose ≠ Agent Permission to Implement.

Human and Organizational Request Governance

Human-generated requests must also preserve applicable identity and authority distinctions.

A developer may identify a defect.

A product owner may request new functionality.

An operations engineer may report an unacceptable deployment condition.

A security function may require remediation.

These roles do not automatically possess identical authority to approve,

implement, release, or deploy the proposed changes.

Request Review

CRMM must establish an initial review sufficient to determine the request's next governance step.

Review may evaluate:

Request Identity

Purpose Clarity

Affected Object Identification

Evidence Availability

Preliminary Classification

Known Constraints

Related Requests

Material Uncertainty

Assessment Readiness

The review must not substitute for CIAM's substantive change impact assessment.

Request Clarification

Clarification may be required when the request lacks information necessary for meaningful assessment.

Examples include:

Unclear Problem Description

Undefined Software Product

Unknown Affected Component

Ambiguous Intended Outcome

Unsupported Urgency

Conflicting Evidence

Unclear Requesting Source

Missing Operational Context

Clarification requirements must remain identifiable and traceable.

Request Rejection

A request may be rejected at the request-management stage when an appropriate basis exists.

Potential reasons include:

Outside Governed Software Scope

Insufficient Justification After Reasonable Clarification

Request Withdrawn by Applicable Actor

Duplicate Already Governed Elsewhere

Proposed Change Incompatible With Mandatory Constraints

No Identifiable Change Need

Rejection must preserve its basis and applicable decision authority.

Rejection at this stage must not be used to conceal material defects, security findings, or operational risks that require governance through another mechanism.

Request Deferral

A request may be deferred when assessment or processing cannot appropriately proceed.

Potential reasons include:

Missing Evidence

Unresolved Dependency

Insufficient Context

Resource Constraint

External Decision Pending

Related Change Pending

Required Clarification

Deferral must preserve the reason, responsible function, and applicable reassessment condition.

Request Withdrawal

A request may be withdrawn when the applicable actor no longer seeks the proposed modification.

CRMM must preserve:

Withdrawal Actor

Withdrawal Authority or Eligibility

Withdrawal Reason

Withdrawal Time

Related Obligations

Remaining Risk References

Disposition Evidence

Withdrawal of a request must not automatically eliminate independently established safety, security, or operational obligations.

Request Supersession

A request may be superseded by a newer or more appropriate change request.

Relevant elements include:

Original Request

Successor Request

Reason for Supersession

Transferred Evidence

Changed Scope

Changed Purpose

Outstanding Obligations

Historical Continuity

Supersession must not erase the earlier request's decisions or evidence.

Request Revision

A request may require revision when its purpose, scope, evidence, or proposed modification changes.

CRMM must preserve:

Request Version

Previous Version

Revised Elements

Revision Reason

Revision Actor

Revision Time

New Evidence

Assessment Implications

Material revisions must not silently inherit earlier conclusions that no longer apply.

Request Status

CRMM recognizes the following principal request states:

Identified

Draft

Submitted

Clarification Required

Evidence Required

Under Initial Review

Complete for Initial Assessment

Ready for Impact Assessment

Deferred

Rejected

Withdrawn

Superseded

Transferred

Unknown

Conflicted

Request status must remain distinct from the broader change implementation and closure status governed by SCMS.

Initial Request Disposition

CRMM must establish an appropriate initial disposition.

Possible outcomes include:

Proceed to Impact Assessment

Request Clarification

Request Additional Evidence

Consolidate With Related Request

Separate Into Distinct Requests

Defer

Reject

Withdraw

Supersede

Transfer to Another Applicable Governance Mechanism

The disposition must remain associated with its basis and competent decision reference.

Request-to-Assessment Handoff

Where a request proceeds to CIAM, CRMM must provide sufficient information for substantive impact assessment.

The handoff should include:

Change Request ID

Software Product

Change Origin

Purpose

Current-State Reference

Proposed Modification

Affected Objects

Preliminary Scope

Evidence

Known Constraints

Known Dependencies

Uncertainty

Related Requests

Request Classification

The handoff must preserve the distinction between preliminary request information and verified assessment findings.

Request Evidence Continuity

CRMM must preserve material evidence throughout the request lifecycle.

This includes:

Original Submission

Supporting Evidence

Clarifications

Request Revisions

Duplicate Assessments

Relationship Decisions

Initial Reviews

Disposition Decisions

Handoff Records

Historical evidence must remain reconstructable even when the request is later revised, merged, or superseded.

Request Governance Classification

CRMM recognizes the following governance classifications:

Request Identifiable

Request Incomplete

Purpose Unclear

Scope Unclear

Evidence Insufficient

Origin Uncertain

Related Request Identified

Duplicate Candidate

Request Conflicted

Ready for Assessment

Deferred

Rejected

Withdrawn

Superseded

Transferred

Unknown

Classification must remain associated with the relevant request version and evidence.

Governance Requirements

A conforming CRMM implementation must satisfy the following requirements. Request Identity and Formation

- Establish a stable change request identity
- Identify the software product or relevant software context
- Preserve the request origin
- Identify the requesting actor or system
- Preserve the creation time
- Establish the change purpose
- Identify the proposed modification
- Preserve the request version

Scope and Context

- Identify the current-state reference
- Identify the intended change outcome where available
- Identify affected software objects
- Establish preliminary scope
- Preserve known exclusions
- Preserve material assumptions
- Preserve known constraints
- Identify related requests and dependencies

Evidence and Classification

- Preserve supporting evidence
- Identify evidence provenance
- Distinguish observations from assumptions
- Evaluate request completeness
- Identify missing evidence
- Establish initial change classification
- Preserve urgency justification
- Distinguish urgency from authority

Request Control

- Identify duplicate requests
- Preserve consolidation decisions
- Preserve request separation references
- Maintain request relationships
- Preserve material revisions
- Govern deferral and withdrawal
- Preserve rejection and supersession decisions

• Maintain historical request continuity

Automation and Handoff

• Identify automated request sources

• Preserve AI-generated request limitations

• Distinguish agent proposal authority from implementation authority

• Preserve initial review outcomes

• Establish request disposition

• Identify assessment readiness

• Provide structured handoff to CIAM

• Preserve the distinction between request acceptance and change

authorization.

Governance Logic

Change Signal → Origin Identification → Requesting Actor → Software

Context → Problem or Opportunity → Change Purpose → Proposed Modification → Affected Objects → Preliminary Scope → Evidence → Assumptions and Constraints → Request Completeness → Initial Classification → Related Request Assessment → Initial Review → Request Disposition → CIAM Handoff / Clarification / Deferral / Rejection / Withdrawal / Supersession

Where information is insufficient:

Request → Missing Information → Clarification or Evidence Requirement →

Revised Request → Completeness Reassessment

Where duplicate or related requests exist:

Request → Relationship Assessment → Preserve Distinct Evidence →

Consolidate / Separate / Maintain Relationship

Where an autonomous agent generates a request:

Agent Observation → Proposed Change → Agent Identity and Scope → Evidence

Assessment → Request Formation → Authorized Review Pathway

Where the request changes materially:

Original Request → Material Revision → Version Preservation → Reassessment

of Initial Conclusions → Updated Disposition

CRMM must not convert request formation, automated detection, or preliminary acceptance into permission to implement software modifications.

Governance Outputs

The principal output is:

Software Change Request Profile

Supporting outputs may include:

Change Request Identity

Change Origin Record

Requesting Actor Reference

Change Purpose Statement

Problem or Opportunity Record

Current-State Reference

Proposed Modification Record

Affected Software Object Register

Preliminary Change Scope

Supporting Evidence Register

Request Completeness Assessment

Initial Change Classification

Urgency Reference

Known Constraint Register

Related Request Register

Request Revision History

Request Disposition Record

Assessment Readiness Classification

CIAM Handoff Record

Minimum Implementation Framework

A minimum CRMM implementation must establish eight governance layers. Layer 1 — Request Identity

Identify the change request, software context, requesting actor, origin,

and creation time. Layer 2 — Change Purpose

Establish the problem or opportunity, proposed modification, and intended

outcome. Layer 3 — Software Context

Identify affected software objects, current-state references, preliminary

scope, assumptions, and known constraints. Layer 4 — Evidence

Preserve supporting observations, reports, source references, provenance,

and material uncertainties. Layer 5 — Classification

Establish the initial request category, urgency, completeness, and known

related requests. Layer 6 — Request Control

Manage clarification, duplicate detection, consolidation, separation,

revision, deferral, withdrawal, rejection, and supersession. Layer 7 — Disposition

Determine whether the request is sufficiently defined for impact assessment or requires another legitimate disposition. Layer 8 — Governed Output and Handoff

Establish the Software Change Request Profile and preserve the necessary

information for CIAM.

The implementation may use manual forms, issue-tracking systems, repository workflows, automated detection, or AI-assisted request generation.

The governance logic remains independent of the technology used.

Cross-Module Boundary

CRMM → CIAM

CRMM establishes the request identity, purpose, preliminary scope, affected

objects, and available evidence.

CIAM evaluates the direct and indirect consequences, feasibility,

dependencies, and lifecycle implications of the proposed change.

CRMM → CHCM

CRMM preserves initial relationships among requests.

CHCM governs coordination among affected actors, activities, decisions, and

related changes.

CRMM → LRECM

CRMM identifies the proposed change.

LRECM determines the applicable lifecycle re-entry requirements based on

the assessed impact and governance conditions.

CRMM → CCLM

CRMM preserves the original request and its historical revisions.

CCLM evaluates the final change disposition, verified outcome, and closure

obligations.

Request Ready for Assessment ≠ Change Ready for Implementation.

Cross-Architecture Boundary

CRMM may receive change signals and evidence from:

SRS — Software Requirements Standard

New or revised software requirements.

SCS — Software Creation Standard

Development findings and artifact modification proposals.

SDS — Software Dependency Standard

Dependency changes, incompatibilities, and continuity requirements.

SCFS — Software Configuration Standard

Configuration changes, build findings, and artifact identity discrepancies.

SQS — Software Quality Standard

Defects, failed tests, and quality findings.

SSS — Software Security Standard

Vulnerabilities, security findings, and remediation requirements.

SRLS — Software Release Standard

Release readiness findings, release restrictions, and withdrawal-related

change needs.

SDPS — Software Deployment Standard

Deployment failures, production version inconsistencies, and

recovery-related change needs.

SRTS — Software Retirement Standard

Migration, replacement, and retirement-related modifications.

SLCS — Software Lifecycle Control Standard

Lifecycle inconsistencies, exceptions, and transition requirements.

CRMM establishes the change request associated with these signals.

It does not replace the governance mechanisms responsible for the originating conditions.

Practical Applications

Software Development Teams

Creating traceable requests for defects, enhancements, technical

improvements, and compatibility modifications.

Security Operations

Converting vulnerability findings into identifiable change requests without

confusing remediation urgency with implementation authorization.

Automated Dependency Management

Maintaining evidence-supported requests for dependency updates detected by

automated tools.

AI-Assisted Software Engineering

Preserving the identity, purpose, assumptions, and evidence of AI-generated

software modification proposals.

Autonomous Agent Systems

Separating an agent's ability to detect or propose changes from its

permission to modify software or production infrastructure.

Industrial and Embedded Software

Establishing traceable requests for firmware, controller configuration, and

software behavior changes.

Use Case 1 — An AI Agent Proposes a Security Fix Without Sufficient

Evidence

An autonomous coding agent analyzes a software repository and identifies a

dependency it considers vulnerable.

The agent proposes replacing the dependency with a newer version.

Its report contains a vulnerability claim but does not establish which dependency version is actually used in the relevant production release.

CRMM evaluates:

Agent Recommendation → Request Identity → Software Context → Vulnerability

Claim → Proposed Modification → Supporting Evidence → Request Completeness

The assessment identifies:

• The agent's proposal is a recognizable change request

• The software object and proposed modification are identifiable

• The production dependency reference is not sufficiently established

• The evidence does not yet support the full vulnerability claim

• Additional clarification or evidence is required before substantive

impact assessment.

The appropriate classification is:

Evidence Insufficient — Clarification Required.

AI-Generated Security Recommendation ≠ Verified Software Change

Requirement.

Use Case 2 — Multiple Teams Report the Same Defect With Different

Operational Evidence

Three teams submit requests concerning failures in a shared authentication

service.

The development team reports a failed test.

The operations team reports intermittent production authentication failures.

The customer support team reports user access problems.

All three requests appear to concern the same software component.

CRMM evaluates:

Request A → Request B → Request C → Shared Software Object → Evidence

Comparison → Relationship Assessment → Consolidation Decision

The assessment identifies:

• The requests may concern one underlying problem

• Each request contains distinct evidence

• The operational consequences differ across the reports

• Consolidation may be appropriate if the common change need is

• Original evidence and request identities must remain traceable

established.

The appropriate classification is:

Related Requests Identified — Consolidation Assessment Required.

Shared Software Component ≠ Automatically Identical Change Request.

Critical Distinctions

Change Need Identified ≠ Proposed Solution Validated.

Change Request Accepted ≠ Change Implementation Authorized.

Requested Action ≠ Established Change Purpose.

Reported Problem ≠ Verified Defect.

Observed Failure ≠ Established Root Cause.

Preliminary Scope ≠ Complete Change Impact.

Request Urgency ≠ Implementation Authority.

Urgent Request ≠ Authorized Emergency Change.

Supporting Evidence Available ≠ Evidence Sufficient.

AI-Generated Recommendation ≠ Verified Change Requirement.

Automated Detection ≠ Validated Change Requirement.

Agent Permission to Propose ≠ Agent Permission to Implement.

Related Requests ≠ Duplicate Requests.

Request Consolidation ≠ Evidence Elimination.

Request Withdrawal ≠ Underlying Risk Resolved.

Request Ready for Assessment ≠ Change Ready for Implementation.

Canonical Boundary

CRMM governs the recognition, identification, formation, preliminary

classification, evidence association, maintenance, and initial disposition of software change requests.

It establishes the relationship between a change origin, requesting actor,

software context, proposed modification, preliminary scope, supporting evidence, and request governance status.

CRMM does not determine complete change impact, authorize implementation, coordinate the full change lifecycle, or verify the resulting software state.

Its principal governance output is the:

Software Change Request Profile

This output establishes the identifiable and evidence-supported starting

point for subsequent software change governance.

Canonical Closing Statement

A software change cannot be governed effectively if the original reason

for the change, the proposed modification, and the evidence supporting the request cannot be distinguished.

A defect report may describe a real problem while proposing the wrong solution.

A security alert may justify urgent investigation without establishing the correct remediation.

A dependency update may appear routine while its operational consequences remain unknown.

Multiple teams may report the same underlying problem through different observations.

An AI agent may generate a convincing change proposal without possessing sufficient evidence or authority to implement it.

And a software modification may be technically possible before anyone has established why it should be made.

CRMM establishes the governance structure required to distinguish these

conditions.

It requires software change requests to remain identifiable, attributable,

sufficiently described, and connected to their supporting evidence.

It preserves uncertainty, preliminary assumptions, request relationships,

and material revisions.

It ensures that requests can be clarified, assessed, consolidated, deferred, rejected, or transferred without losing their origin and historical continuity.

And it establishes a disciplined boundary between recognizing a change need

and deciding whether or how the proposed change should proceed.

A change request is not permission to modify software. It is the identifiable governance starting point from which the necessity, consequences, legitimacy, and appropriate lifecycle pathway of a proposed modification can be established.

Change Request Management Module (CRMM) transforms software change

proposals from informal instructions or automated recommendations into traceable, evidence-supported governance objects suitable for structured lifecycle assessment.

Modul2

Change Impact Assessment Module (CIAM)

Architecture

SLGA™ — Software Lifecycle Governance Architecture

Parent Standard

Software Change Management Standard (SCMS)

Category

Governance & Enforcement

Subcategory

Software Change Management Governance

Type

Core Module

Governed Space

Software Change Impact Governance

Module ID

SLGA-SCMS-CIAM-02

Origin ID

OOF-SLGA-SCMS-CIAM

Version

1.0

Status

Canonical Open

Effective Date

10.2026

Compatibility

Methodology OS — Modular Compatibility

GIE-Compatible

Yes

AI-Readable

Yes

Authority

OOF® — OriginOpen® Foundation

Protection

Protected Methodological Work

Canonical Language

English

Canonical Definition

Change Impact Assessment Module (CIAM) is the Core Module within Software Change Management Standard (SCMS) that governs how the potential and observed consequences of proposed software changes are identified, analyzed, classified, evidenced, and translated into requirements for subsequent software lifecycle governance.

CIAM establishes change impact as a multidimensional relationship between

an identifiable proposed modification, the software objects it directly affects, the dependencies and conditions through which its consequences may propagate, and the resulting implications for software functionality, quality, security, configuration, operational continuity, and lifecycle integrity.

The module distinguishes the technical size of a modification from its

governance significance, direct impact from indirect impact, anticipated consequences from verified consequences, and impact assessment from implementation authorization.

It requires material effects, dependencies, assumptions, uncertainties,

reversibility limitations, and verification needs to remain identifiable rather than being concealed within simplified change classifications or aggregate risk scores.

CIAM applies to human-proposed changes, automated modifications,

AI-generated code, dependency updates, configuration revisions, security remediation, infrastructure-related software changes, and autonomous software systems.

Its principal governance output is the Software Change Impact Profile.

Operational Role

CIAM establishes the substantive assessment layer between change request

formation and coordinated lifecycle processing.

It governs:

• Change impact assessment identity

• Proposed change interpretation

• Assessment scope

• Current-state and intended-state references

• Directly affected software objects

• Indirect impact propagation

• Functional consequences

• Dependency consequences

• Configuration and build consequences

• Quality and security consequences

• Data and interface compatibility

• Operational consequences

• Reversibility and recovery implications

• Combined and concurrent change effects

• Assessment evidence

• Assumptions and uncertainty

• Impact significance

• Feasibility considerations

• Required verification

• Lifecycle governance implications

• Material reassessment triggers

• Impact assessment conclusions

CIAM does not authorize implementation, perform technical changes, replace specialized lifecycle assessments, or establish final operational acceptance.

Operational Question

How can the direct, indirect, combined, and potentially evolving consequences of a proposed software change be identified, evaluated, and supported by evidence so that its governance significance, affected lifecycle mechanisms, uncertainty, and required controls remain visible before the change proceeds?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

SCMS Core Module Flow

Change Request Management Module (CRMM) → Change Impact Assessment Module

(CIAM) → Change Coordination Control Module (CHCM) → Lifecycle Re-entry Control Module (LRECM) → Change Closure Management Module (CCLM)

CIAM is the second Core Module within SCMS.

It receives identifiable change requests and supporting evidence from CRMM.

It evaluates the consequences of the proposed modification and establishes

the impact references required for coordination, lifecycle re-entry, and subsequent verification.

Its conclusions may be revised when implementation evidence, dependency conditions, or operational observations reveal additional consequences.

• Governed Space

Software Change Impact Governance

This governed space concerns the identification, evaluation,

classification, and evidence-supported interpretation of software change consequences.

It begins when an identifiable software change requires substantive assessment of its potential effects.

It includes direct modifications, indirect dependency effects, interactions

with other changes, and consequences extending beyond the initially proposed scope.

It continues through assessment, clarification, impact classification, uncertainty treatment, reassessment, and the establishment of applicable lifecycle governance implications.

The governed space does not end merely because an initial impact report has been produced.

Material new evidence may require the impact assessment to be reopened.

Governed Object

The primary Governed Object is:

A Software Change Impact Assessment representing the evidence-supported

relationship between an identifiable proposed software change, its current and intended software conditions, directly and indirectly affected objects, material consequences, assumptions, uncertainties, significance, feasibility considerations, and required lifecycle governance responses.

The governed object must preserve, where applicable:

Impact Assessment ID

Change Request ID

Software Product ID

Assessment Scope

Current-State Reference

Intended-State Reference

Directly Affected Objects

Indirectly Affected Objects

Dependency References

Impact Dimensions

Impact Evidence

Assumptions

Uncertainty

Impact Significance

Feasibility Conditions

Required Verification

Lifecycle Implications

Assessment Status

Assessment Version

Scope

CIAM governs:

Change Impact Recognition

Assessment Boundary Definition

Current–Intended State Comparison

Direct Impact Identification

Indirect Impact Propagation

Dependency Impact

Configuration Impact

Quality Impact

Security Impact

Data Impact

Interface Impact

Operational Impact

Resource Impact

Reversibility Assessment

Recovery Implications

Concurrent Change Impact

Impact Evidence

Impact Significance

Impact Uncertainty

Verification Requirements

Lifecycle Implications

Impact Reassessment

CIAM does not prescribe a particular risk-scoring system, static analysis tool, dependency scanner, testing framework, or software architecture methodology.

Foundational Principle

The governance significance of a software change is determined by the consequences it may introduce, not by the apparent size, simplicity, or speed of the modification.

A one-line code change may alter authorization behavior.

A dependency update may affect multiple services.

A configuration modification may create data integrity consequences.

A small firmware change may alter physical system behavior.

An automated correction may resolve one defect while introducing another.

Modification Size ≠ Impact Significance.

Local Correctness ≠ System-Level Compatibility.

Known Direct Impact ≠ Complete Change Impact.

Impact Assessment Identity

Every governed impact assessment must have a stable identity.

Relevant elements include:

Impact Assessment ID

Change Request ID

Software Product ID

Assessment Actor or Function

Assessment Time

Assessment Scope

Evidence Reference

Assessment Status

Assessment Version

Material reassessments must preserve their relationship to previous assessment versions.

Change Request Reference

CIAM must preserve the change request on which the assessment is based.

Relevant elements include:

Change Request ID

Request Version

Change Purpose

Proposed Modification

Affected Software Objects

Preliminary Scope

Supporting Evidence

Known Constraints

Known Assumptions

Request Classification

A material change to the request may invalidate part or all of an earlier impact assessment.

Assessment Purpose

The assessment purpose identifies the governance decisions and lifecycle

implications that the analysis must support.

Potential purposes include:

Determine Material Consequences

Identify Affected Components

Establish Dependency Effects

Identify Verification Requirements

Evaluate Operational Exposure

Identify Reversibility Limitations

Support Change Coordination

Support Lifecycle Re-entry

Identify Required Specialized Assessments

Establish Reassessment Conditions

The purpose must remain distinguishable from approval of the proposed change.

Assessment Scope

CIAM must define the scope of the impact assessment.

Relevant dimensions include:

Software Objects

Functional Behavior

Dependencies

Configuration

Build Conditions

Security

Quality

Data

Interfaces

Operational Environments

External Systems

Physical-System Effects Where Applicable

The assessment scope must be justified by the proposed change and available evidence.

Assessment Boundary

The assessment boundary identifies which consequences are evaluated directly and which require referral to other governance mechanisms.

Relevant boundaries include:

Software Object Boundary

Dependency Boundary

Data Boundary

Infrastructure Boundary

Security Boundary

Operational Boundary

Organizational Boundary

External System Boundary

Physical-World Boundary

A defined assessment boundary must not be used to dismiss material effects that extend beyond it.

Such effects must be recorded and referred to an applicable governance mechanism.

Current-State Baseline

CIAM must establish a sufficiently reliable reference to the software state before the proposed modification.

Relevant elements include:

Current Software Version

Current Artifact Identity

Current Configuration

Current Dependencies

Current Functional Behavior

Current Quality Conditions

Current Security Conditions

Current Data Structures

Current Operational Conditions

Known Restrictions

Where the current state cannot be sufficiently established, the limitation must remain visible.

Intended-State Reference

CIAM must identify the software state the change is expected to establish.

Relevant elements include:

Target Functionality

Target Behavior

Target Artifact

Target Configuration

Target Dependencies

Target Security Conditions

Target Quality Conditions

Target Data Compatibility

Target Operational Outcome

Expected Verification Conditions

The intended state is a proposed target, not evidence of implementation success.

Current–Intended State Comparison

CIAM must compare relevant current and intended conditions.

The comparison may include:

Existing Functionality ↔ Proposed Functionality

Existing Artifacts ↔ Proposed Artifacts

Existing Dependencies ↔ Proposed Dependencies

Existing Configuration ↔ Proposed Configuration

Existing Security Conditions ↔ Proposed Security Conditions

Existing Data Structures ↔ Proposed Data Structures

Existing Operational Behavior ↔ Expected Behavior

The comparison establishes the initial change surface.

It does not establish that all indirect consequences have been discovered.

Direct Impact

Direct impact concerns software objects and conditions intentionally

modified by the proposed change.

Examples include:

Source Code Modification

Dependency Replacement

Configuration Revision

Interface Modification

Database Schema Change

Firmware Update

Authentication Logic Change

Automation Policy Modification

Direct impact must be associated with identifiable software objects and the proposed modification.

Indirect Impact

Indirect impact concerns consequences arising through relationships with other objects, systems, or conditions.

Examples include:

Dependent Service Failure

Transitive Dependency Incompatibility

Changed Data Consumer Behavior

Altered Security Exposure

Performance Degradation

Infrastructure Capacity Effects

Unexpected Operational Interaction

Recovery Incompatibility

Indirect impact may extend beyond the software component initially modified.

Impact Propagation

CIAM must evaluate how a change may propagate through connected software relationships.

Potential propagation paths include:

Component → Dependent Component

Library → Application

Interface → External Consumer

Configuration → Runtime Behavior

Database Schema → Data Consumer

Security Policy → Access Behavior

Firmware → Device Operation

Agent Tool Configuration → Downstream Actions

Propagation assessment must distinguish supported relationships from hypothetical possibilities.

Impact Propagation Boundary

CIAM must identify where impact propagation is sufficiently understood and where material uncertainty remains.

Relevant elements include:

Known Dependency Relationships

Known Consumers

Known Integration Points

Observed Runtime Connections

Unmapped Dependencies

External System Boundaries

Unavailable Evidence

Unverified Propagation Paths

An incomplete dependency map must not be treated as proof that no additional impact exists.

Functional Impact

Functional impact concerns changes to software capabilities and behavior.

Relevant dimensions include:

Introduced Functionality

Removed Functionality

Modified Behavior

Changed Business Logic

Changed Error Handling

Changed User Interaction

Changed Automation Behavior

Changed System Outputs

CIAM must identify whether the intended functional modification may affect other established requirements.

Requirements Impact

Software changes may alter the validity or applicability of existing requirements.

Relevant elements include:

Affected Requirement IDs

Changed Functional Requirements

Changed Nonfunctional Requirements

New Requirements

Obsolete Requirements

Conflicting Requirements

Requirements Traceability

Required Reassessment

SRS governs the applicable requirements mechanisms.

CIAM identifies their change-related impact.

Software Architecture Impact

Changes may affect software structure and component relationships.

Relevant elements include:

Component Boundaries

Service Responsibilities

Interface Contracts

Data Flows

Execution Paths

Shared Infrastructure Dependencies

Architectural Assumptions

System Integration Conditions

An apparently local modification may affect architectural relationships outside its immediate implementation scope.

Dependency Impact

CIAM must evaluate material consequences for software dependencies.

Relevant elements include:

Introduced Dependencies

Removed Dependencies

Version Changes

Transitive Dependencies

Compatibility Requirements

Dependency Availability

Security Exposure

Maintenance Continuity

External Service Dependencies

SDS governs dependency identity, suitability, introduction, and continuity.

CIAM establishes which dependency governance activities are implicated by the change.

Configuration Impact

Configuration impact concerns changes to settings, baselines, and

environment-dependent behavior.

Relevant elements include:

Configuration Baseline

Runtime Parameters

Feature Flags

Environment Variables

Build Configuration

Service Configuration

Access Configuration

Infrastructure Configuration

Configuration Compatibility

Configuration impact must remain distinguishable from source code impact.

Build and Artifact Impact

CIAM must identify whether the proposed change affects software build conditions or artifact identity.

Relevant elements include:

Build Inputs

Build Dependencies

Compiler or Toolchain

Artifact Composition

Artifact Version

Reproducibility

Artifact Integrity

Distribution Requirements

Compatibility With Existing Releases

SCFS governs the relevant build and configuration mechanisms.

Quality Impact

Quality impact concerns potential effects on software quality requirements

and conditions.

Relevant dimensions include:

Functional Correctness

Reliability

Performance

Availability

Maintainability

Compatibility

Testability

Regression Exposure

Known Defects

CIAM identifies required quality reassessment.

SQS governs the applicable quality verification mechanisms.

Test Impact

CIAM must identify how the proposed change affects verification coverage.

Relevant elements include:

Existing Test Applicability

Required New Tests

Regression Test Requirements

Integration Test Requirements

Security Test Requirements

Performance Test Requirements

Environment-Specific Tests

Unverified Conditions

Evidence Reuse Limitations

Existing tests must not automatically be treated as sufficient for a materially changed software state.

Security Impact

Security impact concerns potential changes to security requirements,

exposure, and controls.

Relevant elements include:

Authentication

Authorization

Access Permissions

Attack Surface

Data Exposure

Dependency Vulnerabilities

Security Configuration

Secrets and Credentials

Auditability

Security Monitoring

Remediation Requirements

A security-motivated change may itself introduce new security consequences.

Data Impact

CIAM must evaluate potential effects on data structures, integrity, availability, and compatibility.

Relevant dimensions include:

Database Schema

Data Format

Data Migration

Data Retention

Data Integrity

Data Access

Data Consumers

Backup Compatibility

Restoration Feasibility

External Data Exchange

Data impact may remain significant even when the software artifact can be restored to its previous version.

Interface and Integration Impact

Changes may affect interfaces between software components or external systems.

Relevant elements include:

API Contracts

Message Formats

Communication Protocols

Data Exchange Semantics

Authentication Interfaces

Version Compatibility

External Consumers

Integration Timing

Failure Handling

An interface change may affect systems outside the direct control of the change requester.

Operational Impact

Operational impact concerns potential effects on software operating

conditions.

Relevant dimensions include:

Service Availability

Service Continuity

Runtime Behavior

Infrastructure Requirements

Deployment Complexity

Monitoring Coverage

Operational Restrictions

Recovery Capability

User Disruption

External System Consequences

Operational impact must be assessed according to the actual environment in which the software operates.

Infrastructure Impact

Software changes may alter infrastructure requirements.

Relevant elements include:

Compute Resources

Memory Consumption

Storage Requirements

Network Traffic

Infrastructure Configuration

Scaling Requirements

Platform Compatibility

Availability Architecture

Resource Allocation

A software modification may be technically correct while exceeding the capacity or constraints of the intended infrastructure.

Performance and Resource Impact

CIAM must identify material changes in resource consumption and performance.

Relevant dimensions include:

Execution Time

Latency

Throughput

Memory Usage

Storage Consumption

Network Consumption

Computational Demand

Energy Demand Where Relevant

Infrastructure Cost Exposure

Performance improvements in one component may produce increased consumption elsewhere.

User and External Stakeholder Impact

Software changes may affect users and external parties.

Relevant elements include:

User Functionality

Access Conditions

Service Availability

Compatibility Requirements

Required User Actions

External Integration Changes

Contractual Dependencies

Operational Communication Needs

The assessment must distinguish direct user effects from consequences mediated through dependent services.

Physical-System Impact

Where software controls or influences physical systems, CIAM must evaluate material physical-world consequences.

Examples include:

Robotic Movement

Industrial Equipment Control

Vehicle Behavior

Sensor Interpretation

Actuator Commands

Physical Safety Conditions

Equipment Availability

Emergency Stop Behavior

Software verification alone may be insufficient to establish acceptable physical-system behavior.

Autonomous System Impact

Changes affecting autonomous systems may alter decision-making or execution behavior.

Relevant dimensions include:

Agent Objectives

Decision Logic

Tool Selection

Permission Interpretation

Delegation Boundaries

Memory Access

External Actions

Human Override

Escalation Behavior

Operational Autonomy

A change to an agent's instructions or tool configuration may have consequences beyond the apparent size of the modification.

Authority and Permission Impact

CIAM must identify whether a proposed change affects legitimate authority or permissions.

Relevant elements include:

Actor Permissions

Service Accounts

Agent Tool Access

Delegation Conditions

Execution Boundaries

Approval Requirements

Human Intervention Rights

External System Access

Audit Requirements

Changes to authority-related software behavior require appropriate

governance references beyond technical implementation.

Reversibility Impact

CIAM must evaluate whether the consequences of the change can be reversed.

Relevant elements include:

Artifact Reversion

Configuration Reversion

Dependency Reversion

Data Reversion

Schema Compatibility

External Side Effects

Infrastructure State

Physical-System Consequences

A change may be reversible at the source code level while remaining irreversible at the operational level.

Rollback and Recovery Impact

CIAM must identify consequences affecting future rollback and recovery.

Relevant considerations include:

Previous Artifact Compatibility

Data Restoration Requirements

Backup Applicability

Dependency Availability

Configuration Compatibility

Recovery Execution Complexity

Potential Service Interruption

Recovery Verification

Irreversible Consequences

SDPS and its Rollback & Recovery Module govern the applicable deployment

recovery mechanisms.

CIAM identifies the change-related implications.

Concurrent Change Impact

Multiple proposed or active changes may interact.

Relevant elements include:

Shared Software Objects

Overlapping Dependencies

Concurrent Configuration Changes

Competing Interface Changes

Shared Data Structures

Release Coordination

Deployment Sequencing

Combined Verification Requirements

CIAM must not assume that individually acceptable changes remain acceptable when combined.

Cumulative Change Impact

A sequence of individually limited changes may create a material cumulative effect.

Examples include:

Progressive Dependency Drift

Accumulated Configuration Changes

Repeated Permission Expansion

Gradual Performance Degradation

Increasing Integration Complexity

Erosion of Recovery Compatibility

Accumulated Technical Constraints

Cumulative impact must be considered where the assessment evidence indicates that prior changes materially affect the proposed modification.

Change Interaction Assessment

CIAM must evaluate material interactions among changes.

Relevant relationships include:

Compatible

Conditionally Compatible

Dependent

Conflicting

Sequence-Sensitive

Unverified

Unknown

An interaction may require coordinated implementation, additional testing, or a revised lifecycle pathway.

Impact Feasibility

Impact feasibility concerns whether the proposed change can be implemented and verified under applicable conditions.

Relevant factors include:

Technical Capability

Required Expertise

Required Resources

Dependency Availability

Environment Compatibility

Verification Capability

Implementation Constraints

Operational Restrictions

Recovery Feasibility

External Dependencies

Feasibility must remain distinct from implementation authorization.

Alternative Change Strategies

Where material impact warrants comparison, CIAM may assess alternative approaches.

Potential alternatives include:

Proposed Modification

Different Technical Solution

Configuration Adjustment

Dependency Replacement

Temporary Mitigation

Functional Restriction

Deferred Change

Software Replacement

Retirement or Migration

Alternatives must be evaluated against the underlying change purpose rather than solely against implementation convenience.

Impact Evidence

CIAM must preserve evidence supporting material impact conclusions.

Potential evidence includes:

Dependency Graphs

Source Code Analysis

Test Results

Security Assessments

Configuration Comparisons

Performance Measurements

Operational Observations

Interface Specifications

Data Migration Analysis

Historical Incident Records

Simulation Results

Expert Assessments

Evidence must remain attributable to its source and applicable assessment scope.

Evidence Applicability

Existing evidence may support impact conclusions only where its assumptions remain applicable.

Relevant considerations include:

Software Version

Artifact Identity

Configuration

Dependency State

Test Environment

Operational Conditions

Evidence Age

Assessment Scope

Known Limitations

Previously Valid Evidence ≠ Automatically Applicable Evidence.

Impact Assumptions

CIAM must identify assumptions materially influencing the assessment.

Examples include:

Assumed Dependency Compatibility

Assumed Deployment Environment

Assumed Data State

Assumed User Behavior

Assumed Infrastructure Capacity

Assumed Recovery Capability

Assumed External Service Availability

Assumed Agent Behavior

Assumptions must not be presented as verified conditions.

Impact Uncertainty

CIAM must preserve uncertainty that materially affects assessment conclusions.

Potential uncertainty categories include:

Unknown Dependency

Incomplete Software Inventory

Unverified Interface Consumer

Insufficient Test Coverage

Unknown Operational Condition

Unverified Data Compatibility

Uncertain Recovery Feasibility

Conflicting Evidence

Unknown Physical-System Consequence

An absence of identified impact must not be treated as proof of impact absence when assessment coverage is insufficient.

Impact Coverage

Impact coverage concerns whether the assessment has examined the material

dimensions relevant to the proposed change.

Relevant elements include:

Known Affected Objects Covered

Known Dependencies Covered

Relevant Interfaces Covered

Security Implications Covered

Quality Implications Covered

Operational Conditions Covered

Recovery Implications Covered

Material External Effects Covered

Coverage must be assessed relative to the change's actual context.

Impact Significance

CIAM must classify material consequences according to justified assessment criteria.

Relevant factors include:

Severity

Likelihood Where Assessable

Affected Scope

Operational Criticality

Security Consequences

Data Consequences

Reversibility

Recovery Complexity

Uncertainty

Verification Difficulty

A numerical score may support classification but must not replace the underlying evidence and reasoning.

Impact Classification

CIAM may classify impact using the following governance states:

Impact Identified

Impact Under Assessment

Limited Impact Supported

Material Impact Identified

Critical Impact Identified

Indirect Impact Identified

Combined Impact Identified

Impact Conditionally Acceptable

Impact Unacceptable Under Current Conditions

Assessment Incomplete

Evidence Insufficient

Impact Conflicted

Reassessment Required

Unknown

Classification must preserve its assessment basis and scope.

Verification Implications

CIAM must identify the verification activities materially required by the change.

Potential requirements include:

Requirements Verification

Functional Testing

Regression Testing

Integration Testing

Security Testing

Performance Testing

Configuration Verification

Dependency Compatibility Testing

Data Integrity Verification

Operational Verification

Physical-System Validation Where Applicable

CIAM identifies the verification implications.

The applicable specialized standards govern their execution and acceptance.

Lifecycle Governance Implications

CIAM must identify which SLGA™ governance mechanisms may require renewed or additional activity.

Potential implications include:

Requirements Reassessment

Software Creation

Dependency Assessment

Configuration Revision

Quality Verification

Security Verification

Release Reassessment

Deployment Reassessment

Recovery Planning

Retirement or Migration Assessment

These implications provide input to Lifecycle Re-entry Control Module

(LRECM).

CIAM does not independently establish completion of lifecycle re-entry.

Impact Reassessment Triggers

Impact reassessment may be required when:

Change Scope Expands

Proposed Solution Changes

New Dependencies Are Identified

Implementation Deviates

Verification Fails

Security Findings Emerge

Operational Conditions Change

Concurrent Changes Introduce New Interactions

Recovery Feasibility Changes

New Evidence Contradicts Earlier Conclusions

Reassessment must preserve earlier assessment versions and identify which conclusions remain applicable.

Assessment Continuity

CIAM must preserve the continuity of impact assessment through change evolution.

Relevant elements include:

Original Assessment

Revised Assessment

Changed Assumptions

Newly Identified Consequences

Resolved Uncertainty

New Uncertainty

Changed Significance

Changed Verification Requirements

Changed Lifecycle Implications

Historical assessment conclusions must not be silently overwritten.

Governance Requirements

A conforming CIAM implementation must satisfy the following requirements. Assessment Identity and Scope

• Establish a stable impact assessment identity

- Associate the assessment with an identifiable change request
- Preserve the applicable request version
- Establish assessment purpose and scope
- Identify the current software state
- Identify the intended software state
- Identify assessment boundaries
- Preserve assessment limitations

Direct and Indirect Consequences

- Identify directly affected software objects
- Evaluate material indirect effects
- Assess known dependency propagation
- Identify configuration and artifact implications
- Evaluate functional consequences
- Evaluate quality consequences
- Evaluate security consequences
- Evaluate data and interface consequences
- Evaluate operational consequences
- Identify physical-system consequences where applicable

Interaction and Feasibility

- Identify relevant concurrent changes
- Evaluate material change interactions
- Consider cumulative impact where justified
- Evaluate implementation feasibility
- Identify reversibility limitations
- Identify rollback and recovery implications
- Preserve material alternatives where relevant
- Identify resource and capability constraints

Evidence and Classification

- Preserve impact evidence
- Identify evidence provenance
- Evaluate evidence applicability
- Preserve material assumptions
- Preserve uncertainty
- Evaluate assessment coverage

• Establish impact significance

• Preserve the basis for impact classification

• Distinguish assessed consequences from verified outcomes

Lifecycle and Continuity

• Identify required verification activities

• Identify affected lifecycle governance mechanisms

• Preserve applicable specialized assessment references

• Establish reassessment triggers

• Preserve material assessment revisions

• Provide structured impact outputs to CHCM and LRECM

• Prevent impact assessment conclusions from being treated as

implementation authorization.

Governance Logic

Change Request → Assessment Identity → Current-State Baseline →

Intended-State Reference → Direct Change Surface → Dependency Relationships → Indirect Impact Propagation → Functional Impact → Configuration and Artifact Impact → Quality Impact → Security Impact → Data and Interface Impact → Operational Impact → Reversibility and Recovery Impact → Concurrent Change Interaction → Evidence Assessment → Coverage and Uncertainty → Impact Significance → Verification Requirements → Lifecycle Implications → Software Change Impact Profile

Where material impact is uncertain:

Impact Question → Evidence Gap → Additional Analysis → Updated Assessment

/ Unresolved Uncertainty

Where consequences extend beyond the initial scope:

Direct Impact → Propagation Path → Additional Affected Object → Expanded

Assessment → Revised Impact Classification

Where concurrent changes interact:

Change A + Change B → Shared Dependency or Condition → Combined Impact →

Coordination Requirement

Where new evidence changes the assessment:

Previous Impact Profile → New Evidence → Assumption Reassessment → Revised

Consequences → Updated Profile

CIAM must preserve the distinction between assessing a proposed change and authorizing its implementation.

Governance Outputs

The principal output is:

Software Change Impact Profile

Supporting outputs may include:

Impact Assessment Identity

Current–Intended State Comparison

Direct Impact Register

Indirect Impact Register

Impact Propagation Map

Dependency Impact Record

Configuration Impact Record

Quality Impact Record

Security Impact Record

Data and Interface Impact Record

Operational Impact Record

Infrastructure Impact Record

Reversibility Assessment

Recovery Implication Record

Concurrent Change Interaction Record

Impact Evidence Register

Assumption Register

Uncertainty Register

Impact Coverage Assessment

Impact Significance Classification

Verification Requirement Profile

Lifecycle Governance Implication Profile

Impact Reassessment Record

Minimum Implementation Framework

A minimum CIAM implementation must establish eight governance layers. Layer 1 — Assessment Identity

Identify the change request, applicable request version, software product,

assessment scope, and responsible assessment function. Layer 2 — State Comparison

Establish the relevant current software state, intended state, and proposed

modification. Layer 3 — Direct Impact

Identify the software objects, functions, artifacts, configurations, and

conditions directly affected. Layer 4 — Indirect Impact

Evaluate dependency propagation, interfaces, data relationships, external

systems, and other material indirect consequences. Layer 5 — Consequence Assessment

Evaluate quality, security, operational, resource, reversibility, recovery,

and combined-change implications. Layer 6 — Evidence and Uncertainty

Preserve supporting evidence, assumptions, assessment coverage,

limitations, and unresolved material questions. Layer 7 — Governance Implications

Establish impact significance, verification requirements, coordination

implications, and affected lifecycle mechanisms. Layer 8 — Governed Output and Continuity

Establish the Software Change Impact Profile, preserve reassessment

triggers, and provide structured outputs to CHCM and LRECM.

The implementation may be manual, tool-assisted, automated, or AI-supported.

The methodological requirements remain independent of the assessment

technology.

Cross-Module Boundary

CRMM → CIAM

CRMM establishes the change request identity, origin, purpose, preliminary

scope, and supporting evidence.

CIAM evaluates the substantive consequences of the proposed modification.

CIAM → CHCM

CIAM identifies affected objects, dependencies, interactions, risks, and

coordination implications.

CHCM governs the coordination of actors, decisions, activities, and related

changes.

CIAM → LRECM

CIAM identifies affected lifecycle mechanisms and required verification

implications.

LRECM determines and controls the applicable lifecycle re-entry pathway.

CIAM → CCLM

CIAM establishes expected consequences and verification implications.

CCLM uses relevant assessment references when evaluating the resulting change outcome and unresolved obligations.

Impact Assessed ≠ Change Authorized.

Cross-Architecture Boundary

CIAM identifies change-related consequences across SLGA™ but does not replace specialized lifecycle governance.

SRS — Software Requirements Standard

Requirements impact and traceability.

SCS — Software Creation Standard

Software artifact and development implications.

SDS — Software Dependency Standard

Dependency identity, suitability, compatibility, and continuity.

SCFS — Software Configuration Standard

Configuration baselines, builds, artifact identities, and reproducibility.

SQS — Software Quality Standard

Quality requirements, testing, defects, and quality assessment.

SSS — Software Security Standard

Security exposure, vulnerabilities, and remediation.

SRLS — Software Release Standard

Release readiness, authorization, and release records.

SDPS — Software Deployment Standard

Deployment conditions, production versions, monitoring, rollback, and

recovery.

SRTS — Software Retirement Standard

Migration, retirement, and residual dependency implications.

SLCS — Software Lifecycle Control Standard

Lifecycle transitions, exceptions, state consistency, and continuity.

Applicable OOF® authority, accountability, autonomous systems, and

operational reality governance references may also be required when a change affects those domains.

Practical Applications

Enterprise Software Maintenance

Assessing modifications across interconnected services, shared libraries,

and production environments.

Dependency Modernization

Identifying compatibility, security, configuration, and operational

consequences of dependency upgrades.

AI-Assisted Development

Evaluating AI-generated code changes beyond their immediate functional

correctness.

Autonomous Software Operations

Assessing how agent-generated changes may alter permissions, tool behavior, infrastructure interactions, or downstream execution.

Critical Infrastructure Software

Identifying software change consequences for service continuity, recovery

capability, and physical-system operation.

Robotics and Embedded Systems

Assessing firmware and control logic modifications that may affect sensors, actuators, or operational safety.

Use Case 1 — A One-Line Permission Change Creates System-Wide Security

Exposure

A developer proposes a one-line modification to an authorization function.

The modification is intended to resolve an access problem affecting a limited user group.

Local tests confirm that the affected users can access the required function.

However, the modified condition also grants access to other users who

previously lacked permission.

CIAM evaluates:

Change Request → Direct Code Modification → Authorization Logic → Affected

User Groups → Security Impact → Verification Requirements → Lifecycle Implications

The assessment identifies:

• The source code modification is small

• The intended functional correction is achieved in local testing

• The modification changes the effective permission boundary

• Additional users may receive unintended access

• Security assessment and broader authorization testing are required

The appropriate classification is:

Material Security Impact Identified — Additional Verification Required.

One-Line Modification ≠ Limited Governance Impact.

• Use Case 2 — A Robotic Firmware Update Alters Physical Behavior Through

an Indirect Dependency

A robotics manufacturer proposes a firmware update to improve sensor

processing efficiency.

The updated sensor library passes its component-level tests.

However, the update changes the timing characteristics of sensor data

delivered to the movement-control system.

The movement-control software depends on timing assumptions established

under the previous firmware.

CIAM evaluates:

Firmware Change → Sensor Processing → Timing Dependency → Movement Control

→ Physical-System Consequences → Verification Requirements

The assessment identifies:

• The direct change affects sensor processing

• The movement-control system is indirectly affected

• Existing timing assumptions may no longer remain valid

• Component-level testing does not establish complete system

• Integration and physical-system verification are required before the

compatibility. impact can be considered sufficiently assessed.

The appropriate classification is:

Material Indirect Impact Identified — System-Level Verification Required.

Component Test Success ≠ Physical-System Compatibility.

Critical Distinctions

Modification Size ≠ Impact Significance.

Local Correctness ≠ System-Level Compatibility.

Known Direct Impact ≠ Complete Change Impact.

Proposed State ≠ Verified Resulting State.

Directly Modified Object ≠ Only Affected Object.

Limited Change Scope ≠ Limited Operational Consequences.

Dependency Identified ≠ Dependency Impact Fully Understood.

No Known Impact ≠ Verified Absence of Impact.

Existing Test Coverage ≠ Sufficient Change Verification.

Previously Valid Evidence ≠ Automatically Applicable Evidence.

Individual Change Acceptability ≠ Combined Change Compatibility.

Source Code Reversibility ≠ Operational Reversibility.

Assessment Completed ≠ Material Uncertainty Resolved.

Impact Classification ≠ Implementation Authorization.

Feasible Change ≠ Legitimately Authorized Change.

Impact Assessed ≠ Change Authorized.

Canonical Boundary

CIAM governs the identification, assessment, classification, and

evidence-supported interpretation of the direct, indirect, combined, and evolving consequences of proposed software changes.

It establishes the relationship between a change request, current and

intended software states, affected objects, impact propagation, material consequences, assumptions, uncertainty, verification requirements, and lifecycle governance implications.

CIAM does not independently authorize implementation, execute software modifications, replace specialized quality or security verification, or establish final operational acceptance.

Its principal governance output is the:

Software Change Impact Profile

This output provides the evidence-supported basis for change coordination,

lifecycle re-entry, and subsequent change outcome assessment.

Canonical Closing Statement

A software change cannot be assessed reliably by examining only the

component being modified. Its significance depends on the relationships, conditions, and operational consequences that the modification may alter.

A one-line permission change may create system-wide security exposure.

A dependency update may affect services that the requester does not directly control.

A configuration adjustment may invalidate earlier quality evidence.

A database migration may prevent safe rollback.

An AI-generated patch may pass local tests while introducing new operational risks.

And a firmware modification may alter physical behavior through dependencies that remain invisible at the source code level.

CIAM establishes the governance structure required to identify and assess

these consequences.

It requires change impact to be evaluated across relevant software objects,

dependencies, interfaces, quality conditions, security requirements, operational environments, and recovery constraints.

It preserves direct and indirect effects, evidence limitations, material

assumptions, and unresolved uncertainty.

It recognizes that individually acceptable changes may produce unacceptable combined consequences.

It identifies the lifecycle governance mechanisms that must be revisited without replacing their specialized responsibilities.

And it ensures that impact conclusions remain traceable and subject to reassessment when new evidence or changing conditions invalidate earlier assumptions.

The purpose of software change impact assessment is not merely to predict whether a modification will work. It is to establish what the modification may affect, how its consequences may propagate, which conditions require verification, and which governance obligations must be satisfied before the resulting software state can be considered acceptable.

Change Impact Assessment Module (CIAM) transforms software change analysis

from a local technical estimate into an evidence-supported governance assessment of system-wide consequences, uncertainty, and lifecycle obligations.

Modul3

Change Coordination Control Module (CHCM)

Architecture

SLGA™ — Software Lifecycle Governance Architecture

Parent Standard

Software Change Management Standard (SCMS)

Category

Governance & Enforcement

Subcategory

Software Change Management Governance

Type

Core Module

Governed Space

Software Change Coordination Governance

Module ID

SLGA-SCMS-CHCM-03

Origin ID

OOF-SLGA-SCMS-CHCM

Version

1.0

Status

Canonical Open

Effective Date

10.2026

Compatibility

Methodology OS — Modular Compatibility

GIE-Compatible: Yes

AI-Readable: Yes

Authority

OOF® — OriginOpen® Foundation

Protection

Protected Methodological Work

Canonical Language

English

Canonical Definition

Change Coordination Control Module (CHCM) is the Core Module within Software Change Management Standard (SCMS) that governs how multiple actors, software changes, dependencies, decisions, implementation activities, operational conditions, and lifecycle governance requirements are coordinated when their relationships may affect the legitimacy, consistency, sequencing, or outcome of software modifications.

CHCM establishes change coordination as a governed relationship between

identifiable change cases and the actors, resources, dependencies, conditions, and activities required to process them without creating unmanaged conflicts or unsupported assumptions of compatibility.

The module distinguishes communication from coordination, coordination from

authorization, individual change readiness from combined change readiness, and planned implementation sequences from verified execution conditions.

It governs the identification of shared software objects, overlapping

changes, prerequisite relationships, conflicting modifications, resource constraints, coordination responsibilities, execution dependencies, decision boundaries, and material changes to coordination conditions.

CHCM recognizes that individually justified and technically valid changes may become incompatible when executed together, that implementation order can alter operational consequences, and that coordination failures may arise even when each participating actor performs its assigned task correctly.

It applies to human-operated development processes, distributed engineering

teams, continuous integration and delivery systems, cloud-native environments, automated change pipelines, AI-assisted software engineering, autonomous software agents, industrial systems, and robotic platforms.

Its principal governance output is the Software Change Coordination Profile.

Operational Role

CHCM establishes the coordination governance layer between change impact

assessment and lifecycle re-entry.

It governs:

• Change coordination identity

• Coordination scope

• Participating change cases

• Actor and responsibility references

• Shared software objects

• Interdependent modifications

• Concurrent changes

• Change sequencing

• Prerequisite conditions

• Resource and environment conflicts

• Compatibility relationships

• Decision coordination

• Authorization boundaries

• Implementation scheduling dependencies

• Integration requirements

• Coordination checkpoints

• Automated execution coordination

• Autonomous agent interactions

• Coordination evidence

• Conflict detection and resolution

• Coordination uncertainty

• Material reassessment

• Coordination continuity

• Handoff to lifecycle re-entry

CHCM does not independently authorize software implementation, replace change impact assessment, execute technical changes, or establish final change closure.

Operational Question

How can multiple software changes, actors, dependencies, decisions, and implementation activities remain coordinated when their execution order, shared resources, authority boundaries, compatibility conditions, and

operational consequences may interact or change, while ensuring that conflicts, unresolved prerequisites, and coordination failures remain identifiable and governable?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

SCMS Core Module Flow

Change Request Management Module (CRMM) → Change Impact Assessment Module

(CIAM) → Change Coordination Control Module (CHCM) → Lifecycle Re-entry Control Module (LRECM) → Change Closure Management Module (CCLM)

CHCM is the third Core Module within SCMS.

It receives identifiable change requests from CRMM and assessed

consequences from CIAM.

It establishes coordination conditions for related changes, participating

actors, implementation dependencies, and applicable governance decisions.

Its outputs support LRECM in determining the practical coordination

requirements associated with affected lifecycle pathways.

Coordination may continue during implementation, verification, deployment, and recovery when material dependencies remain active.

• Governed Space

Software Change Coordination Governance

This governed space concerns the coordination of software change cases

whose relationships may affect implementation consistency, governance decisions, resource availability, compatibility, or operational outcomes.

It begins when a software change requires coordination with another actor, change, dependency, lifecycle activity, operational condition, or decision process.

It continues through coordination planning, dependency establishment, compatibility evaluation, sequencing, conflict management, execution alignment, reassessment, and coordination closure.

The governed space includes both planned coordination and emerging

interactions discovered during implementation or operation.

It does not replace organizational project management or the specialized governance of individual software lifecycle activities.

Governed Object

The primary Governed Object is:

A Software Change Coordination Case representing the evidence-supported

relationship between identifiable software changes, participating actors, affected software objects, shared dependencies, coordination requirements, execution sequences, applicable authority references, material conflicts, and the conditions necessary for compatible lifecycle processing.

The governed object must preserve, where applicable:

Coordination Case ID

Related Change Request IDs

Software Product References

Participating Actor References

Shared Software Objects

Dependency Relationships

Coordination Scope

Sequence Requirements

Compatibility Conditions

Resource Constraints

Authority References

Coordination Checkpoints

Conflict References

Coordination Evidence

Coordination Status

Coordination Version

Scope

CHCM governs:

Coordination Case Formation

Participating Change Identification

Actor Coordination

Responsibility Interfaces

Shared Software Object Coordination

Change Dependency Coordination

Concurrent Change Coordination

Implementation Sequence Control

Resource Coordination

Environment Coordination

Compatibility Coordination

Decision Coordination

Authority Boundary Preservation

Integration Coordination

Execution Checkpoints

Conflict Detection

Conflict Resolution

Coordination Evidence

Coordination Reassessment

Coordination Continuity

Lifecycle Re-entry Handoff

CHCM does not prescribe a particular scheduling platform, project management tool, team structure, repository strategy, or software delivery methodology.

Foundational Principle

Software changes must be coordinated according to their actual relationships and shared operational consequences, not merely according to the independent readiness of their implementation teams.

Two changes may be individually acceptable but mutually incompatible.

Two implementation actors may possess legitimate authority over separate components while lacking authority to alter a shared dependency.

A technically correct change sequence may fail when its prerequisite conditions are not established.

An automated pipeline may complete its assigned task while invalidating another active change.

Individual Change Readiness ≠ Combined Change Readiness.

Communication Completed ≠ Coordination Established.

Planned Sequence ≠ Verified Execution Sequence.

Coordination Case Identity

Every governed coordination case must have a stable identity.

Relevant elements include:

Coordination Case ID

Related Change Request IDs

Software Product References

Coordination Origin

Creation Time

Responsible Coordination Function

Coordination Scope

Current Status

Evidence Reference

Coordination Version

A coordination case may include multiple changes and revisions without losing its historical identity.

Coordination Trigger

Coordination may be required when:

Multiple Changes Affect the Same Component

Changes Share Dependencies

Implementation Activities Overlap

Release or Deployment Conditions Interact

Multiple Teams Modify Shared Interfaces

Resource Constraints Affect Execution

A Change Depends on Another Change

Conflicting Requirements Are Identified

Automated Systems Execute Related Changes

Autonomous Agents Operate on Shared Resources

Operational Conditions Require Joint Action

A coordination trigger must be associated with an identifiable relationship or material governance concern.

Coordination Purpose

Every coordination case must establish its purpose.

Potential purposes include:

Prevent Conflicting Modifications

Establish Implementation Order

Maintain Dependency Compatibility

Coordinate Shared Resources

Align Verification Activities

Preserve Authority Boundaries

Prevent Concurrent Deployment Conflicts

Coordinate Emergency Changes

Support Lifecycle Re-entry

Preserve Operational Continuity

The coordination purpose must describe the required relationship or outcome rather than merely identify a meeting or communication activity.

Coordination Scope

CHCM must define the scope of coordination.

Relevant dimensions include:

Participating Changes

Affected Software Products

Shared Components

Dependencies

Configuration

Build Artifacts

Testing Activities

Release Conditions

Deployment Environments

Operational Resources

Participating Actors

Decision Processes

Scope must reflect material interactions identified through impact assessment and subsequent evidence.

Coordination Boundary

The coordination boundary identifies the relationships and actions governed

by the coordination case.

Relevant boundaries include:

Change Case Boundary

Software Object Boundary

Actor Boundary

Authority Boundary

Resource Boundary

Environment Boundary

Execution Boundary

Temporal Boundary

External System Boundary

Operational Boundary

Coordination participation must not be treated as permission to perform actions outside an actor's legitimate authority.

Participating Change Identification

CHCM must identify all changes materially involved in the coordination case.

Relevant elements include:

Change Request ID

Impact Assessment Reference

Affected Software Objects

Proposed Modification

Change Status

Implementation Constraints

Required Lifecycle Activities

Known Dependencies

Known Conflicts

The relationship between participating changes must remain traceable.

Participating Actor Identification

CHCM must identify relevant actors and functions.

Potential participants include:

Software Developers

Software Maintainers

Quality Engineers

Security Functions

Configuration Managers

Release Managers

Deployment Operators

Infrastructure Teams

External Providers

Automated Pipelines

AI Software Agents

Autonomous Systems

Actor identity must remain distinct from assigned coordination responsibility and implementation authority.

Coordination Responsibility

CHCM must establish which functions are responsible for coordination activities.

Relevant elements include:

Coordination Function

Assigned Responsibilities

Affected Change Cases

Required Decisions

Communication Obligations

Escalation Responsibilities

Evidence Requirements

Continuity Arrangements

Coordination responsibility does not automatically establish authority to approve or execute every participating change.

Shared Software Object Identification

Multiple changes may affect the same software object.

Relevant objects include:

Source Components

Shared Libraries

Interfaces

Database Schemas

Configuration Files

Build Definitions

Runtime Services

Deployment Manifests

Automation Policies

Agent Tool Configurations

Shared-object relationships must be evaluated for compatibility, sequencing, and potential conflicts.

Change Relationship Identification

CHCM must identify material relationships among changes.

Potential relationships include:

Independent

Related

Dependent

Prerequisite

Blocking

Overlapping

Compatible

Conditionally Compatible

Conflicting

Sequence-Sensitive

Mutually Exclusive

Unknown

A relationship may change when new evidence or implementation conditions emerge.

Change Dependency Coordination

Some changes require other changes to be completed or verified first.

Relevant elements include:

Prerequisite Change

Dependent Change

Required State

Completion Evidence

Dependency Validity

Failure Condition

Alternative Pathway

Reassessment Trigger

A dependency must not be treated as satisfied solely because its associated work item is marked complete.

Implementation Sequence

CHCM must identify when implementation order materially affects the outcome.

Relevant elements include:

Required First Change

Required Subsequent Change

Intermediate Compatibility State

Prerequisite Verification

Permitted Parallel Execution

Sequence Restrictions

Failure Response

Sequence Evidence

The sequence must reflect actual dependencies and operational conditions.

Sequence Feasibility

A planned sequence must be evaluated for feasibility.

Relevant considerations include:

Artifact Availability

Dependency Compatibility

Resource Availability

Environment Readiness

Required Actors

Verification Capacity

Operational Restrictions

Recovery Capability

A theoretically valid sequence may remain infeasible under current operational conditions.

Parallel Change Execution

CHCM may permit parallel execution where material compatibility and resource conditions support it.

Relevant elements include:

Independent Change Scope

Shared Resource Exposure

Dependency Independence

Concurrent Modification Risk

Integration Requirements

Verification Requirements

Rollback Interaction

Coordination Monitoring

Parallel execution must not be justified solely by the absence of a known conflict.

Concurrent Modification Conflict

Concurrent modifications may produce conflicts.

Examples include:

Two Changes Modify the Same Interface

Two Pipelines Modify the Same Configuration

Two Agents Update a Shared Dependency

A Database Migration Conflicts With a Service Update

A Security Change Invalidates Another Change's Access Requirements

A Deployment Replaces an Artifact Required by Another Process

Material conflicts must remain identifiable and subject to appropriate resolution.

Dependency Compatibility Coordination

CHCM must coordinate compatibility conditions involving shared dependencies.

Relevant elements include:

Required Dependency Versions

Supported Version Ranges

Transitive Dependency Effects

Interface Compatibility

Runtime Compatibility

Migration Requirements

Dependency Availability

Verification Evidence

Dependency suitability remains governed by SDS.

CHCM governs the coordination relationships among affected changes.

Configuration Coordination

Multiple changes may alter shared configuration conditions.

Relevant elements include:

Configuration Baseline

Affected Parameters

Shared Configuration Objects

Environment-Specific Values

Change Sequence

Configuration Compatibility

Conflict Evidence

Restoration Requirements

Configuration coordination must preserve applicable SCFS governance references.

Build and Artifact Coordination

CHCM must identify material interactions among build and artifact activities.

Relevant elements include:

Shared Build Inputs

Artifact Dependencies

Build Sequence

Artifact Identity

Version Compatibility

Build Environment

Reproducibility Conditions

Release Candidate Dependencies

One change must not silently invalidate the artifact assumptions of another.

Quality Verification Coordination

Related changes may require coordinated verification.

Relevant elements include:

Shared Test Environments

Integration Tests

Regression Tests

Combined Behavior Tests

Test Data

Verification Sequence

Evidence Reuse

Conflicting Results

Individually successful tests do not establish combined compatibility.

Security Coordination

Changes may interact through shared security conditions.

Relevant elements include:

Authentication

Authorization

Credentials

Access Policies

Security Configuration

Dependency Exposure

Security Testing

Incident Remediation

Security-related coordination must preserve applicable SSS governance requirements.

Data Change Coordination

Changes affecting shared data structures require coordination.

Relevant elements include:

Database Schema

Data Migration

Data Producers

Data Consumers

Data Compatibility

Migration Sequence

Restoration Conditions

Data Integrity Verification

Uncoordinated data changes may create consequences that cannot be reversed through software artifact rollback alone.

Interface Coordination

CHCM must identify changes affecting shared interfaces.

Relevant elements include:

API Contracts

Message Formats

Communication Protocols

Version Compatibility

External Consumers

Integration Timing

Fallback Behavior

Deprecation Conditions

Interface coordination must account for affected parties outside the direct implementation team where applicable.

Resource Coordination

Multiple changes may depend on shared resources.

Relevant resources include:

Development Environments

Testing Infrastructure

Build Systems

Deployment Pipelines

Production Infrastructure

Database Resources

Network Capacity

Specialized Personnel

External Service Capacity

Resource availability must be assessed against concurrent obligations and operational constraints.

Environment Coordination

CHCM must preserve coordination requirements across software environments.

Potential environments include:

Development

Integration

Testing

Staging

Production

Disaster Recovery

Simulation

Physical-System Test Environment

Environment-specific compatibility and authority conditions must remain identifiable.

Release Coordination

Multiple changes may affect the formation or readiness of software releases.

Relevant elements include:

Release Candidate References

Included Changes

Excluded Changes

Dependency Compatibility

Verification Completion

Release Sequence

Release Restrictions

Release Authorization References

SRLS governs release authorization and records.

CHCM governs coordination among changes affecting those processes.

Deployment Coordination

CHCM must identify material interactions among deployment-related changes.

Relevant elements include:

Deployment Environments

Deployment Windows

Production Version Dependencies

Infrastructure Constraints

Traffic Routing

Service Continuity

Monitoring Requirements

Rollback Compatibility

Recovery Coordination

SDPS governs deployment execution and operational verification.

CHCM governs coordination dependencies among the participating changes.

Operational Continuity Coordination

Software changes may affect operational continuity.

Relevant elements include:

Service Availability

Critical Functions

Redundancy

Failover Conditions

Recovery Resources

Operational Restrictions

User Impact

External Dependencies

Coordination must account for the combined consequences of implementation activities.

Decision Coordination

Different changes may require decisions from different competent functions.

CHCM must preserve:

Required Decision

Decision Actor

Authority Reference

Affected Change

Decision Dependency

Decision Status

Decision Conditions

Evidence

Coordination of decisions does not transfer decision authority between actors.

Authority Boundary Preservation

CHCM must distinguish coordination participation from legitimate authority.

Relevant elements include:

Actor Identity

Assigned Coordination Role

Decision Authority

Implementation Authority

Permitted Software Objects

Permitted Environments

Delegation Conditions

Restrictions

Coordination Participation ≠ Implementation Authorization.

Cross-Actor Execution Boundary

Where multiple actors perform related changes, CHCM must identify boundaries between their actions.

Relevant elements include:

Assigned Tasks

Permitted Resources

Required Handoffs

Shared Dependencies

Execution Preconditions

Decision Boundaries

Intervention Rights

Escalation Requirements

An actor's authority over one component does not automatically extend to shared resources or dependent systems.

Automated Pipeline Coordination

Automated pipelines may execute related changes without direct human intervention.

CHCM must preserve:

Pipeline Identity

Assigned Change Case

Execution Scope

Shared Resources

Prerequisite Conditions

Sequence Constraints

Conflict Detection

Stop Conditions

Execution Evidence

Escalation Requirements

Automated execution must not bypass material coordination requirements merely because each pipeline is independently authorized.

Autonomous Agent Coordination

Autonomous agents may propose or execute changes affecting shared software resources.

CHCM must identify:

Agent Identity

Assigned Objective

Affected Software Objects

Permitted Tools

Shared Resources

Coordination Requirements

Execution Dependencies

Conflict Conditions

Human Override

Escalation Requirements

Multiple agents operating under individually legitimate objectives may still create incompatible outcomes.

Agent Tool-Path Coordination

A change may require an autonomous agent to use multiple tools and execution paths.

Relevant elements include:

Objective

Agent Authority

Tool Selection

Repository Access

Dependency Modification

Configuration Modification

Build Execution

Deployment Action

Evidence Collection

Resulting State

Each material tool action must remain within applicable authority and coordination boundaries.

Task Authority ≠ Constituent Action Authority.

Coordination Checkpoints

CHCM must establish checkpoints where material dependencies or conflicts require verification.

Relevant elements include:

Checkpoint ID

Required Condition

Participating Changes

Responsible Function

Evidence Requirement

Decision Reference

Permitted Continuation

Failure Response

Checkpoints may occur before implementation, during integration, before release, or before deployment.

Coordination Preconditions

A coordinated change activity may proceed only when applicable prerequisites are satisfied.

Potential prerequisites include:

Required Change Completed

Dependency Compatible

Shared Resource Available

Environment Ready

Required Verification Completed

Authority Applicable

Conflict Resolved

Recovery Capability Established

Monitoring Available

Unverified prerequisites must remain distinguishable from satisfied conditions.

Coordination Condition States

CHCM recognizes the following condition states:

Satisfied

Not Satisfied

Partially Satisfied

Unknown

Not Applicable

Exception

Each condition must retain supporting evidence or an applicable exception reference.

Unknown ≠ Satisfied.

Scheduled ≠ Ready.

Reported Complete ≠ Verified Prerequisite.

Coordination Conflict

A coordination conflict exists when material requirements, dependencies, actions, resources, or decisions cannot simultaneously be satisfied under the proposed conditions.

Examples include:

Incompatible Software Versions

Conflicting Configuration Changes

Competing Resource Requirements

Contradictory Execution Sequences

Overlapping Unauthorized Actions

Incompatible Release Conditions

Conflicting Deployment Requirements

Unresolved Authority Boundaries

A conflict must be associated with identifiable evidence and affected change cases.

Conflict Classification

CHCM may classify conflicts as:

Software Object Conflict

Dependency Conflict

Configuration Conflict

Interface Conflict

Resource Conflict

Sequence Conflict

Authority Conflict

Verification Conflict

Release Conflict

Deployment Conflict

Operational Conflict

Evidence Conflict

Multiple classifications may apply to one conflict.

Conflict Resolution

Conflict resolution may require:

Change Scope Revision

Implementation Resequencing

Dependency Adjustment

Configuration Reconciliation

Resource Reallocation

Additional Verification

Change Separation

Change Consolidation

Alternative Implementation Strategy

Escalation

Deferral

Rejection

Resolution must remain supported by the applicable governance evidence and decisions.

Conflict Escalation

Escalation is required when a conflict cannot be resolved within the applicable coordination authority or conditions.

Relevant elements include:

Conflict ID

Affected Changes

Unresolved Conditions

Required Decision

Competent Authority

Operational Consequences

Time Sensitivity

Evidence

Interim Restrictions

Escalation must not be treated as permission to continue conflicting actions.

Coordination Failure

Coordination failure occurs when required relationships or conditions cannot be maintained.

Examples include:

Prerequisite Not Established

Execution Order Violated

Shared Resource Unavailable

Conflicting Changes Executed

Required Decision Missing

Integration Verification Failed

Agent Exceeds Coordination Boundary

Operational Conditions Become Unacceptable

Coordination failure must trigger reassessment or appropriate intervention.

Coordination Uncertainty

CHCM must preserve uncertainty affecting coordination conclusions.

Examples include:

Unknown Dependency Relationship

Unverified Change Completion

Uncertain Resource Availability

Incomplete Actor Authority

Unknown Execution State

Unverified Compatibility

Missing Integration Evidence

Conflicting Status Reports

An absence of reported conflicts does not establish that coordination conditions are satisfied.

Coordination Evidence

CHCM must preserve evidence supporting material coordination conclusions.

Potential evidence includes:

Change Request References

Impact Assessment Profiles

Dependency Maps

Configuration Records

Actor Assignment Records

Authority References

Execution Plans

Checkpoint Results

Integration Test Evidence

Release and Deployment Records

Conflict Resolution Records

Operational Observations

Evidence must remain attributable to its source, relevant change cases, and observation time.

Planned–Actual Coordination Alignment

CHCM must distinguish planned coordination from actual execution conditions.

Relevant comparisons include:

Planned Participants ↔ Actual Participants

Planned Sequence ↔ Actual Sequence

Expected Dependencies ↔ Observed Dependencies

Planned Resources ↔ Available Resources

Expected Compatibility ↔ Verified Compatibility

Required Decisions ↔ Recorded Decisions

Planned Execution State ↔ Observed Execution State

Material differences must remain identifiable.

Coordination Monitoring

Coordination may require continued observation during execution.

Relevant elements include:

Change Progress

Prerequisite Completion

Resource Availability

Execution Sequence

Dependency Conditions

Conflict Indicators

Verification Results

Operational Deviations

Required Escalation

Monitoring must reflect the coordination conditions that materially affect the participating changes.

Material Coordination Change

A coordination case may require reassessment when:

A New Change Is Introduced

A Participating Change Is Withdrawn

Scope Expands

Dependencies Change

Implementation Order Changes

Resources Become Unavailable

Authority Changes

Verification Fails

Operational Conditions Change

An Autonomous Agent Introduces an Unplanned Action

Material changes must not silently inherit earlier coordination conclusions.

Coordination Reassessment

Reassessment must identify:

Previous Coordination Conditions

Changed Conditions

Affected Change Cases

Invalidated Assumptions

New Conflicts

Revised Sequence

Updated Prerequisites

Additional Decisions

Required Verification

Revised Coordination Status

Historical conclusions must remain traceable.

Coordination Continuity

CHCM must preserve coordination continuity when:

Responsible Actors Change

Change Ownership Transfers

Implementation Is Deferred

Automated Execution Replaces Manual Execution

Changes Are Consolidated or Separated

Execution Is Interrupted

Conflicts Are Reopened

Lifecycle Activities Are Repeated

Continuity requires the coordination case, participating changes,

decisions, evidence, and unresolved conditions to remain reconstructable.

Coordination Closure

Coordination closure requires an evidence-supported conclusion that the relevant coordination obligations have been resolved or appropriately transferred.

Relevant elements include:

Participating Changes

Required Coordination Conditions

Resolved Conflicts

Outstanding Dependencies

Execution Alignment

Verification References

Residual Coordination Obligations

Closure Decision

Closure Evidence

Coordination closure does not establish that every participating software change has achieved its final outcome.

Coordination Governance Classification

CHCM recognizes the following governance classifications:

Coordination Required

Coordination Case Established

Under Coordination Assessment

Dependencies Identified

Prerequisites Pending

Conflict Identified

Coordination Conditionally Established

Coordination Established

Execution Coordination Active

Coordination Deviation Identified

Coordination Paused

Reassessment Required

Evidence Insufficient

Coordination Failed

Escalation Required

Residual Obligations Outstanding

Coordination Closed

Unknown

Conflicted

Classification must remain associated with the relevant coordination case, time, scope, and supporting evidence.

Governance Requirements

A conforming CHCM implementation must satisfy the following requirements. Coordination Identity and Scope

• Establish a stable coordination case identity

• Identify participating change requests

• Preserve relevant impact assessment references

• Identify participating actors and functions

• Define coordination purpose

• Establish coordination scope and boundaries

• Identify shared software objects

• Preserve material coordination relationships

Dependencies and Compatibility

• Identify prerequisite changes

• Identify blocking dependencies

• Evaluate shared dependency compatibility

• Identify configuration interactions

• Identify interface interactions

• Evaluate concurrent change effects

• Establish sequence requirements

• Identify relevant resource constraints

• Preserve material compatibility evidence

• Identify unresolved dependency conditions

Authority and Execution

• Identify applicable coordination responsibilities

• Preserve decision authority references

• Preserve implementation authority boundaries

• Identify required execution handoffs

• Establish coordination checkpoints

• Establish execution prerequisites

• Preserve automated pipeline boundaries

• Preserve autonomous agent action boundaries

• Identify material stop or escalation conditions

• Prevent coordination participation from substituting for implementation authorization.

Conflict and Evidence

• Identify material coordination conflicts

• Classify conflicts according to their nature

• Preserve conflict evidence

• Establish appropriate resolution pathways

• Preserve escalation requirements

• Evaluate planned–actual coordination alignment

• Preserve material uncertainty

• Identify coordination failures

Continuity and Handoff

• Establish coordination monitoring where required

• Identify reassessment triggers

• Preserve material coordination revisions

• Maintain historical coordination continuity

• Establish coordination-state classification

• Preserve residual coordination obligations

• Establish evidence-supported coordination closure

• Provide structured coordination references to LRECM

• Preserve relevant references for CCLM

• Distinguish coordination completion from final change verification

Governance Logic

Change Requests → Impact Assessment References → Coordination Trigger →

Coordination Case Identity → Participating Changes → Participating Actors → Shared Software Objects → Dependency Relationships → Compatibility Assessment → Resource Conditions → Authority References → Implementation Sequence → Coordination Preconditions → Checkpoints → Conflict Assessment → Coordination Decision → Execution Alignment → Monitoring → Reassessment Where Required → Coordination Classification → Handoff / Closure

Where changes are compatible:

Related Changes → Compatibility Evidence → Sequence and Resource

Conditions → Coordination Established

Where prerequisites are unresolved:

Coordination Plan → Missing Prerequisite → Hold / Clarification /

Reassessment

Where conflicts are identified:

Conflicting Changes → Conflict Classification → Resolution Assessment →

Revised Coordination / Escalation / Deferral

Where automated systems act concurrently:

Automation A + Automation B → Shared Resource or Dependency → Execution

Boundary Assessment → Coordination Conditions → Permitted Execution / Restriction

Where actual execution differs materially from the plan:

Planned Coordination → Observed Deviation → Impact and Dependency

Reassessment → Revised Coordination Decision

CHCM must not classify coordination as established merely because participating actors have communicated or implementation schedules have been published.

Governance Outputs

The principal output is:

Software Change Coordination Profile

Supporting outputs may include:

Coordination Case Identity

Participating Change Register

Actor Coordination Register

Responsibility Interface Record

Shared Software Object Register

Change Relationship Map

Dependency Coordination Record

Compatibility Assessment Reference

Implementation Sequence Profile

Resource Coordination Record

Environment Coordination Record

Decision Coordination Register

Authority Boundary Reference

Coordination Prerequisite Register

Coordination Checkpoint Record

Conflict Register

Conflict Resolution Record

Coordination Evidence Register

Planned–Actual Alignment Record

Coordination Reassessment Record

Residual Coordination Obligation Register

Coordination Closure Record

Minimum Implementation Framework

A minimum CHCM implementation must establish eight governance layers. Layer 1 — Coordination Identity

Identify the coordination case, participating changes, responsible

functions, and coordination purpose. Layer 2 — Relationship Mapping

Identify shared software objects, dependencies, interfaces, configuration

relationships, and affected environments. Layer 3 — Compatibility and Sequence

Evaluate change compatibility, prerequisite relationships, execution order,

and permitted parallel activities. Layer 4 — Actors and Authority

Identify participating actors, responsibilities, decision references,

implementation permissions, and escalation boundaries. Layer 5 — Resources and Preconditions

Establish required resources, environment readiness, verification

conditions, coordination checkpoints, and stop conditions. Layer 6 — Conflict Governance

Identify conflicts, evaluate resolution options, preserve evidence, and

establish escalation where necessary. Layer 7 — Execution Alignment and Continuity

Compare planned coordination with actual conditions, monitor material

dependencies, and reassess changed circumstances. Layer 8 — Governed Output and Handoff

Establish the Software Change Coordination Profile, preserve unresolved

obligations, and provide structured references to LRECM and CCLM.

The implementation may be manual, software-supported, automated, or agent-assisted.

The governance requirements remain applicable regardless of the

coordination technology used.

Cross-Module Boundary

CRMM → CHCM

CRMM establishes change request identities and initial relationships.

CHCM governs the coordination of materially related changes and their

participating actors.

CIAM → CHCM

CIAM identifies impact, dependencies, interactions, and material

consequences.

CHCM translates these findings into coordination conditions, sequencing,

conflict management, and execution alignment requirements.

CHCM → LRECM

CHCM establishes coordination dependencies and conditions.

LRECM determines how affected changes must enter or re-enter applicable software lifecycle governance mechanisms.

CHCM → CCLM

CHCM preserves coordination outcomes, unresolved conflicts, and residual

obligations.

CCLM evaluates their relevance to final change disposition and closure.

Coordination Established ≠ Lifecycle Requirements Completed.

Cross-Architecture Boundary

CHCM coordinates change relationships across SLGA™ without replacing

specialized governance.

SRS — Software Requirements Standard

Requirements consistency and change-related requirements coordination.

SCS — Software Creation Standard

Development activities, software artifact creation, and implementation

dependencies.

SDS — Software Dependency Standard

Dependency compatibility and change relationships.

SCFS — Software Configuration Standard

Configuration baselines, builds, artifact identities, and configuration

conflicts.

SQS — Software Quality Standard

Testing coordination, integration verification, and quality evidence.

SSS — Software Security Standard

Security conditions, remediation dependencies, and access boundaries.

SRLS — Software Release Standard

Release candidate composition, readiness, and authorization dependencies.

SDPS — Software Deployment Standard

Deployment sequencing, production version transitions, operational

monitoring, rollback, and recovery.

SRTS — Software Retirement Standard

Migration sequencing and retirement-related change coordination.

SLCS — Software Lifecycle Control Standard

Lifecycle transitions, exceptions, and continuity.

Applicable OOF® authority, accountability, autonomous systems, and

operational reality governance references may also be required when coordination affects those domains.

Practical Applications

Enterprise Software Development

Coordinating modifications across multiple teams working on shared software

products and services.

Continuous Integration and Delivery

Preventing independently executing pipelines from introducing incompatible

artifacts, configurations, or deployment states.

Cloud-Native Systems

Coordinating changes across distributed services, shared dependencies,

databases, and infrastructure.

AI-Assisted Software Engineering

Managing multiple AI-generated modifications affecting shared repositories

and implementation assumptions.

Autonomous Software Agents

Establishing coordination boundaries when multiple agents perform related actions using shared tools and resources.

Critical Infrastructure

Preserving implementation sequencing, authority, operational continuity,

and verification across consequential software changes.

Robotics and Embedded Systems

Coordinating firmware, sensor, control logic, and operational configuration

changes across connected physical systems.

Use Case 1 — Two Independently Approved Changes Break a Shared Interface

Two software teams prepare changes to connected services.

Team A modifies an API response format.

Team B updates a dependent application using the previous response

structure.

Both teams complete their individual development and testing activities.

The changes are scheduled for deployment during the same maintenance window.

CHCM evaluates:

Change A → Change B → Shared Interface → Compatibility Conditions →

Implementation Sequence → Integration Verification

The assessment identifies:

• Both changes are individually identifiable and supported by relevant

• The changes share an interface dependency

• The new API response is incompatible with the dependent application's

• Independent testing does not establish combined compatibility

• Coordinated implementation and integration verification are required

evidence. current expectations.

The appropriate classification is:

Coordination Conflict Identified — Compatibility Resolution Required.

Individually Approved Changes ≠ Combined Operational Compatibility.

Use Case 2 — Two Autonomous Agents Modify the Same Production

Configuration

An organization operates two autonomous software maintenance agents.

Agent A is assigned to improve application performance.

Agent B is assigned to address a security configuration issue.

Both agents identify the same production configuration file as relevant to

their objectives.

Agent A modifies a runtime parameter to increase throughput.

Agent B modifies related access restrictions.

Each agent possesses permission to propose configuration changes, but

neither has unrestricted authority to apply conflicting production modifications.

CHCM evaluates:

Agent A Objective → Agent B Objective → Shared Configuration → Proposed

Actions → Authority Boundaries → Compatibility → Coordination Conditions

The assessment identifies:

• Both objectives are legitimate within their assigned scope

• The agents operate on a shared configuration object

• Their proposed modifications may interact

• Independent task authority does not establish authority for

• Coordination, compatibility assessment, and applicable authorization

unrestricted concurrent execution. are required.

The appropriate classification is:

Coordination Required — Concurrent Execution Not Established.

Individually Legitimate Agent Objectives ≠ Legitimate Uncoordinated

Execution.

Critical Distinctions

Individual Change Readiness ≠ Combined Change Readiness.

Communication Completed ≠ Coordination Established.

Planned Sequence ≠ Verified Execution Sequence.

Related Changes ≠ Compatible Changes.

Shared Schedule ≠ Established Execution Coordination.

Dependency Identified ≠ Dependency Satisfied.

Scheduled ≠ Ready.

Reported Complete ≠ Verified Prerequisite.

Unknown ≠ Satisfied.

Parallel Execution Possible ≠ Parallel Execution Acceptable.

Independent Test Success ≠ Combined Integration Verification.

Coordination Participation ≠ Implementation Authorization.

Coordination Responsibility ≠ Decision Authority.

Task Authority ≠ Constituent Action Authority.

Conflict Escalated ≠ Conflict Resolved.

Coordination Established ≠ Lifecycle Requirements Completed.

Coordination Closure ≠ Final Change Closure.

Canonical Boundary

CHCM governs the identification, coordination, compatibility, sequencing,

authority boundaries, conflict management, execution alignment, and continuity of relationships among software changes and their participating actors.

It establishes the evidence-supported conditions under which related

software modifications may proceed without unmanaged conflicts, unsupported compatibility assumptions, or ungoverned shared-resource interactions.

CHCM does not independently authorize implementation, replace impact assessment, execute software changes, or establish final software change closure.

Its principal governance output is the:

Software Change Coordination Profile

This output preserves the coordination relationships, prerequisites,

decisions, evidence, conflicts, and unresolved obligations necessary for controlled software lifecycle processing.

Canonical Closing Statement

Software changes do not become compatible merely because each change is individually justified, approved, or technically correct.

Two development teams may produce valid modifications that cannot operate together.

Two deployment pipelines may execute successfully while creating an inconsistent production state.

A dependency update may invalidate another team's implementation assumptions.

A database migration may require a sequence that independent delivery schedules fail to preserve.

An autonomous agent may perform its assigned task correctly while interfering with another agent's legitimate objective.

And a coordination plan may appear complete even though required prerequisites, authority boundaries, or compatibility conditions remain unverified.

CHCM establishes the governance structure required to distinguish these

realities.

It requires related changes to remain connected through identifiable dependencies, shared software objects, participating actors, execution sequences, and operational conditions.

It preserves the distinction between communication and coordination,

between scheduling and readiness, and between individual authority and authority over shared actions.

It ensures that material conflicts, unresolved prerequisites, resource constraints, and execution deviations remain visible and subject to appropriate resolution or escalation.

It supports automation and autonomous software engineering without allowing

parallel technical capability to substitute for coordinated governance.

And it maintains continuity as changes evolve, participants change,

dependencies shift, and implementation conditions become more complex.

The purpose of software change coordination is not merely to ensure that different actors know what others are doing. It is to establish whether their actions can legitimately and compatibly occur within the same software and operational reality.

Change Coordination Control Module (CHCM) transforms software change

coordination from administrative communication into an evidence-supported governance mechanism for compatibility, sequencing, authority alignment, and controlled lifecycle execution.

Modul4

Lifecycle Re-entry Control Module (LRECM)

Architecture

SLGA™ — Software Lifecycle Governance Architecture

Parent Standard

Software Change Management Standard (SCMS)

Category

Governance & Enforcement

Subcategory

Software Change Management Governance

Type

Core Module

Governed Space

Software Change Lifecycle Re-entry Governance

Module ID

SLGA-SCMS-LRECM-04

Origin ID

OOF-SLGA-SCMS-LRECM

Version

1.0

Status

Canonical Open

Effective Date

10.2026

Compatibility

Methodology OS — Modular Compatibility

GIE-Compatible

Yes

AI-Readable

Yes

Authority

OOF® — OriginOpen® Foundation

Protection

Protected Methodological Work

Canonical Language

English

Canonical Definition

Lifecycle Re-entry Control Module (LRECM) is the Core Module within Software Change Management Standard (SCMS) that governs how proposed, approved, or materially evolving software changes are directed into the applicable software lifecycle governance mechanisms according to their assessed consequences, affected software objects, existing evidence, operational conditions, and required verification obligations.

LRECM establishes lifecycle re-entry as a controlled, evidence-supported

relationship between an identifiable software change and the specific lifecycle governance activities that must be performed, repeated, extended, or verified because of that change.

The module distinguishes the need to revisit a lifecycle governance

mechanism from the need to repeat every activity previously performed within it, the existence of historical verification evidence from its continued applicability, and the authorization of a change from the satisfaction of its lifecycle obligations.

It requires each material lifecycle re-entry decision to preserve its

basis, affected governance mechanisms, entry conditions, required activities, evidence reuse decisions, completion criteria, applicable authority, and unresolved obligations.

LRECM recognizes that software changes may originate at any lifecycle stage and that their consequences may require movement backward, forward, or across the established software lifecycle governance structure.

It applies to human-directed modifications, automated development

processes, dependency changes, security remediation, production corrections, AI-generated software changes, autonomous agents, and software changes affecting embedded, industrial, or robotic systems.

Its principal governance output is the Software Lifecycle Re-entry Profile .

Operational Role

LRECM establishes the governance mechanism through which software changes are connected to the lifecycle activities affected by their consequences.

It governs:

• Lifecycle re-entry identification

• Change-to-lifecycle mapping

• Re-entry scope

• Affected Parent Standards

• Applicable Core Modules

• Re-entry triggers

• Lifecycle entry conditions

• Required reassessment

• Required verification

• Historical evidence applicability

• Evidence reuse

• Reverification requirements

• Lifecycle activity sequencing

• Cross-standard dependencies

• Applicable authority references

• Re-entry exceptions

• Emergency lifecycle pathways

• Automated lifecycle re-entry

• Autonomous agent boundaries

• Re-entry monitoring

• Re-entry deviations

• Completion evidence

• Residual lifecycle obligations

• Lifecycle re-entry continuity

• Handoff to change closure

LRECM does not replace the specialized governance requirements of the lifecycle standards into which a change is directed.

Operational Question

How can a software change be directed back into precisely those lifecycle governance mechanisms materially affected by its consequences, while preserving applicable evidence, avoiding unjustified repetition, preventing necessary verification from being bypassed, and ensuring that every required lifecycle obligation remains identifiable and verifiably satisfied?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

SCMS Core Module Flow

Change Request Management Module (CRMM) → Change Impact Assessment Module

(CIAM) → Change Coordination Control Module (CHCM) → Lifecycle Re-entry Control Module (LRECM) → Change Closure Management Module (CCLM)

LRECM is the fourth Core Module within SCMS.

CRMM establishes the change request.

CIAM identifies the consequences and affected governance mechanisms.

CHCM establishes material coordination relationships, prerequisites, and

sequencing conditions.

LRECM determines the required lifecycle governance pathway and controls the

traceability of the change through the applicable lifecycle mechanisms.

CCLM subsequently evaluates whether the resulting change and its

obligations have been appropriately resolved.

Lifecycle re-entry may be reassessed repeatedly when new evidence, changed scope, or operational deviations alter the applicable governance requirements.

• Governed Space

Software Change Lifecycle Re-entry Governance

This governed space concerns the controlled return, continuation, or

transfer of software changes into lifecycle governance mechanisms affected by the proposed or implemented modification.

It begins when an identifiable change requires determination of the lifecycle activities necessary to establish a legitimate resulting software state.

It includes:

Lifecycle Re-entry Determination

Affected Standard Identification

Required Activity Selection

Evidence Applicability Assessment

Lifecycle Sequencing

Entry Condition Verification

Re-entry Execution Traceability

Completion Verification

Residual Obligation Preservation

It continues until the applicable lifecycle obligations have been evidenced

as satisfied, legitimately deferred or excepted under competent governance, or transferred as unresolved conditions.

Governed Object

The primary Governed Object is:

A Software Lifecycle Re-entry Case representing the evidence-supported

relationship between an identifiable software change, its assessed consequences, the affected software lifecycle governance mechanisms, applicable historical evidence, required activities, entry conditions, completion requirements, and resulting lifecycle governance status.

The governed object must preserve, where applicable:

Lifecycle Re-entry Case ID

Change Request ID

Change Impact Assessment Reference

Change Coordination Reference

Affected Software Objects

Current Lifecycle State

Required Lifecycle Mechanisms

Re-entry Scope

Entry Conditions

Required Activities

Evidence Reuse Decisions

Required Reverification

Authority References

Completion Criteria

Execution Evidence

Residual Obligations

Re-entry Status

Re-entry Version

Scope

LRECM governs:

Lifecycle Re-entry Recognition

Re-entry Case Identification

Lifecycle Impact Mapping

Affected Parent Standard Identification

Affected Core Module Identification

Re-entry Necessity Assessment

Re-entry Scope Definition

Historical Evidence Review

Evidence Reuse Determination

Reverification Requirements

Lifecycle Entry Conditions

Lifecycle Sequencing

Cross-Standard Dependencies

Re-entry Authority References

Re-entry Exceptions

Emergency Re-entry

Automated Re-entry

Agent-Initiated Re-entry

Lifecycle Completion Evidence

Re-entry Reassessment

Residual Obligation Control

Closure Handoff

LRECM does not prescribe a specific development process, workflow automation platform, release pipeline, or organizational approval structure.

Foundational Principle

A software change must re-enter every lifecycle governance mechanism materially affected by the change, but it must not be forced through unrelated mechanisms without a justified governance need.

Lifecycle governance must be proportional to the actual change consequences.

A documentation correction may require limited governance activity.

A security-sensitive dependency replacement may require dependency, configuration, quality, security, release, and deployment reassessment.

A change to robotic control logic may require renewed functional, integration, operational, and physical-system verification.

The appropriate pathway depends on the governed reality rather than the

apparent size of the modification.

Lifecycle Re-entry ≠ Complete Lifecycle Repetition.

Historical Evidence Available ≠ Historical Evidence Still Applicable.

Change Authorized ≠ Lifecycle Obligations Satisfied.

Lifecycle Re-entry Identity

Every governed re-entry case must have a stable identity.

Relevant elements include:

Re-entry Case ID

Change Request ID

Software Product ID

Impact Assessment Reference

Coordination Reference

Creation Time

Responsible Governance Function

Current Re-entry Status

Evidence Reference

Re-entry Version

Material revisions must preserve their relationship to earlier re-entry decisions.

Lifecycle Re-entry Trigger

Lifecycle re-entry may be triggered by:

New Functional Requirement

Software Defect Correction

Security Vulnerability

Dependency Replacement

Configuration Modification

Build or Artifact Change

Quality Failure

Release Restriction

Deployment Failure

Operational Incident

Data Migration

Infrastructure Transition

Autonomous Agent Modification

Retirement-Related Change

The trigger must be associated with an identifiable software change and its assessed consequences.

Re-entry Necessity

LRECM must determine whether a proposed change materially affects an existing lifecycle governance conclusion.

Relevant questions include:

Does the change alter a governed software object?

Does it invalidate an existing requirement?

Does it introduce or modify a dependency?

Does it affect configuration or artifact identity?

Does it alter quality or security conditions?

Does it affect release or deployment readiness?

Does it change operational assumptions?

Does it introduce new verification obligations?

A negative re-entry determination must be supported by a documented assessment basis.

Lifecycle State Reference

LRECM must identify the relevant lifecycle condition of the affected software.

Potential states include:

Requirements Definition

Software Creation

Dependency Assessment

Configuration Establishment

Quality Verification

Security Verification

Release Preparation

Release Authorization

Deployment Preparation

Production Operation

Change Processing

Retirement Preparation

Retired

A software product may simultaneously have different components or versions occupying different lifecycle states.

Re-entry Scope

The re-entry scope identifies the lifecycle governance mechanisms requiring

renewed or additional activity.

Relevant elements include:

Affected Software Objects

Affected Parent Standards

Affected Core Modules

Required Assessments

Required Implementation Activities

Required Verification

Required Decisions

Applicable Environments

Required Evidence

Known Exclusions

The scope must remain traceable to the change impact assessment.

Re-entry Boundary

The re-entry boundary distinguishes required lifecycle activities from

unaffected governance mechanisms.

Relevant boundaries include:

Functional Boundary

Artifact Boundary

Dependency Boundary

Configuration Boundary

Security Boundary

Quality Boundary

Environment Boundary

Operational Boundary

Authority Boundary

Evidence Applicability Boundary

Exclusion of a lifecycle activity must not conceal a material unresolved consequence.

Lifecycle Impact Mapping

LRECM must map assessed change consequences to the relevant lifecycle governance mechanisms.

The mapping must preserve:

Change Impact Reference

Affected Software Object

Affected Governance Requirement

Relevant Parent Standard

Relevant Core Module

Required Activity

Evidence Requirement

Completion Condition

Dependency Reference

This mapping forms the basis of the lifecycle re-entry pathway.

Requirements Re-entry

Re-entry into SRS may be required when a change affects:

Software Purpose

Functional Requirements

Nonfunctional Requirements

Requirements Scope

Requirements Consistency

Requirements Traceability

Requirements Change Control

LRECM identifies the required requirements governance activities.

SRS governs their execution and acceptance.

Software Creation Re-entry

Re-entry into SCS may be required when a change affects:

Source Code

Software Artifacts

Development Method

Creator Attribution

Automated Software Creation

Implementation Traceability

Development Evidence

A software change may require new creation activities without requiring every original creation decision to be repeated.

Dependency Governance Re-entry

Re-entry into SDS may be required when a change affects:

Dependency Identity

Dependency Origin

Dependency Version

Dependency Suitability

Transitive Dependencies

Component Introduction

Dependency Compatibility

Dependency Continuity

A dependency update must not bypass suitability or compatibility governance merely because an automated tool recommends it.

Configuration Governance Re-entry

Re-entry into SCFS may be required when a change affects:

Configuration Baseline

Build Definition

Build Inputs

Artifact Identity

Software Version

Configuration Change Control

Build Reproducibility

Artifact Equivalence

A configuration change may require renewed verification even when source code remains unchanged.

Quality Governance Re-entry

Re-entry into SQS may be required when a change affects:

Quality Requirements

Quality Criteria

Functional Correctness

Regression Exposure

Test Coverage

Defect Status

Performance

Reliability

Compatibility

The required verification scope must reflect the actual consequences of the modification.

Security Governance Re-entry

Re-entry into SSS may be required when a change affects:

Security Requirements

Attack Surface

Authentication

Authorization

Software Exposure

Security Testing

Vulnerability Conditions

Security Remediation

Security Configuration

A security-motivated change may require verification of both the intended remediation and any new security consequences.

Release Governance Re-entry

Re-entry into SRLS may be required when a change affects:

Release Candidate Composition

Release Readiness

Release Verification Evidence

Release Authorization Conditions

Release Records

Release Compatibility

Release Withdrawal Conditions

An existing release authorization must not automatically be treated as applicable to a materially modified release candidate.

Deployment Governance Re-entry

Re-entry into SDPS may be required when a change affects:

Deployment Planning

Deployment Authorization

Production Version

Deployment State

Operational Monitoring

Rollback Feasibility

Recovery Requirements

Production Compatibility

A change may require deployment governance even when its implementation has already passed development and quality verification.

Retirement Governance Re-entry

Re-entry into SRTS may be required when a change affects:

Retirement Planning

End-of-Support Conditions

Migration Requirements

Residual Dependencies

Data Continuity

Retirement Closure

Replacement Compatibility

A retirement-related modification may require renewed lifecycle activity before the affected software can be legitimately withdrawn.

Lifecycle Control Interaction

SLCS provides cross-cutting governance for lifecycle state consistency,

transitions, exceptions, control transfer, and continuity.

LRECM must identify where a change affects:

Lifecycle State Consistency

Lifecycle Transition Conditions

Lifecycle Exceptions

Lifecycle Control Ownership

Lifecycle Continuity

Lifecycle Control Assessment

LRECM governs change-driven re-entry.

SLCS governs the integrity of the broader lifecycle control mechanisms.

Re-entry Granularity

Lifecycle re-entry may be required at different levels.

Potential levels include:

Individual Governance Requirement

Specific Core Module Activity

Core Module Reassessment

Parent Standard Reassessment

Multiple Parent Standards

Cross-Architecture Governance

The selected level must be justified by the assessed change consequences.

A change affecting one requirement does not automatically require repeating an entire Parent Standard.

Proportional Re-entry

LRECM must establish a proportional relationship between change impact and required lifecycle activity.

Relevant considerations include:

Change Significance

Affected Objects

Impact Propagation

Existing Evidence

Verification Coverage

Operational Criticality

Security Exposure

Uncertainty

Reversibility

Applicable Mandatory Requirements

Proportionality must not be used to bypass mandatory verification.

Historical Evidence Identification

LRECM must identify existing evidence potentially relevant to the modified software state.

Evidence may include:

Requirements Records

Development Records

Dependency Assessments

Configuration Baselines

Build Evidence

Quality Test Results

Security Assessments

Release Records

Deployment Records

Operational Observations

Historical evidence must remain associated with its original scope and conditions.

Evidence Applicability Assessment

Before historical evidence is reused, LRECM must evaluate whether its assumptions remain valid.

Relevant elements include:

Software Version

Artifact Identity

Dependency State

Configuration

Test Environment

Operational Conditions

Verification Scope

Evidence Age

Changed Requirements

Known Limitations

Evidence reuse must be supported by an identifiable applicability basis.

Evidence Reuse

Historical evidence may be reused when it remains materially applicable to the changed software state.

Potential classifications include:

Reusable Without Material Change

Reusable With Additional Verification

Partially Reusable

Not Reusable

Applicability Uncertain

The decision must preserve the supporting rationale and relevant evidence references.

Evidence Exists ≠ Evidence Remains Valid.

Reverification Requirements

Reverification is required where material changes invalidate or limit earlier verification conclusions.

Relevant elements include:

Affected Requirement

Changed Software Object

Previous Verification

Invalidated Assumption

Required New Verification

Verification Environment

Acceptance Condition

Evidence Reference

Reverification must target the affected governance conditions without unjustified duplication.

New Governance Obligations

A change may introduce obligations that did not exist in the previous software state.

Examples include:

New Security Requirement

New Dependency Assessment

New Compatibility Test

New Data Migration Verification

New Operational Monitoring Requirement

New Recovery Requirement

New Interface Validation

New Human Oversight Condition

LRECM must distinguish new obligations from repeated historical activities.

Re-entry Preconditions

Each required lifecycle activity may have entry conditions.

Potential conditions include:

Identifiable Change Request

Sufficient Impact Assessment

Required Coordination Established

Applicable Authority

Required Software Artifacts Available

Relevant Baseline Established

Necessary Resources Available

Prerequisite Governance Activity Completed

Material Uncertainty Addressed

An activity must not be classified as legitimately entered when mandatory prerequisites remain unsatisfied.

Re-entry Condition States

LRECM recognizes the following condition states:

Satisfied

Not Satisfied

Partially Satisfied

Unknown

Not Applicable

Exception

Each condition must preserve its evidence or applicable exception reference.

Unknown ≠ Satisfied.

Activity Scheduled ≠ Entry Conditions Established.

Re-entry Sequencing

Lifecycle activities may require a particular sequence.

Examples include:

Requirements Revision → Software Creation

Dependency Assessment → Configuration Revision

Configuration Revision → Quality Verification

Security Remediation → Security Reverification

Quality and Security Readiness → Release Reassessment

Release Authorization → Deployment Authorization

The sequence must reflect applicable governance dependencies rather than an assumption that every change must repeat the complete original lifecycle order.

Parallel Lifecycle Activities

Some lifecycle activities may proceed in parallel where their prerequisites and dependencies permit.

Relevant considerations include:

Independent Governance Scope

Shared Software Objects

Evidence Dependencies

Authority Conditions

Resource Constraints

Compatibility Requirements

Verification Interactions

Parallel processing must not compromise required sequencing or evidence validity.

Cross-Standard Dependency

A lifecycle re-entry pathway may contain dependencies between standards.

Relevant elements include:

Source Governance Activity

Dependent Governance Activity

Required Evidence

Prerequisite Condition

Completion Status

Failure Response

Reassessment Requirement

LRECM must preserve these dependencies within the Software Lifecycle Re-entry Profile.

Re-entry Decision

Every material re-entry determination must preserve:

Decision Reference

Change Request

Impact Assessment

Affected Governance Mechanisms

Required Activities

Excluded Activities

Evidence Reuse Decisions

Applicable Authority

Decision Conditions

Decision Time

A re-entry decision establishes the required governance pathway.

It does not independently authorize every technical action within that pathway.

Authority and Permission Boundary

LRECM must preserve the distinction between authority to direct a change into lifecycle governance and authority to execute specific activities.

Relevant elements include:

Decision Authority

Implementation Authority

Verification Responsibility

Release Authority

Deployment Authority

Agent Permissions

Delegation Conditions

Escalation Requirements

Lifecycle Re-entry Decision ≠ Universal Execution Permission.

Change Authorization Interaction

A software change may have an applicable change-level decision while still requiring specialized lifecycle decisions.

Examples include:

Change Accepted

Development Activity Permitted

Security Verification Required

Release Authorization Pending

Deployment Authorization Pending

These conditions must remain distinct.

An approved change must not bypass required release or deployment authorization.

Automated Lifecycle Re-entry

Automated systems may support lifecycle re-entry determination and execution.

Examples include:

Automated Test Selection

Dependency Assessment Triggers

Configuration Validation

Security Scanning

Build Reproduction

Release Readiness Checks

Deployment Preconditions

Automation must preserve:

Trigger Identity

Applicable Change

Selected Governance Activity

Selection Basis

Execution Authority

Evidence

Failure Conditions

Automated selection must remain subject to the actual governance requirements of the affected lifecycle mechanism.

AI-Assisted Re-entry Determination

AI systems may recommend which lifecycle activities should be revisited.

LRECM must preserve:

AI System Identity

Input Change Context

Impact Assessment Reference

Recommended Lifecycle Activities

Recommendation Basis

Known Limitations

Required Review

Applicable Authority

An AI recommendation must not be treated as an independently established governance decision without an appropriate validation and authority basis.

Autonomous Agent Re-entry Boundary

Autonomous agents may participate in lifecycle activities where legitimately authorized.

Relevant elements include:

Agent Identity

Assigned Objective

Permitted Lifecycle Activities

Tool Permissions

Affected Software Objects

Environment Restrictions

Decision Boundaries

Human Override

Escalation Conditions

Execution Evidence

Authority to perform one lifecycle activity does not automatically establish authority for subsequent release or deployment actions.

Emergency Lifecycle Re-entry

Urgent changes may require accelerated lifecycle processing.

Potential triggers include:

Critical Security Vulnerability

Production Service Failure

Severe Operational Incident

Critical Dependency Failure

Safety-Relevant Software Condition

Infrastructure Continuity Threat

Emergency processing may alter the sequence or timing of applicable activities where legitimately permitted.

It must not erase the underlying governance obligations.

Emergency Re-entry Conditions

Emergency re-entry must preserve:

Emergency Origin

Affected Software

Urgency Evidence

Required Activities

Applicable Authority

Permitted Accelerations

Deferred Verification Obligations

Operational Restrictions

Post-Implementation Verification

Closure Conditions

Deferred obligations must remain explicitly governed until resolved.

Emergency Acceleration ≠ Governance Exemption.

Lifecycle Re-entry Exception

An exception may be required when a normal lifecycle activity cannot be performed under the applicable conditions.

Relevant elements include:

Exception ID

Affected Governance Requirement

Reason

Applicable Authority

Alternative Control

Residual Risk

Duration or Validity Condition

Required Follow-up

Evidence

Exceptions must not be inferred from missing records or incomplete implementation.

Re-entry Exception Boundary

A lifecycle exception must remain limited to its legitimate scope.

It must not automatically authorize:

Unrelated Software Modifications

Unverified Dependency Changes

Unrestricted Production Access

Bypassing Unaffected Mandatory Requirements

Uncontrolled Agent Actions

Permanent Removal of Deferred Obligations

The exception must remain traceable to the relevant governance requirement and competent decision.

Re-entry Execution Traceability

LRECM must preserve the relationship between required lifecycle activities and their actual processing.

Relevant elements include:

Required Activity

Responsible Function

Applicable Standard

Applicable Core Module

Entry Condition

Execution Reference

Evidence

Completion Status

Outstanding Obligation

Technical execution records may be referenced without replacing the specialized governance records.

Re-entry Activity Status

Each required activity must have an identifiable status.

Potential states include:

Not Started

Entry Conditions Pending

Ready

In Progress

Verification Pending

Completed With Evidence

Failed

Blocked

Deferred Under Exception

Reassessment Required

Unknown

An activity marked complete without sufficient evidence must not automatically be treated as a satisfied governance obligation.

Re-entry Verification

LRECM must verify whether required lifecycle activities have produced the applicable evidence and conclusions.

Relevant elements include:

Required Activity

Completion Evidence

Applicable Acceptance Criteria

Responsible Governance Function

Evidence Validity

Unresolved Findings

Completion Decision

Remaining Dependencies

LRECM verifies the fulfillment of the re-entry pathway.

It does not replace the substantive verification performed under specialized standards.

Re-entry Completion Conditions

A lifecycle re-entry case may be considered complete only when its applicable completion conditions are established.

Relevant conditions include:

Required Activities Identified

Mandatory Preconditions Satisfied

Applicable Decisions Recorded

Required Evidence Available

Required Verification Completed

Material Conflicts Resolved

Outstanding Exceptions Identified

Residual Obligations Preserved

Handoff Conditions Established

Completion must be supported by evidence appropriate to the governed change.

Partial Re-entry Completion

Some required lifecycle activities may be completed while others remain unresolved.

LRECM must preserve:

Completed Activities

Pending Activities

Failed Activities

Deferred Activities

Outstanding Verification

Unresolved Dependencies

Applicable Restrictions

Required Follow-up

Partial completion must not be presented as full lifecycle compliance.

Re-entry Failure

Re-entry failure may occur when:

Required Activity Cannot Be Performed

Mandatory Preconditions Fail

Verification Fails

Required Authority Is Missing

Evidence Is Insufficient

Dependency Conditions Become Invalid

Operational Conditions Become Unacceptable

Required Governance Activity Is Bypassed

Failure must trigger appropriate reassessment, restriction, escalation, or transfer to another governed pathway.

Re-entry Deviation

A deviation occurs when actual lifecycle processing differs materially from the established re-entry pathway.

Examples include:

Unplanned Activity Omission

Unapproved Sequence Change

Different Software Artifact

Changed Dependency

Unexpected Environment

Unverified Evidence Reuse

Unauthorized Automated Execution

Incomplete Verification

Material deviations must be assessed before the pathway can be treated as completed.

Re-entry Reassessment

Reassessment may be required when:

Change Scope Expands

Impact Assessment Changes

Coordination Conditions Change

New Dependencies Are Identified

Historical Evidence Becomes Inapplicable

Verification Fails

Implementation Deviates

Operational Conditions Change

New Security Findings Emerge

An Agent Performs an Unplanned Action

Reassessment must preserve the earlier pathway and identify which conclusions remain valid.

Lifecycle Re-entry Continuity

LRECM must preserve continuity across:

Change Revisions

Lifecycle Reassessment

Actor Changes

Responsibility Transfers

Interrupted Implementation

Deferred Activities

Emergency Processing

Automated Execution

Repeated Verification

Long-Term Software Maintenance

The history of required activities, decisions, evidence, and unresolved

obligations must remain reconstructable.

Residual Lifecycle Obligations

A lifecycle re-entry case may retain obligations after particular activities have been completed.

Examples include:

Deferred Security Verification

Outstanding Compatibility Test

Post-Deployment Monitoring

Data Migration Verification

Recovery Validation

External Integration Confirmation

Physical-System Validation

Exception Expiration Review

Residual obligations must remain associated with responsible functions and applicable resolution conditions.

Re-entry Governance Classification

LRECM recognizes the following governance classifications:

Re-entry Required

Re-entry Not Required With Justification

Re-entry Scope Under Assessment

Re-entry Pathway Established

Entry Conditions Pending

Ready for Lifecycle Processing

Lifecycle Processing Active

Evidence Reuse Approved

Reverification Required

Partially Completed

Completed With Evidence

Blocked

Failed

Exception Active

Residual Obligations Outstanding

Reassessment Required

Transferred to Change Closure

Unknown

Conflicted

Classification must remain associated with the relevant re-entry case, scope, version, and evidence.

Governance Requirements

A conforming LRECM implementation must satisfy the following requirements. Re-entry Identity and Determination

• Establish a stable lifecycle re-entry case identity

• Associate the case with an identifiable software change

• Preserve relevant impact assessment references

• Preserve relevant coordination references

• Identify the affected software objects

• Establish the current lifecycle state

• Determine whether lifecycle re-entry is required

• Preserve the basis for the determination

Scope and Governance Mapping

• Identify materially affected Parent Standards

• Identify applicable Core Modules

• Establish required lifecycle activities

• Define the re-entry scope

• Preserve justified exclusions

• Identify cross-standard dependencies

• Establish required activity sequencing

• Identify permitted parallel activities

Evidence and Verification

• Identify relevant historical evidence

• Evaluate historical evidence applicability

• Preserve evidence reuse decisions

• Identify required reverification

• Identify new governance obligations

• Establish required evidence for each activity

• Preserve specialized verification references

• Prevent unsupported evidence reuse

Authority and Conditions

• Identify applicable decision authority

• Preserve execution authority boundaries

• Establish entry conditions

• Identify prerequisite activities

• Preserve applicable restrictions

• Govern emergency re-entry conditions

• Preserve exception authority and scope

• Prevent re-entry decisions from substituting for specialized authorization.

Execution and Continuity

• Preserve lifecycle activity status

• Maintain execution traceability

• Identify material deviations

• Establish reassessment triggers

• Preserve material pathway revisions

• Verify required activity completion

• Identify partial completion

• Preserve unresolved obligations

• Maintain historical lifecycle continuity

• Establish evidence-supported re-entry classification

• Provide structured references to CCLM

• Prevent incomplete lifecycle processing from being presented as

fully satisfied.

Governance Logic

Software Change → Impact Assessment → Coordination References → Lifecycle

Re-entry Trigger → Current Lifecycle State → Affected Governance Mechanisms → Re-entry Necessity → Re-entry Scope → Historical Evidence → Evidence Applicability → Required Activities → Entry Conditions → Authority References → Lifecycle Sequence → Governed Processing → Verification Evidence → Completion Assessment → Residual Obligations → Software Lifecycle Re-entry Profile

Where historical evidence remains applicable:

Existing Evidence → Applicability Assessment → Reuse Decision → Preserved

Evidence Reference

Where evidence is invalidated:

Material Change → Invalidated Assumption → Reverification Requirement →

Specialized Governance Activity

Where multiple standards are affected:

Change Impact → Affected Standards → Dependency Mapping → Coordinated

Re-entry Pathway

Where emergency processing is required:

Emergency Condition → Applicable Authority → Accelerated Pathway →

Preserved Mandatory Obligations → Deferred Verification Tracking

Where an autonomous agent participates:

Agent Objective → Applicable Authority → Permitted Lifecycle Activity →

Tool-Path Boundary → Execution Evidence → Verification

Where implementation deviates:

Established Pathway → Material Deviation → Reassessment → Revised Pathway

/ Restriction / Escalation

LRECM must not equate a completed automation pipeline with satisfied lifecycle governance unless the required activities, evidence, and decisions have been established.

Governance Outputs

The principal output is:

Software Lifecycle Re-entry Profile

Supporting outputs may include:

Lifecycle Re-entry Case Identity

Change-to-Lifecycle Mapping

Affected Parent Standard Register

Affected Core Module Register

Re-entry Necessity Assessment

Re-entry Scope Record

Historical Evidence Register

Evidence Applicability Assessment

Evidence Reuse Decision Record

Reverification Requirement Register

Lifecycle Activity Register

Entry Condition Register

Cross-Standard Dependency Map

Lifecycle Sequence Profile

Authority Boundary Reference

Emergency Re-entry Record

Lifecycle Exception Register

Execution Traceability Record

Lifecycle Verification Record

Re-entry Reassessment Record

Residual Obligation Register

Re-entry Completion Assessment

Change Closure Handoff Record

Minimum Implementation Framework

A minimum LRECM implementation must establish eight governance layers. Layer 1 — Change and Lifecycle Identity

Identify the software change, current lifecycle state, relevant impact

assessment, and coordination references. Layer 2 — Governance Mapping

Determine which Parent Standards, Core Modules, and governance requirements

are materially affected. Layer 3 — Evidence Applicability

Identify historical evidence, assess its continued validity, and establish

reuse or reverification requirements. Layer 4 — Re-entry Pathway

Define required activities, applicable exclusions, sequencing,

dependencies, and completion conditions. Layer 5 — Authority and Preconditions

Identify responsible functions, decision boundaries, execution permissions,

mandatory prerequisites, and applicable exceptions. Layer 6 — Governed Processing

Preserve the status and evidence of required lifecycle activities without

replacing specialized governance mechanisms. Layer 7 — Verification and Continuity

Assess completion, identify deviations, preserve reassessment triggers, and

maintain unresolved obligations. Layer 8 — Governed Output and Handoff

Establish the Software Lifecycle Re-entry Profile and provide

evidence-supported references to CCLM.

The implementation may be manual, workflow-supported, automated, or AI-assisted.

The software used to implement LRECM must adapt to its governance requirements rather than redefine them.

Cross-Module Boundary

CRMM → LRECM

CRMM establishes the original change request identity, purpose, preliminary

scope, and evidence.

LRECM preserves these references while determining the applicable lifecycle

pathway.

CIAM → LRECM

CIAM identifies direct and indirect consequences, uncertainty, verification

implications, and affected lifecycle mechanisms.

LRECM determines which governance activities must be performed or revisited.

CHCM → LRECM

CHCM establishes coordination dependencies, actor relationships,

compatibility conditions, and sequencing constraints.

LRECM incorporates applicable coordination conditions into the re-entry

pathway.

LRECM → CCLM

LRECM establishes the required lifecycle obligations and their completion

status.

CCLM evaluates the resulting change outcome, outstanding obligations, and

legitimate closure disposition.

Lifecycle Processing Completed ≠ Change Outcome Verified.

Cross-Architecture Boundary

LRECM directs software changes into applicable SLGA™ governance mechanisms

without replacing their specialized requirements.

SRS — Software Requirements Standard

Requirements definition, scope, change control, and traceability.

SCS — Software Creation Standard

Software creation, artifact generation, and development traceability.

SDS — Software Dependency Standard

Dependency suitability, introduction, compatibility, and continuity.

SCFS — Software Configuration Standard

Configuration baselines, builds, artifact identification, and

reproducibility.

SQS — Software Quality Standard

Quality requirements, testing, defect governance, and quality condition

assessment.

SSS — Software Security Standard

Security requirements, exposure, testing, vulnerabilities, and remediation.

SRLS — Software Release Standard

Release candidates, readiness, authorization, records, and withdrawal.

SDPS — Software Deployment Standard

Deployment planning, authorization, production versions, monitoring,

rollback, and recovery.

SRTS — Software Retirement Standard

Retirement planning, end-of-support, migration, residual dependencies, and

closure.

SLCS — Software Lifecycle Control Standard

Lifecycle state consistency, transitions, exceptions, control transfer, and

continuity.

Applicable OOF® authority, accountability, autonomous systems, and

operational reality governance references remain independently applicable where the change affects those governed domains.

Practical Applications

Enterprise Software Maintenance

Directing changes into only those lifecycle governance activities

materially affected by the modification.

Continuous Integration and Delivery

Determining which verification and governance stages must be repeated after changes to software artifacts, dependencies, or configurations.

Security Remediation

Establishing appropriate dependency, configuration, quality, security,

release, and deployment re-entry after vulnerability corrections.

AI-Assisted Software Development

Preventing AI-generated patches from bypassing necessary lifecycle

governance while avoiding unnecessary repetition of unaffected activities.

Autonomous Software Operations

Constraining agents to legitimately assigned lifecycle activities and

preserving the distinction between automated execution and governance completion.

Critical Infrastructure

Establishing traceable lifecycle re-entry for changes affecting service

continuity, security, operational restrictions, or recovery capability.

Robotics and Embedded Systems

Determining when firmware or control logic modifications require renewed integration, operational, or physical-system verification.

Use Case 1 — A Security Dependency Update Requires Selective Lifecycle

Re-entry

A software team identifies a security vulnerability in a third-party

dependency.

A corrected dependency version is available.

The proposed update does not alter the application's stated functional requirements.

However, the new dependency version changes internal interfaces and

introduces different runtime behavior.

LRECM evaluates:

Change Request → Impact Assessment → Dependency Change → Configuration

Consequences → Quality Verification → Security Verification → Release and Deployment Requirements

The assessment establishes:

• Existing functional requirements remain applicable

• Dependency suitability must be reassessed

• Configuration and build compatibility must be verified

• Relevant regression testing is required

• Security remediation effectiveness must be verified

• Release and deployment governance must be revisited where the updated

• Unaffected requirements-definition activities do not need to be

artifact is introduced. repeated without justification.

The appropriate classification is:

Selective Lifecycle Re-entry Required — Multiple Affected Governance

Mechanisms.

A dependency update does not require repeating every lifecycle activity, but it must satisfy every lifecycle obligation materially affected by the update.

Use Case 2 — An AI-Generated Patch Passes Tests but Cannot Bypass

Deployment Governance

An autonomous software maintenance agent is authorized to propose and implement limited corrections within a development environment.

The agent identifies a defect, generates a patch, and executes the

permitted automated tests.

The tests pass.

The agent then attempts to promote the modified artifact directly into

production.

LRECM evaluates:

Agent-Generated Change → Creation Evidence → Configuration and Artifact

Identity → Quality Verification → Security Implications → Release Requirements → Deployment Authorization

The assessment establishes:

• The agent's development activities are within its assigned scope

• Relevant test evidence is available

• The modified artifact requires applicable configuration and release

• Production deployment requires a distinct authorization basis

• Passing development tests does not establish permission to execute

• The agent's tool permissions must remain restricted to its legitimate

governance. production deployment. authority.

The appropriate classification is:

Lifecycle Re-entry Partially Satisfied — Release and Deployment

Obligations Outstanding.

Automated Test Success ≠ Authorization to Enter Production.

Critical Distinctions

Lifecycle Re-entry ≠ Complete Lifecycle Repetition.

Change Authorized ≠ Lifecycle Obligations Satisfied.

Historical Evidence Available ≠ Historical Evidence Still Applicable.

Evidence Exists ≠ Evidence Remains Valid.

Evidence Reuse ≠ Verification Bypass.

Previously Completed Activity ≠ Currently Satisfied Requirement.

Limited Modification ≠ Limited Lifecycle Impact.

Affected Standard Identified ≠ Required Activity Completed.

Re-entry Decision ≠ Universal Execution Permission.

Activity Scheduled ≠ Entry Conditions Established.

Unknown ≠ Satisfied.

Emergency Acceleration ≠ Governance Exemption.

Automated Pipeline Completion ≠ Lifecycle Governance Completion.

Agent Task Authority ≠ Authority for Every Lifecycle Action.

Partial Re-entry Completion ≠ Full Lifecycle Compliance.

Lifecycle Processing Completed ≠ Change Outcome Verified.

Canonical Boundary

LRECM governs the determination, structuring, evidence applicability,

sequencing, execution traceability, and verification of software lifecycle governance activities required because of a proposed or implemented software change.

It establishes the relationship between assessed change consequences and

the specific lifecycle mechanisms that must be performed, repeated, extended, or verified.

It preserves justified evidence reuse, required reverification, authority

boundaries, exceptions, material deviations, and unresolved lifecycle obligations.

LRECM does not replace specialized lifecycle standards, independently authorize implementation, or establish final change closure.

Its principal governance output is the:

Software Lifecycle Re-entry Profile

This output provides the evidence-supported record of which lifecycle

governance mechanisms were required, why they were required, how their obligations were addressed, and which conditions remain unresolved.

Canonical Closing Statement

Software lifecycle governance loses its meaning when changes can invalidate earlier conclusions without triggering the activities necessary to establish their continued validity.

A security patch may require renewed dependency and quality assessment.

A configuration change may invalidate previously verified runtime conditions.

A new interface may require integration testing even when its individual components function correctly.

A deployment correction may require renewed release and production authorization.

An AI-generated modification may satisfy development tests while remaining outside its legitimate operational authority.

And a minor change may require extensive governance because of the consequences it introduces, while a larger change may leave some historical evidence fully applicable.

LRECM establishes the methodological mechanism required to govern these

differences.

It does not require every software change to restart the entire lifecycle.

It does not permit necessary lifecycle obligations to disappear merely because a modification is urgent, automated, or technically simple.

Instead, it determines which governance mechanisms are materially affected, which historical evidence remains applicable, which activities must be repeated or extended, and which conditions must be verified before the resulting software state can be considered adequately governed.

It preserves the distinction between a completed technical action and a

satisfied governance requirement.

It maintains continuity across lifecycle revisions, exceptions,

reassessments, and automated execution.

And it ensures that unresolved obligations remain visible until they are legitimately resolved or transferred.

The purpose of lifecycle re-entry is not to repeat the past. It is to determine which conclusions from the past remain valid after software reality has changed—and to re-establish those that no longer do.

Lifecycle Re-entry Control Module (LRECM) transforms software change

processing into a proportional, evidence-supported mechanism for preserving lifecycle governance integrity as software evolves.

Modul5

Change Closure Management Module (CCLM)

Architecture

SLGA™ — Software Lifecycle Governance Architecture

Parent Standard

Software Change Management Standard (SCMS)

Category

Governance & Enforcement

Subcategory

Software Change Management Governance

Type

Core Module

Governed Space

Software Change Closure Governance

Module ID

SLGA-SCMS-CCLM-05

Origin ID

OOF-SLGA-SCMS-CCLM

Version

1.0

Status

Canonical Open

Effective Date

10.2026

Compatibility

Methodology OS — Modular Compatibility

GIE-Compatible

Yes

AI-Readable

Yes

Authority

OOF® — OriginOpen® Foundation

Protection

Protected Methodological Work

Canonical Language

UCL — Universal Canonical Language (English)

Canonical Definition

Change Closure Management Module (CCLM) is the Core Module within Software Change Management Standard (SCMS) that governs how software change cases are evaluated, classified, resolved, and formally closed according to their intended outcomes, actual implementation results, verified software conditions, fulfilled lifecycle obligations, remaining uncertainties, and residual responsibilities.

CCLM establishes software change closure as an evidence-supported

governance determination concerning the disposition of an identifiable change case rather than a simple administrative status indicating that a modification has been completed.

The module distinguishes implementation completion from outcome

verification, successful change realization from unsuccessful or partial realization, administrative closure from verified resolution, and the closure of a change case from the elimination of all obligations associated with its consequences.

It requires the original change purpose, approved or otherwise legitimately

established scope, implemented modification, resulting software state, verification evidence, lifecycle completion records, deviations, unresolved conditions, and closure decision to remain connected.

CCLM recognizes that software changes may conclude through successful implementation, partial implementation, failed implementation, rejection, withdrawal, supersession, deferral, cancellation, or transfer into another governed change case.

It also recognizes that a change may produce its intended technical result while leaving security, compatibility, operational, recovery, or lifecycle obligations unresolved.

CCLM applies to human-directed changes, automated development and delivery

systems, AI-generated modifications, autonomous software agents, enterprise applications, cloud infrastructure, embedded software, and robotic systems.

Its principal governance output is the Software Change Closure Profile.

Operational Role

CCLM establishes the final governance disposition mechanism within SCMS.

It governs:

• Change closure identity

• Change case continuity

• Original purpose preservation

• Intended outcome identification

• Implemented modification reference

• Actual software state identification

• Intended–actual outcome comparison

• Implementation completion assessment

• Verification evidence assessment

• Lifecycle obligation reconciliation

• Coordination obligation reconciliation

• Change deviation assessment

• Unintended consequence recognition

• Partial implementation assessment

• Failed change disposition

• Rejected and withdrawn change disposition

• Supersession and transfer

• Residual obligation management

• Closure eligibility

• Closure decision

• Closure evidence

• Post-closure discovery

• Closure reopening

• Historical continuity

• Final change governance classification

CCLM does not replace technical verification, release authorization, deployment monitoring, incident resolution, or specialized lifecycle governance.

Operational Question

How can a software change be legitimately and verifiably closed when its intended modification, actual implementation, resulting operational state, lifecycle obligations, unresolved consequences, and responsible actors may differ, without confusing administrative completion with successful change resolution?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

SCMS Core Module Flow

Change Request Management Module (CRMM) → Change Impact Assessment Module

(CIAM) → Change Coordination Control Module (CHCM) → Lifecycle Re-entry Control Module (LRECM) → Change Closure Management Module (CCLM)

CCLM is the fifth Core Module within SCMS.

CRMM establishes the change request and its original purpose.

CIAM establishes the assessed consequences and verification implications.

CHCM establishes coordination conditions and related obligations.

LRECM identifies and tracks required lifecycle governance activities.

CCLM evaluates the resulting change case against these references and

establishes its legitimate disposition.

Closure may require additional assessment, verification, correction, or transfer when the evidence does not support the intended outcome.

• Governed Space

Software Change Closure Governance

This governed space concerns the final evaluation and disposition of

identifiable software change cases.

It begins when a change case requires a determination of whether its intended outcome has been achieved, its implementation has failed, its processing has been discontinued, or another legitimate disposition applies.

It includes:

Outcome Verification

Implementation Reconciliation

Lifecycle Obligation Reconciliation

Coordination Obligation Reconciliation

Deviation Assessment

Residual Obligation Identification

Closure Eligibility

Closure Decision

Historical Closure Continuity

Post-Closure Reassessment

The governed space continues where new evidence materially challenges an earlier closure conclusion.

Governed Object

The primary Governed Object is:

A Software Change Closure Case representing the evidence-supported

relationship between an identifiable software change, its original purpose, applicable decisions, implemented or unimplemented modification, actual software state, verification results, fulfilled and outstanding obligations, final disposition, and closure authority.

The governed object must preserve, where applicable:

Change Closure Case ID

Change Request ID

Change Impact Assessment Reference

Change Coordination Reference

Lifecycle Re-entry Reference

Original Change Purpose

Intended Outcome

Applicable Change Decision

Implementation Reference

Actual Software State

Verification Evidence

Deviation Record

Residual Obligations

Closure Classification

Closure Decision

Closure Authority

Closure Time

Closure Evidence

Closure Version

Scope

CCLM governs:

Closure Case Formation

Change Purpose Reconciliation

Intended Outcome Assessment

Implementation Status Assessment

Actual State Identification

Outcome Verification

Change Success Classification

Partial Outcome Assessment

Failed Change Disposition

Rejected Change Disposition

Withdrawn Change Disposition

Superseded Change Disposition

Deferred Change Disposition

Lifecycle Obligation Reconciliation

Coordination Obligation Reconciliation

Residual Obligation Management

Closure Eligibility

Closure Authority

Closure Evidence

Post-Closure Reassessment

Closure Reopening

Historical Continuity

CCLM does not prescribe a specific issue-tracking system, software delivery methodology, incident management platform, or organizational closure workflow.

Foundational Principle

A software change may be considered successfully resolved only when its intended outcome is supported by applicable evidence, its resulting software state is adequately established, and its material governance obligations have been satisfied or explicitly preserved under a legitimate disposition.

Technical completion is not sufficient evidence of successful change resolution.

A change may be implemented without producing the expected outcome.

A defect may be corrected while introducing another defect.

A security vulnerability may be remediated while leaving operational compatibility unresolved.

A deployment may succeed while required monitoring remains incomplete.

An autonomous agent may report task completion without establishing that its actions remained within legitimate authority.

Implementation Complete ≠ Intended Outcome Verified.

Administrative Closure ≠ Successful Change Resolution.

Closure Case Identity

Every governed closure case must have a stable identity.

Relevant elements include:

Closure Case ID

Change Request ID

Software Product ID

Closure Origin

Responsible Closure Function

Closure Assessment Time

Applicable Change Version

Closure Status

Evidence Reference

Closure Version

A closure case must preserve its relationship to the original software change throughout revisions and reassessment.

Closure Trigger

Closure assessment may be triggered by:

Implementation Reported Complete

Verification Completed

Change Objective Achieved

Implementation Failed

Change Rejected

Change Withdrawn

Change Superseded

Change Deferred

Implementation Cancelled

Alternative Resolution Established

Change Transferred

Operational Conditions Require Final Disposition

A closure trigger does not establish closure eligibility.

Closure Purpose

The closure purpose must identify what the closure assessment is intended to establish.

Potential purposes include:

Confirm Successful Change Resolution

Determine Partial Completion

Classify Failed Implementation

Record Legitimate Rejection

Record Withdrawal

Establish Supersession

Document Deferral

Transfer Remaining Obligations

Preserve Final Evidence

Establish Closure Authority and Decision

The purpose must remain distinct from the outcome ultimately established.

Original Change Purpose

CCLM must preserve the purpose established through CRMM.

Relevant elements include:

Original Problem or Opportunity

Requested Modification

Intended Benefit

Affected Software Objects

Expected Result

Known Constraints

Original Evidence

Material Request Revisions

The original purpose must not be silently rewritten to match whatever result was achieved.

Intended Change Outcome

CCLM must identify the intended outcome against which the result is evaluated.

Relevant dimensions include:

Functional Outcome

Security Outcome

Quality Outcome

Compatibility Outcome

Dependency Outcome

Configuration Outcome

Operational Outcome

Performance Outcome

Data Integrity Outcome

Recovery Outcome

The intended outcome must remain connected to applicable change decisions and requirements.

Applicable Change Decision

CCLM must identify the decision under which the change proceeded or was otherwise disposed of.

Relevant elements include:

Decision Reference

Decision Authority

Approved or Accepted Scope

Applicable Conditions

Restrictions

Required Verification

Required Lifecycle Activities

Decision Validity

Decision Revisions

The existence of a decision does not establish that its conditions were satisfied.

Implementation Reference

Where implementation occurred, CCLM must identify the actual modification.

Relevant elements include:

Implemented Software Artifact

Source Modification

Dependency Change

Configuration Change

Data Migration

Firmware Modification

Implementation Actor

Execution Environment

Implementation Time

Execution Evidence

The implementation reference must remain distinguishable from the original proposal.

Implementation Identity

CCLM must establish which software state or artifact was actually modified.

Relevant elements include:

Software Product ID

Artifact ID

Version ID

Build Reference

Configuration Baseline

Dependency State

Deployment Reference

Affected Environment

Observed State

A reported implementation must not be accepted as sufficient when the resulting software object cannot be reliably identified.

Implementation Completion

Implementation completion concerns whether the intended technical

activities were performed.

Relevant elements include:

Required Modification Performed

Artifact Produced

Configuration Applied

Dependency Updated

Migration Executed

Implementation Evidence Available

Execution Errors

Uncompleted Activities

Implementation completion must remain separate from outcome verification and lifecycle governance completion.

Actual Software State

CCLM must identify the resulting software state relevant to closure.

Potential elements include:

Actual Software Version

Actual Artifact Identity

Actual Configuration

Actual Dependencies

Observed Functionality

Observed Security Conditions

Observed Quality Conditions

Observed Data State

Observed Operational Behavior

Known Restrictions

The actual state must be established through applicable evidence rather than inferred solely from implementation records.

Intended–Actual State Comparison

CCLM must compare the intended change outcome with the actual resulting state.

Relevant comparisons include:

Expected Functionality ↔ Observed Functionality

Target Artifact ↔ Actual Artifact

Target Configuration ↔ Actual Configuration

Expected Dependency State ↔ Actual Dependency State

Expected Security Condition ↔ Verified Security Condition

Expected Performance ↔ Observed Performance

Expected Operational Behavior ↔ Observed Operational Behavior

Material differences must remain identifiable.

Outcome Verification

Outcome verification establishes whether the intended result has been

achieved under applicable conditions.

Relevant elements include:

Verification Objective

Applicable Acceptance Criteria

Verification Method

Verification Environment

Evidence Reference

Observed Result

Known Limitations

Verification Conclusion

CCLM references verification performed under the appropriate lifecycle

mechanisms.

It does not replace their specialized technical procedures.

Outcome Evidence

CCLM must preserve evidence supporting the final outcome classification.

Potential evidence includes:

Test Results

Security Verification

Build and Artifact Records

Configuration Records

Dependency Compatibility Evidence

Release Records

Deployment Evidence

Operational Monitoring

Data Integrity Verification

Recovery Verification

Physical-System Validation

Responsible Function Assessment

Evidence must remain associated with the software state and conditions to which it applies.

Evidence Applicability

Closure evidence must be evaluated for applicability to the resulting software state.

Relevant elements include:

Artifact Identity

Software Version

Configuration

Dependency State

Verification Environment

Operational Conditions

Evidence Time

Verification Scope

Known Limitations

Evidence concerning an earlier software state must not automatically establish the acceptability of a materially modified state.

Verification Coverage

CCLM must evaluate whether the verification evidence covers the material closure conditions.

Relevant elements include:

Intended Outcome Coverage

Affected Software Object Coverage

Required Quality Verification

Required Security Verification

Compatibility Verification

Operational Verification

Recovery Verification

Material Residual Risk Assessment

Insufficient coverage must remain explicit.

Change Success Assessment

A change may be classified as successfully implemented when:

The Intended Modification Was Performed

The Applicable Outcome Criteria Were Satisfied

The Resulting Software State Was Identified

Required Verification Evidence Is Available

Material Lifecycle Obligations Are Satisfied

Applicable Authority Conditions Were Respected

No Unresolved Condition Prevents Successful Classification

Successful classification must be limited to the scope actually supported by evidence.

Partial Change Outcome

A change may achieve only part of its intended result.

Examples include:

Partial Functionality Delivered

One Defect Corrected While Another Remains

Security Exposure Reduced but Not Eliminated

Migration Partially Completed

Compatibility Established for Limited Environments

Operational Improvement Achieved With Restrictions

CCLM must preserve the achieved outcome and the unresolved portion separately.

Failed Change Outcome

A change may fail when:

The Intended Modification Cannot Be Completed

The Expected Outcome Is Not Achieved

Verification Fails

The Resulting State Is Unacceptable

Required Dependencies Cannot Be Satisfied

Operational Consequences Become Unacceptable

Implementation Must Be Reversed or Abandoned

Failure must remain a legitimate, evidence-supported disposition rather than being concealed through administrative closure.

Unintended Consequences

CCLM must identify material unintended consequences discovered during or after implementation.

Potential consequences include:

New Functional Defect

Security Exposure

Dependency Incompatibility

Configuration Conflict

Data Integrity Problem

Performance Degradation

Operational Instability

Recovery Limitation

Unexpected Agent Behavior

Physical-System Deviation

Unintended consequences may require impact reassessment, additional lifecycle processing, or a new change case.

Change Deviation

A change deviation occurs when actual implementation or resulting conditions differ materially from the established change scope or requirements.

Relevant elements include:

Expected Modification

Actual Modification

Deviation Origin

Affected Software Objects

Operational Consequences

Authority Implications

Required Reassessment

Disposition

A deviation must not be treated as acceptable merely because the resulting software appears functional.

Scope Reconciliation

CCLM must reconcile the scope originally established for the change with the actual implementation.

Relevant elements include:

Original Scope

Approved Scope

Implemented Scope

Unimplemented Scope

Additional Modifications

Affected Objects

Scope Deviations

Applicable Decisions

Material unapproved scope expansion must remain visible.

Impact Assessment Reconciliation

CCLM must compare the assessed consequences from CIAM with the observed outcome.

Relevant elements include:

Predicted Direct Impact

Observed Direct Impact

Predicted Indirect Impact

Observed Indirect Impact

Known Uncertainty

Newly Discovered Consequences

Impact Assumptions

Required Reassessment

Material differences may require the impact assessment to be reopened.

Coordination Obligation Reconciliation

CCLM must evaluate relevant obligations established through CHCM.

Potential elements include:

Required Coordination Activities

Participating Change Cases

Dependency Conditions

Compatibility Requirements

Conflict Resolution

Sequence Compliance

Coordination Evidence

Residual Coordination Obligations

Closure of one change must not conceal unresolved conflicts affecting related changes.

Lifecycle Obligation Reconciliation

CCLM must evaluate the obligations established through LRECM.

Relevant elements include:

Required Parent Standards

Required Core Modules

Required Activities

Entry Conditions

Verification Requirements

Evidence Reuse Decisions

Completed Activities

Failed Activities

Deferred Activities

Unresolved Obligations

A change cannot be classified as fully governance-complete when mandatory lifecycle obligations remain unsatisfied.

Requirements Closure

Where requirements were affected, CCLM must preserve relevant SRS conclusions.

Potential elements include:

Applicable Requirements

Requirement Changes

Requirements Traceability

Acceptance Evidence

Outstanding Requirements

Requirements Conflicts

CCLM does not independently replace requirements governance.

Creation and Artifact Closure

Where software creation activities were required, CCLM must preserve relevant SCS and SCFS conclusions.

Potential elements include:

Created Artifact

Creator Attribution

Implementation Traceability

Artifact Identity

Configuration Baseline

Build Evidence

Reproducibility Evidence

Unresolved Artifact Conditions

Technical artifact completion must remain distinguishable from verified change success.

Dependency Closure

Where dependencies were affected, CCLM must preserve relevant SDS conclusions.

Potential elements include:

Dependency Identity

Dependency Version

Suitability Evidence

Compatibility Evidence

Dependency Origin

Continuity Conditions

Residual Dependency Risk

An updated dependency does not automatically establish that every dependent software object remains compatible.

Quality Closure

Where quality governance was required, CCLM must preserve relevant SQS conclusions.

Potential elements include:

Required Tests

Executed Tests

Test Results

Regression Verification

Defect Status

Quality Condition Assessment

Unresolved Quality Findings

Passing a subset of tests must not substitute for satisfaction of the applicable quality requirements.

Security Closure

Where security governance was required, CCLM must preserve relevant SSS conclusions.

Potential elements include:

Security Requirements

Vulnerability Status

Security Testing

Remediation Evidence

Residual Exposure

Security Exceptions

Outstanding Security Obligations

A security change may be implemented while the underlying vulnerability remains unresolved.

Release Closure

Where release governance was required, CCLM must preserve relevant SRLS conclusions.

Potential elements include:

Release Candidate Identity

Release Readiness

Release Authorization

Release Conditions

Release Records

Release Restrictions

Withdrawal Requirements

Release readiness must not be inferred solely from the completion of development activities.

Deployment Closure

Where deployment governance was required, CCLM must preserve relevant SDPS conclusions.

Potential elements include:

Deployment Authorization

Production Version

Deployment State

Operational Monitoring

Deployment Verification

Rollback or Recovery

Unresolved Production Conditions

Successful deployment execution does not automatically establish successful change outcome.

Operational Outcome

CCLM must evaluate operational consequences where they are material to the change purpose.

Relevant elements include:

Service Availability

Functional Behavior

Performance

Resource Consumption

Security Conditions

User Impact

Data Integrity

System Compatibility

Recovery Capability

Operational Restrictions

Operational observations must be associated with the actual software state and relevant environment.

Post-Implementation Observation

Some outcomes cannot be established immediately after implementation.

Examples include:

Intermittent Defect Resolution

Long-Term Performance Improvement

Memory Leak Correction

Reliability Improvement

Security Monitoring Effectiveness

Resource Efficiency

Recovery Stability

Physical-System Behavior Over Repeated Operation

Where observation is necessary, CCLM must preserve the required observation conditions and completion criteria.

Observation Window

A required observation period must have identifiable conditions.

Relevant elements include:

Observation Purpose

Start Condition

Applicable Software State

Monitored Indicators

Required Duration or Event Coverage

Acceptance Criteria

Failure Conditions

Responsible Function

Evidence

An observation window must be justified by the governed change rather than treated as a universal fixed duration.

Outcome Uncertainty

CCLM must preserve uncertainty that materially affects closure conclusions.

Potential categories include:

Insufficient Verification

Unknown Operational Impact

Unverified Dependency Compatibility

Incomplete Monitoring

Unresolved Security Finding

Unknown Data Integrity Condition

Uncertain Recovery Capability

Conflicting Evidence

Unverified Physical-System Consequence

Uncertainty must not be converted into a successful outcome classification without an appropriate evidence basis.

Residual Obligation Identification

CCLM must identify obligations remaining after implementation or another change disposition.

Potential obligations include:

Additional Verification

Security Remediation

Compatibility Correction

Operational Monitoring

Data Migration Completion

Recovery Validation

Documentation Update

External Stakeholder Notification

Related Change Resolution

Exception Review

Each material obligation must remain identifiable.

Residual Obligation Ownership

Every material residual obligation must have an applicable responsibility or control reference.

Relevant elements include:

Obligation ID

Originating Change

Required Action

Responsible Function

Applicable Authority

Due Condition or Review Trigger

Evidence Requirement

Current Status

Escalation Condition

An obligation must not disappear merely because the originating change case is administratively closed.

Residual Obligation Transfer

An unresolved obligation may be transferred to another governed process where legitimate.

Potential destinations include:

New Software Change Case

Existing Related Change Case

Security Remediation Process

Defect Management

Deployment Recovery

Operational Monitoring

Migration Process

Retirement Governance

Transfer must preserve:

Original Obligation

Destination Reference

Receiving Responsibility

Transfer Decision

Evidence

Continuity

Transfer does not itself establish resolution.

Closure Eligibility

CCLM must determine whether the change case is eligible for its proposed closure classification.

Relevant elements include:

Identifiable Change Case

Applicable Disposition

Sufficient Supporting Evidence

Actual Outcome Established Where Relevant

Lifecycle Obligations Reconciled

Coordination Obligations Reconciled

Material Deviations Addressed

Residual Obligations Identified

Applicable Authority Established

Closure Conditions Satisfied

Eligibility depends on the proposed closure category.

Successful implementation and legitimate rejection require different

evidence.

Closure Classification

CCLM recognizes the following principal closure classifications:

Successfully Implemented and Verified

Implemented With Residual Obligations

Partially Implemented

Implementation Failed

Rejected Without Implementation

Withdrawn Without Implementation

Superseded

Cancelled

Deferred — Not Finally Resolved

Transferred to Another Governed Case

Closure Pending Verification

Closure Blocked

Reassessment Required

Outcome Unknown

Outcome Conflicted

A classification must not imply greater completion than the supporting evidence establishes.

Successful Closure

Successful closure requires evidence supporting the intended outcome and

applicable governance completion.

Relevant conditions include:

Intended Outcome Verified

Resulting Software State Identified

Required Lifecycle Activities Satisfied

Material Coordination Obligations Resolved

Applicable Authority Conditions Respected

No Unresolved Condition Invalidates Success

Closure Decision Recorded

Successful closure must remain limited to the verified change scope.

Closure With Residual Obligations

A change may have an established implementation outcome while retaining obligations that do not invalidate the specific closure classification.

Such closure requires:

Explicit Residual Obligation Identification

Applicable Responsibility

Transfer or Continuation Mechanism

Evidence Preservation

Restrictions Where Necessary

Follow-up Conditions

Competent Closure Decision

A case must not be described as fully resolved if material obligations remain unresolved.

Failed Change Closure

A failed change may be administratively closed under an explicit failed disposition.

Relevant conditions include:

Failure Identified

Actual Software State Established

Operational Consequences Assessed

Recovery or Restriction Requirements Identified

Residual Obligations Preserved

Required Follow-up Established

Closure Authority Recorded

Failed closure is not successful change resolution.

Rejected Change Closure

A rejected request may be closed without implementation.

CCLM must preserve:

Original Request

Rejection Decision

Applicable Authority

Rejection Reason

Supporting Evidence

Related Obligations

Final Request Status

Rejection of a proposed solution must not conceal an independently established defect, vulnerability, or operational obligation.

Withdrawn Change Closure

A withdrawn request may receive a final administrative disposition when the withdrawal is legitimate.

Relevant elements include:

Withdrawal Actor

Withdrawal Eligibility

Withdrawal Reason

Withdrawal Time

Implementation Status

Outstanding Obligations

Closure Decision

Withdrawal must not be treated as evidence that the original problem no longer exists.

Superseded Change Closure

A change may be superseded by another change case.

CCLM must preserve:

Original Change ID

Successor Change ID

Supersession Reason

Transferred Evidence

Transferred Obligations

Changed Scope

Applicable Decision

Continuity Reference

Supersession must not erase the history or unresolved consequences of the original change.

Deferred Change Disposition

A deferred change remains unresolved unless a separate legitimate final

disposition applies.

Relevant elements include:

Deferral Reason

Responsible Function

Pending Condition

Review Trigger

Operational Restriction

Residual Risk

Next Governance Action

Current Status

Deferral must not be represented as successful closure.

Closure Authority

CCLM must identify the competent authority or governance function responsible for the closure decision.

Relevant elements include:

Closure Decision Actor

Applicable Authority Basis

Decision Scope

Evidence Reviewed

Applicable Restrictions

Decision Conditions

Decision Time

Decision Record

Closure authority must remain distinct from implementation authority and technical verification responsibility.

Automated Closure

Automated systems may support closure assessment where legitimate criteria and authority conditions are established.

Relevant elements include:

Automation Identity

Applicable Change Case

Closure Criteria

Evidence Sources

Verification Conditions

Permitted Closure Categories

Exception Conditions

Escalation Requirements

Decision Evidence

Automated closure must not be based solely on pipeline completion or a work item status.

Autonomous Agent Closure Boundary

An autonomous agent may report completion, submit evidence, or recommend closure within its assigned scope.

CCLM must preserve:

Agent Identity

Assigned Objective

Permitted Actions

Implementation Evidence

Verification Evidence

Closure Recommendation

Applicable Decision Authority

Escalation Requirements

An agent's ability to execute a change does not automatically establish authority to declare that change successfully closed.

Agent Task Completion ≠ Authorized Change Closure.

Closure Evidence Integrity

CCLM must preserve the integrity and provenance of material closure evidence.

Relevant elements include:

Evidence ID

Evidence Source

Evidence Time

Associated Software State

Collection Method

Integrity Reference

Verification Status

Known Limitations

Decision Association

Closure conclusions must remain reconstructable from their supporting evidence.

Closure Record Continuity

The closure record must preserve the relationship between:

Original Change Request

Impact Assessment

Coordination Case

Lifecycle Re-entry Case

Implementation Record

Verification Evidence

Actual Software State

Residual Obligations

Closure Decision

Final Classification

Historical records must not be silently overwritten when closure is reassessed.

Post-Closure Discovery

New evidence may reveal that an earlier closure conclusion was incomplete or incorrect.

Potential triggers include:

Recurring Defect

New Security Finding

Operational Failure

Unexpected Dependency Consequence

Data Integrity Problem

Invalidated Verification Evidence

Unrecognized Implementation Deviation

Physical-System Incident

New evidence must be assessed for its relevance to the original closure basis.

Closure Reopening

A closure case may require reopening when material new evidence invalidates the previous disposition.

Reopening must preserve:

Original Closure Decision

Original Evidence

New Evidence

Reopening Reason

Affected Software State

Required Reassessment

Responsible Function

Revised Status

Historical Continuity

Reopening must not erase the earlier decision or its original basis.

Closure Finality

Closure finality concerns whether the case has reached a legitimate final

disposition under the applicable governance conditions.

Finality must distinguish:

Administrative Finality

Implementation Finality

Verification Finality

Lifecycle Obligation Finality

Operational Outcome Finality

These conditions may not occur simultaneously.

A final administrative record may remain historically valid as a record of the decision made at that time, even if later evidence requires a new assessment.

Final Record ≠ Immutable Truth About the Resulting Software State.

Closure Governance Classification

CCLM recognizes the following governance states:

Closure Assessment Required

Closure Assessment Active

Implementation Evidence Pending

Outcome Verification Pending

Lifecycle Reconciliation Pending

Coordination Reconciliation Pending

Residual Obligations Identified

Closure Eligible

Successfully Closed

Closed With Residual Obligations

Closed as Failed

Closed as Rejected

Closed as Withdrawn

Closed as Superseded

Deferred

Transferred

Closure Blocked

Reassessment Required

Reopened

Unknown

Conflicted

Each classification must preserve its evidence, scope, applicable decision, and time.

Governance Requirements

A conforming CCLM implementation must satisfy the following requirements. Closure Identity and Context

• Establish a stable closure case identity

• Associate closure with an identifiable change request

• Preserve the original change purpose

• Identify the intended outcome

• Preserve applicable change decisions

• Identify the implemented modification where applicable

• Establish the relevant actual software state

• Preserve closure assessment scope

Outcome and Evidence

• Compare intended and actual outcomes

• Identify applicable acceptance criteria

• Preserve outcome verification evidence

• Evaluate evidence applicability

• Evaluate verification coverage

• Identify material deviations

• Identify unintended consequences

• Preserve material uncertainty

• Distinguish implementation completion from outcome verification

Governance Reconciliation

• Preserve relevant impact assessment references

• Reconcile coordination obligations

• Reconcile lifecycle re-entry obligations

• Identify required specialized verification

• Identify incomplete governance activities

• Preserve applicable authority conditions

• Prevent incomplete governance from being classified as fully satisfied.

Disposition and Obligations

• Establish a justified closure classification

• Distinguish successful, partial, failed, and non-implementation

• Identify residual obligations

• Assign or reference responsibility for residual obligations

• Preserve obligation transfer records

• Establish closure eligibility

• Identify competent closure authority

• Preserve the closure decision and its basis

• Prevent deferral from being represented as successful closure

• Prevent administrative closure from concealing unresolved material dispositions. conditions.

Continuity and Reassessment

• Preserve closure evidence provenance

• Maintain the original change-to-closure traceability chain

• Identify post-closure reassessment triggers

• Support evidence-based reopening

• Preserve historical closure decisions

• Preserve revised closure classifications

• Maintain continuity of transferred obligations

• Distinguish final administrative disposition from continuing

operational reality.

Governance Logic

Change Request → Original Purpose → Applicable Change Decision → Impact

Assessment Reference → Coordination Reference → Lifecycle Re-entry Reference → Implementation Status → Actual Software State → Intended–Actual Comparison → Verification Evidence → Lifecycle Obligation Reconciliation → Coordination Obligation Reconciliation → Deviation Assessment → Residual Obligations → Closure Eligibility → Closure Authority → Closure Decision → Software Change Closure Profile

Where the intended outcome is verified:

Implemented Change → Applicable Verification → Obligations Satisfied →

Successful Closure

Where the outcome is partially achieved:

Implemented Change → Partial Outcome → Unresolved Requirements → Residual

Obligation Assessment → Qualified Disposition

Where implementation fails:

Implementation Failure → Actual State Assessment → Recovery or Restriction

Requirements → Failed Disposition → Follow-up Governance

Where the request is rejected or withdrawn:

Change Request → Legitimate Rejection / Withdrawal → Remaining Obligation

Assessment → Administrative Closure

Where the change is superseded:

Original Change → Successor Change → Evidence and Obligation Transfer →

Superseded Disposition

Where material evidence is insufficient:

Closure Request → Evidence Gap → Verification / Clarification /

Reassessment → Updated Closure Eligibility

Where new evidence emerges after closure:

Historical Closure → New Material Evidence → Closure Basis Reassessment →

Reopen / Preserve / Revise Disposition

CCLM must not equate the completion of a technical task, automated workflow, or administrative ticket with successful software change resolution.

Governance Outputs

The principal output is:

Software Change Closure Profile

Supporting outputs may include:

Change Closure Case Identity

Original Purpose Reference

Intended Outcome Profile

Applicable Change Decision Reference

Implementation Status Record

Actual Software State Profile

Intended–Actual Comparison

Outcome Verification Record

Verification Coverage Assessment

Impact Reconciliation Record

Coordination Obligation Reconciliation

Lifecycle Obligation Reconciliation

Deviation Register

Unintended Consequence Register

Residual Obligation Register

Residual Obligation Transfer Record

Closure Eligibility Assessment

Closure Classification

Closure Authority Reference

Closure Decision Record

Closure Evidence Register

Post-Closure Reassessment Record

Closure Reopening Record

Minimum Implementation Framework

A minimum CCLM implementation must establish eight governance layers. Layer 1 — Change Identity and Purpose

Identify the original change request, intended outcome, applicable

decisions, and relevant software objects. Layer 2 — Implementation and Actual State

Establish what was actually implemented, which software state resulted, and

which activities remain incomplete. Layer 3 — Outcome Verification

Compare the intended and actual results using applicable verification

evidence and acceptance criteria. Layer 4 — Governance Reconciliation

Evaluate the fulfillment of impact-related, coordination-related, and

lifecycle re-entry obligations. Layer 5 — Deviations and Uncertainty

Identify material differences, unintended consequences, evidence gaps, and

unresolved conditions. Layer 6 — Residual Obligation Control

Identify outstanding obligations, applicable responsibilities, transfer

conditions, and required follow-up. Layer 7 — Closure Decision

Establish closure eligibility, competent authority, final disposition, and

supporting evidence. Layer 8 — Continuity and Reassessment

Preserve the Software Change Closure Profile, historical evidence,

obligation continuity, and conditions for reopening.

The implementation may use manual governance records, issue-tracking systems, delivery platforms, automated verification, or AI-assisted assessment.

The methodological requirements remain independent of the technology used.

Cross-Module Boundary

CRMM → CCLM

CRMM establishes the original request, purpose, scope, and initial

disposition.

CCLM preserves those references and evaluates the final disposition of the

change case.

CIAM → CCLM

CIAM identifies expected consequences, uncertainty, and verification

implications.

CCLM compares relevant assessed consequences with the observed result and

identifies material differences.

CHCM → CCLM

CHCM establishes coordination requirements, conflicts, dependencies, and

residual coordination obligations.

CCLM reconciles their status as part of closure eligibility.

LRECM → CCLM

LRECM establishes required lifecycle activities, evidence, and outstanding

obligations.

CCLM evaluates whether those obligations have been satisfied, legitimately

excepted, or explicitly transferred.

CCLM does not replace the other SCMS modules. It establishes the final evidence-supported disposition of the change case using their governance outputs.

Cross-Architecture Boundary

CCLM evaluates software change closure using applicable SLGA™ governance

evidence.

SRS — Software Requirements Standard

Requirements satisfaction, change control, and traceability.

SCS — Software Creation Standard

Software creation, artifact attribution, and development traceability.

SDS — Software Dependency Standard

Dependency suitability, compatibility, and continuity.

SCFS — Software Configuration Standard

Configuration baselines, build evidence, artifact identity, and

reproducibility.

SQS — Software Quality Standard

Testing, defects, and quality condition assessment.

SSS — Software Security Standard

Security verification, vulnerabilities, and remediation.

SRLS — Software Release Standard

Release readiness, authorization, records, and withdrawal.

SDPS — Software Deployment Standard

Deployment state, production version, monitoring, rollback, and recovery.

SRTS — Software Retirement Standard

Migration, residual dependencies, and retirement closure.

SLCS — Software Lifecycle Control Standard

Lifecycle state consistency, transitions, exceptions, control transfer, and

continuity.

CCLM may also reference applicable OOF® authority, accountability, autonomous systems, and operational reality governance mechanisms where the resulting software change affects those domains.

Practical Applications

Enterprise Software Maintenance

Establishing evidence-supported closure of defects, enhancements,

configuration changes, and dependency updates.

Continuous Integration and Delivery

Preventing automated pipeline completion from being treated as proof of

successful software change resolution.

Security Remediation

Distinguishing completed security patches from verified remediation of the

underlying vulnerability.

AI-Assisted Software Engineering

Evaluating AI-generated modifications against their intended outcomes,

verification evidence, and applicable lifecycle obligations.

Autonomous Software Operations

Preventing autonomous agents from declaring successful closure merely

because assigned execution tasks have completed.

Critical Infrastructure

Preserving evidence of operational acceptability, unresolved restrictions,

and recovery obligations after consequential software changes.

Robotics and Embedded Systems

Establishing whether firmware and control logic modifications achieved

their intended results without creating unacceptable physical-system consequences.

• Use Case 1 — A Security Patch Is Deployed but the Vulnerability Remains

Exploitable

A software team receives a change request to correct a critical

authorization vulnerability.

A patch is developed, tested, and deployed.

The delivery pipeline reports successful completion.

The change ticket is automatically marked as resolved.

However, a subsequent security verification identifies another execution

path through which the original vulnerability remains exploitable.

CCLM evaluates:

Original Security Objective → Implemented Patch → Deployment Evidence →

Security Verification → Actual Security Condition → Closure Eligibility

The assessment identifies:

• The intended technical modification was implemented

• The deployment completed successfully

• The original security objective was not fully achieved

• The remaining vulnerability represents an unresolved material

• Successful closure is not supported by the available evidence

• Additional remediation and lifecycle reassessment are required

condition.

The appropriate classification is:

Implementation Completed — Security Outcome Not Verified — Closure

Blocked.

Successful Deployment ≠ Successful Security Remediation.

Use Case 2 — An Autonomous Agent Reports a Completed Fix While Leaving

a Data Integrity Obligation

An autonomous software maintenance agent is assigned to correct a data-processing defect.

The agent modifies the processing logic and runs the permitted automated

tests.

The tests pass, and the agent reports that the change is complete.

The corrected software processes new records successfully.

However, records previously processed under the defective logic remain

inconsistent.

CCLM evaluates:

Original Change Purpose → Agent Modification → New Processing Results →

Historical Data Condition → Outcome Verification → Residual Obligations

The assessment identifies:

• The software modification was implemented

• The new processing behavior satisfies the relevant functional tests

• Historical data remains affected by the original defect

• The original change objective may not be fully satisfied if correction

• A data remediation obligation must be established and governed

• The agent's task-completion report does not independently authorize

of existing records was part of the required outcome. successful change closure.

The appropriate classification is:

Implementation Verified — Residual Data Integrity Obligation Outstanding.

A qualified closure or transfer may be considered only if the remaining obligation is explicitly identified, legitimately assigned, and does not invalidate the applicable disposition.

Agent Task Completion ≠ Verified Resolution of All Change Consequences.

Critical Distinctions

Implementation Complete ≠ Intended Outcome Verified.

Technical Task Completed ≠ Change Governance Completed.

Deployment Successful ≠ Change Successfully Resolved.

Original Change Purpose ≠ Actual Implementation Result.

Approved Scope ≠ Implemented Scope.

Verification Evidence Available ≠ Verification Coverage Sufficient.

Historical Evidence ≠ Automatically Applicable Evidence.

Partial Success ≠ Full Resolution.

Failed Change Closure ≠ Successful Change Resolution.

Request Rejected ≠ Underlying Problem Resolved.

Request Withdrawn ≠ Underlying Risk Eliminated.

Change Superseded ≠ Original Obligations Erased.

Obligation Transferred ≠ Obligation Resolved.

Residual Obligation Identified ≠ Residual Obligation Satisfied.

Administrative Closure ≠ Operational Finality.

Agent Task Completion ≠ Authorized Change Closure.

Final Record ≠ Immutable Truth About the Resulting Software State.

Canonical Boundary

CCLM governs the evidence-supported evaluation, classification,

disposition, and formal closure of identifiable software change cases.

It establishes the relationship between the original change purpose,

applicable decisions, actual implementation, resulting software state, verified outcome, lifecycle obligations, coordination conditions, residual responsibilities, and legitimate closure authority.

CCLM does not replace specialized software verification, authorize deployment, execute remediation, or eliminate obligations merely through administrative status changes.

Its principal governance output is the:

Software Change Closure Profile

This output preserves the final governance disposition of the change case

and the evidence necessary to distinguish verified success, partial realization, failure, rejection, withdrawal, supersession, deferral, and continuing obligations.

Canonical Closing Statement

A software change is not successfully resolved merely because the modification was completed, the deployment succeeded, or the work item was closed.

A patch may be deployed while the vulnerability remains exploitable.

A defect may be corrected for new transactions while historical data remains inconsistent.

A dependency update may pass local tests while introducing unresolved compatibility problems.

A production change may function correctly while required recovery verification remains incomplete.

An autonomous agent may report successful task completion without establishing the legitimacy or consequences of every action it performed.

And a change may be legitimately rejected or withdrawn without resolving the original condition that caused it to be proposed.

CCLM establishes the methodological structure required to distinguish these

outcomes.

It requires the intended change to remain connected to the actual

implementation and resulting software state.

It preserves the evidence needed to determine whether the original purpose

was achieved.

It reconciles impact, coordination, and lifecycle obligations before

establishing closure eligibility.

It distinguishes successful resolution from partial completion, failed

implementation, and administrative disposition.

It ensures that unresolved obligations retain identifiable responsibility and continuity.

And it preserves the ability to reassess an earlier closure when material new evidence challenges the conclusion on which that closure was based.

The purpose of software change closure is not to make a change case disappear from an active workflow. It is to establish, with evidence, what happened to the change, what outcome was actually achieved, which obligations were satisfied, and what remains governed after the case is closed.

Change Closure Management Module (CCLM) transforms software change closure

from an administrative endpoint into an evidence-supported determination of outcome, responsibility, and lifecycle finality.